MC23 – Algoritmos

Resumo detalhado: AV1 e conteúdo pós-AV1 (AV2)
(IMPA Tech, 2026.2 – Prof. Uéverton Souza)

Baseado nos slides, aulas manuscritas, monitorias, listas, pesquisas, Testes 1 e 2; complementado com Szwarcfiter & Markenzon (*Estruturas de Dados e seus Algoritmos*) e Cormen et al. (*Introduction to Algorithms*, 3. ed.)

Atualizado em 23 de outubro de 2026 (aulas até 20/10)
AV1: terça-feira, 22 de setembro de 2026

Como usar este resumo

- Ele foi escrito para quem **já estudou a matéria** e quer se localizar rápido: cada seção começa com o essencial, depois vêm as demonstrações e os detalhes que costumam ser cobrados.
- **Parte I** (seções 1–10): conteúdo da AV1, sem alterações. **Parte II** (seções 11–15): material liberado depois da AV1 — programação dinâmica, buscas em grafos, caminhos mínimos, gulosos, matroides e AGM.
- **Caixas verdes**: como o tema cai na prova (na Parte I, o formato anunciado para a AV1 — ver abaixo; na Parte II, receitas e tipos de questão prováveis, pois o formato da AV2 ainda não foi anunciado).
- **Caixas vermelhas**: armadilhas e erros frequentes (inclusive erros encontrados em slides/-monitorias).
- **Caixas azuis**: resumo do que precisa estar na ponta da língua.
- **Caixas laranjas**: complementos (livros/pesquisas) que não foram o foco das aulas.
- Convenção de heap do curso (Szwarcfiter): **heap de máximo, vetor indexado a partir de 1**, pai de i é $\lfloor i/2 \rfloor$, filhos $2i$ e $2i+1$. Balanço AVL: $\text{bal}(v) = h_D(v) - h_E(v)$.
- Todos os exemplos numéricos (AVL, heaps, radix, subvetor máximo, inversões; na Parte II, LCS, cadeia de matrizes, mochila, buscas, Dijkstra, Floyd-Warshall, Kruskal e Prim) foram *conferidos por simulação*.

Formato da AV1 (anunciado na aula de revisão de 18/09)

Questão	O que será pedido	Onde estudar
Q1	**Árvore AVL**: saber como a estrutura funciona, resolver uma tarefa nela, manipular (inserir/remover, rotações, balanços).	Seção 5
Q2	**Union-Find** (a mais fácil): diferentes formas de implementar e em que situações cada uma é melhor.	Seção 4 (Tabela 1)
Q3	**Heap**: saber *demonstrar* propriedades de heap.	§2.7 (provas); depois §2, §3
Q4	Dado um código genérico: qual **estrutura de dados** usar e qual **ordenação** usar.	Seções 3.4, 6.4, 6.5
Q5	**Divisão e conquista**: dado um problema, escrever o **pseudocódigo**.	Seção 8

Sumário

Mapa: aulas e materiais → seções **5**

Cola: complexidades de tudo **6**

I Conteúdo da AV1 **7**

1 Complexidade de algoritmos **7**
1.1 Medidas de complexidade . . . 7
1.2 Notações assintóticas . . . 7
1.3 Complexidade de problemas × de algoritmos . . . 7
1.4 Recorrências . . . 8
1.5 Complexidade amortizada . . . 8

2 Listas de prioridades e heaps binários **9**
2.1 Lista de prioridades . . . 9
2.2 Definição e representação . . . 9
2.3 Subir e descer . . . 10
2.4 Inserção, remoção, alteração . . . 10
2.5 Construção em tempo linear . . . 11
2.6 Heapsort . . . 12
2.7 Propriedades de heap e suas demonstrações . . . 12

3 Heaps avançados: binomiais e de Fibonacci **13**
3.1 Árvores binomiais . . . 14
3.2 Heap binomial . . . 14
3.2.1 Operações . . . 14
3.3 Heap de Fibonacci . . . 15
3.3.1 Por que funciona: grau máximo $O(\log n)$ . . . 16
3.3.2 Análise amortizada . . . 16
3.4 Comparação e escolha da estrutura (Q4 e Teste 1) . . . 17

4 Conjuntos disjuntos: Union-Find **18**
4.1 O problema . . . 18
4.2 Implementações . . . 18

4.3 Rank + compressão: $O(m \log^* n)$ (demonstração da aula de 25/08) . . . 20
4.4 Qual implementação usar? (Q2 da AV1) . . . 20
4.5 Exercícios resolvidos (monitoria de revisão e lista) . . . 22

5 Árvores binárias de busca e árvores AVL 22
5.1 Árvore binária de busca (ABB) . . . 22
5.2 Árvores AVL (Adel'son-Vel'skii e Landis, 1962) . . . 23
5.3 Inserção: atualização dos balanços . . . 23
5.4 As rotações . . . 24
5.5 Pseudocódigo (slides do professor) . . . 24
5.6 Remoção em AVL . . . 26
5.7 Exemplos resolvidos (conferidos por simulação) . . . 26
5.8 Exercícios da lista (Union-Find e AVL) . . . 28
5.9 Árvores rubro-negras (pesquisa) . . . 29

6 Ordenação 30
6.1 Ordenação por comparação . . . 30
6.2 Limite inferior $\Omega(n \log n)$ para ordenação por comparação . . . 31
6.3 Ordenação em tempo linear . . . 31
6.4 Qual algoritmo de ordenação usar? . . . 33
6.5 Q4: dado um código, qual estrutura e qual ordenação? . . . 33

7 Seleção: o algoritmo BFPRT (mediana das medianas) 34
7.1 O algoritmo . . . 34
7.2 Análise . . . 35

8 Divisão e conquista 35
8.1 Contagem de inversões (aula de 11/09) . . . 36
8.2 Subvetor contíguo de soma máxima (Teste 2, Exercício 1) . . . 36
8.3 Multiplicação de matrizes: Strassen (Teste 2, Exercício 2) . . . 37
8.4 Outros exemplos clássicos de D&C (treino para a Q5) . . . 38

9 Complementos 39
9.1 Tabelas hash (pesquisa) . . . 39

10 Banco de questões com respostas 40
10.1 Testes aplicados . . . 40
10.2 Lista – heaps binários, binomiais e de Fibonacci . . . 40
10.3 Lista – Union-Find e AVL . . . 41
10.4 Monitoria de revisão: respostas curtas . . . 41

II Conteúdo pós-AV1 (AV2) 43

11 Programação dinâmica 43
11.1 Subsequência comum máxima – LCS (aula 14, 29/09; Cormen §15.4) . . . 44
11.2 Multiplicação de cadeia de matrizes (aula 15, 02/10; Cormen §15.2) . . . 45
11.3 Mochila 0-1 (monitoria de PD) . . . 46
11.4 Lista de PD resolvida . . . 47

12 Grafos: representação e buscas 48
12.1 Busca em profundidade em grafos (aula 19, 16/10; slides 1–122; Cormen §22.3) . . . 48
12.2 Busca em profundidade em digrafos (aula 20, 20/10; slides 123–170) . . . 50
12.2.1 Ordenação topológica (slides 208–273) . . . 51

12.3 Busca em largura (aula 18, 13/10; slides 274–312; Cormen §22.2) 51

13 Caminhos mínimos: Dijkstra e Floyd-Warshall **53**
13.1 Algoritmo de Dijkstra (aula 18, 13/10; slides 313–341; Cormen §24.3) 53
13.2 Floyd-Warshall: PD em grafos (aula 16, 06/10; slides 342–405; Cormen §25.2) 54

14 Algoritmos gulosos, matroides e árvores geradoras mínimas **55**
14.1 Método guloso (slides de Matroides) 55
14.2 Matroides (slides de Matroides; pesquisa) 56
14.3 Árvore geradora mínima (AGM) 58
14.3.1 Algoritmo de Kruskal (aula 17, 09/10; slides 406–449; Cormen §23.2) 58
14.3.2 Algoritmo de Prim (aula 17; slides 450–461 e slide de AGM) 59
14.4 Lista – Árvores geradoras e métodos gulosos (resolvida) 59

15 Banco de questões pós-AV1 e checklist **60**
15.1 Listas extras: respostas e ideias 61
15.2 Como estudar a parte nova 61

Mapa: aulas e materiais → seções

Aula	Data	Conteúdo	Seção
1	11/08	Complexidade, notações assintóticas, limites superior/inferior, polinomial × exponencial	1
2	12/08	Listas de prioridades e heaps (binário), heapsort	2
3	18/08	Heaps binários (resumo de custos) e heaps binomiais	2, 3
4	21/08	Heaps de Fibonacci e complexidade amortizada	1.5, 3
5	25/08	Union-Find: listas, florestas, rank, compressão, análise $O(m \log^* n)$	4
6	28/08	Árvores binárias de busca e AVL	5
7	01/09	Ordenação por comparação (heap/merge/quick), limite inferior $\Omega(n \log n)$	6.1
8	04/09	Ordenação linear: counting, radix, bucket (e American Flag)	6.3
9	08/09	Seleção: algoritmo BFPRT (mediana das medianas)	7
10	11/09	Divisão e conquista; contagem de inversões	8
11	15/09	**Teste 2** (ordenação e D&C)	10.1
12–13	16 e 18/09	Exercícios; revisão (AVL, D&C); formato da AV1	—
14	29/09	PD: subsequência comum máxima (Cormen §15.4)	11, 11.1
15	02/10	PD: multiplicação de cadeia de matrizes (Cormen §15.2)	11.2
16	06/10	PD em grafos: Floyd-Warshall (slides 342–405 de grafos; Cormen §25.2)	13.2
17	09/10	AGM: Prim e Kruskal (slides 406–461; Cormen §23.2)	14.3
18	13/10	Busca em largura; Dijkstra (slides 274–342; Cormen §22.2, §24.3)	12.3, 13.1
19	16/10	Busca em profundidade (slides 1–122; Cormen §22.3)	12.1
20	20/10	Busca em profundidade em digrafos; componentes fortemente conexas (slides 123–170)	12.2
Monitorias		Heaps binomiais/Fibonacci (26/08), Union-Find, AVL (aula prática 4), Revisão Heap/UF/AVL	3, 4, 5, 10
Monitorias		Programação dinâmica (cadeia de matrizes, mochila); LCS com reconstrução	11.2, 11.3, 11.1
Pesquisas	até 19/09	Árvores rubro-negras; Tabelas hash	5.9, 9.1
Pesquisa		Matroides, algoritmos gulosos e interseção de matroides	14.2
Slides		Matroides e algoritmos gulosos (não listados no registro de aulas)	14.1, 14.2
Listas		Heaps (binário/binomial/Fibonacci); Union-Find e AVL	10
Listas		Programação dinâmica; Árvores geradoras e métodos gulosos	11.4, 14.4
Outros		Listas extras: APA (UFF), MAC5711 lista 6 (USP), MC458 lista 7 (Unicamp)	15

Obs.: a Parte II cobre o material liberado depois da AV1. Os slides de grafos vão além do que foi dado até 20/10: a ordenação topológica (slides 208–273) já está incluída (§12.2.1); emparelhamentos e fluxos em redes (slides 464–609) ainda não.

Cola: complexidades de tudo

Listas de prioridades (n elementos; * = amortizado; heap de máximo)

Operação	Lista não ord.	Lista ordenada	Heap binário	Heap binomial	Heap de Fibonacci
Seleção (máx.)	$\Theta(n)$	$\Theta(1)$	$O(1)$	$O(\log n)$ ou $O(1)$ c/ ponteiro	$O(1)$
Inserção	$\Theta(1)$	$\Theta(n)$	$O(\log n)$	$O(\log n)$; $O(1)$ esperado/*	$O(1)^*$
Remoção do máx.	$\Theta(n)$	$\Theta(1)$	$O(\log n)$	$O(\log n)$	$O(\log n)^*$
Aumentar prioridade	$\Theta(n)$	$\Theta(n)$	$O(\log n)$	$O(\log n)$	$O(1)^*$
Diminuir prioridade	$\Theta(n)$	$\Theta(n)$	$O(\log n)$	$O(\log n)$	$O(\log n)^*$
Construção	$\Theta(n)$	$\Theta(n \log n)$	$O(n)$	$O(n)$	$O(n)$
União (fusão)	$O(1)$ encad.	$O(n)$	$O(n)$	$O(\log n)$	$O(1)^*$

Union-Find (n elementos, m operações)

Implementação	Make-Set	Find	Union
Listas encadeadas c/ ponteiro ao representante (união pelo tamanho)	$O(1)$	$O(1)$	$O(\log n)^*$
Floresta ingênua	$O(1)$	$O(n)$	$O(1)$ (dadas as raízes)
Floresta + união por tamanho/rank	$O(1)$	$O(\log n)$	$O(1)$ (dadas as raízes)
Floresta + rank + compressão de caminhos	$O(1)$	$O(m \log^* n)$ no total (justo: $O(m\,\alpha(m,n))$)	

Árvores de busca ABB: busca/inserção/remoção $\Theta(h)$, com h entre $\lfloor \log n \rfloor + 1$ e n. AVL: tudo $O(\log n)$ no pior caso; inserção faz ≤ 1 rotação (simples ou dupla); remoção pode fazer $O(\log n)$ rotações.

Ordenação

Algoritmo	Pior caso	Caso médio	Melhor caso	Memória extra	Estável?	In-place?
Inserção / Bolha	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n)$	$O(1)$	sim	sim
Mergesort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n)$	sim	não
Heapsort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	$O(1)$	não	sim
Quicksort	$\Theta(n^2)$	$\Theta(n \log n)$	$\Theta(n \log n)$	$O(\log n)$ pilha (média)	não	sim
Counting sort	$\Theta(n + k)$	$\Theta(n + k)$	$\Theta(n + k)$	$\Theta(n + k)$	sim	não
Radix sort (LSD)	$\Theta(d(n + b))$	idem	idem	$\Theta(n + b)$	sim	não
Bucket sort	$\Theta(n \log n)$ ou $\Theta(n^2)$	$\Theta(n)$ (unif.)	$\Theta(n)$	$\Theta(n)$	depende	não

Outros (Parte I) Limite inferior p/ ordenação por comparação: $\Omega(n \log n)$. Seleção (k-ésimo menor): BFPRT $O(n)$ pior caso; quickselect $O(n)$ esperado, $O(n^2)$ pior. Contagem de inversões e subvetor de soma máxima (D&C): $O(n \log n)$. Strassen: $O(n^{\log_2 7}) \approx O(n^{2,807})$.

Programação dinâmica LCS $\Theta(nm)$. Cadeia de matrizes $\Theta(n^3)$ tempo e $\Theta(n^2)$ memória. Mochila 0-1 $\Theta(nW)$, pseudo-polinomial (memória $O(W)$ com um vetor). Maior subsequência crescente $O(n^2)$ ou $O(n \log n)$. Floyd-Warshall $\Theta(n^3)$.

Grafos (n vértices, m arestas)

Algoritmo	Complexidade	Observação
BFS, DFS e aplicações (CFC, topológica, articulações)	$O(n + m)$ listas; $O(n^2)$ matriz	BFS: distância em nº de arestas
Dijkstra (uma origem)	$O(n^2)$ vetor; $O((n + m) \log n)$ heap; $O(m + n \log n)$ Fibonacci	só pesos ≥ 0
Floyd-Warshall (todos os pares)	$\Theta(n^3)$ tempo, $\Theta(n^2)$ memória	pesos negativos ok, sem ciclo negativo
Kruskal	$O(m \log n)$ (a ordenação domina)	Union-Find testa ciclo
Prim	$O(n^2)$ matriz/vetor; $O(m \log n)$ heap; $O(m + n \log n)$ Fibonacci	heap só compensa se esparso
Guloso em matroide	$O(n \log n + n\,h)$	h = custo do oráculo

Parte I
Conteúdo da AV1

1 Complexidade de algoritmos

1.1 Medidas de complexidade

Seja A um algoritmo e $\{E_1, \ldots, E_m\}$ todas as entradas possíveis de tamanho n; T_i é o número de passos de A sobre E_i.

- **Pior caso:** $\max_i T_i$ (é a medida padrão do curso).
- **Caso médio:** $\sum_i p_i T_i$, onde p_i é a probabilidade de E_i.
- **Melhor caso:** $\min_i T_i$.
- **Amortizada:** custo de uma *sequência* de operações dividido pelo número de operações, no pior caso da sequência (§1.5). Não é média probabilística!

Tamanho da entrada. É o comprimento da codificação da entrada (em bits/símbolos). Isso importa com números: um algoritmo é **pseudopolinomial** se é polinomial quando os números da entrada são codificados em *unário*. Exemplo do curso: o *counting sort* é $O(n+k)$; se $k = \Theta(2^n)$ ele é exponencial no tamanho da entrada (um número k ocupa só $\log k$ bits).

1.2 Notações assintóticas

Sejam f, g funções (assintoticamente) não negativas.

Notação	Definição	Via limite (se existir)	Leitura
$f = O(g)$	$\exists c > 0, n_0 :\ f(n) \le c\,g(n)\ \forall n \ge n_0$	$\lim f/g < \infty$	"$\le$"
$f = \Omega(g)$	$\exists c > 0, n_0 :\ f(n) \ge c\,g(n)\ \forall n \ge n_0$	$\lim f/g > 0$	"$\ge$"
$f = \Theta(g)$	$f = O(g)$ e $f = \Omega(g)$	$\lim f/g = c \in (0, \infty)$	"$=$"
$f = o(g)$	$\forall c > 0\ \exists n_0 :\ f(n) < c\,g(n)$	$\lim f/g = 0$	"$<$" (não justo)
$f = \omega(g)$	$\forall c > 0\ \exists n_0 :\ f(n) > c\,g(n)$	$\lim f/g = \infty$	"$>$" (não justo)

Exemplo dos slides: $2n^2 + 1$ é $O(n^3)$, $O(n^2)$, não é $O(n)$; é $\Omega(n)$, $\Omega(n^2)$, não é $\Omega(n^3)$; é $\Theta(n^2)$. $O(g)$ também denota o *conjunto* das funções f com $f = O(g)$ (escreve-se $f \in O(g)$).

Propriedades úteis. Transitividade; $f = O(g) \iff g = \Omega(f)$; $O(f) + O(g) = O(\max\{f, g\})$; $\log_a n = \Theta(\log_b n)$ (a base do log não importa); $\log(n!) = \Theta(n \log n)$; $(\log n)^c = o(n^\varepsilon)$ e $n^c = o(a^n)$ para $a > 1$.

$$1 \ <\ \log\log n \ <\ \log n \ <\ \sqrt{n} \ <\ n \ <\ n\log n \ <\ n^2 \ <\ n^3 \ <\ n^{\log n} \ <\ 2^n \ <\ n! \ <\ n^n.$$

1.3 Complexidade de problemas × de algoritmos

Definição 1.1.

- Um **limite superior** para o problema Π é uma função $f(n)$ tal que *existe* algum algoritmo que resolve Π com no máximo $f(n)$ passos no pior caso. (A complexidade de pior caso de qualquer algoritmo para Π é um limite superior para Π.)
- Um **limite inferior** para Π é uma função $f(n)$ tal que *todo* algoritmo para Π precisa de pelo menos $f(n)$ passos no pior caso.
- Um algoritmo é **ótimo** se sua complexidade de pior caso é igual a um limite inferior do problema (ex.: mergesort e heapsort para ordenação por comparação).

As notações são só sobre funções: nada impede dizer "o pior caso de A é $\Omega(n^3)$" ou "o limite inferior de Π é $O(\log n)$".

Polinomial × exponencial. Consenso: um algoritmo é útil na prática se é **polinomial** ($O(n^c)$; classe P = problemas com algoritmo polinomial). Exponenciais: $2^{O(n)}$, k^n, $n!$, 2^{n^2}, $n^{\log n}$. Polinomiais aproveitam melhor o avanço tecnológico: com máquina $10\times$ mais rápida, o maior n resolvido em um dia é *multiplicado* por uma constante (n^2: $\times 3{,}16$), enquanto para 2^n só *soma* uma constante ($40 \to 43$).

1.4 Recorrências

Algoritmos recursivos (D&C) têm tempo dado por recorrências $T(n) = a\,T(n/b) + f(n)$: a subproblemas de tamanho n/b e custo $f(n)$ para dividir e combinar.

Árvore de recursão. Some o custo por nível. Ex.: $T(n) = 2T(n/2) + cn$: há $\log_2 n$ níveis, cada um custando $cn \Rightarrow T(n) = \Theta(n \log n)$ (mergesort, contagem de inversões, subvetor máximo).

Substituição (indução). Chuta-se $T(n) \le c\,g(n)$ e prova-se por indução forte (é assim que o slide prova $T(n) \le 20cn$ na BFPRT, §7).

Teorema 1.2 (Teorema Mestre – Cormen §4.5)**.** *Seja $T(n) = a\,T(n/b) + f(n)$ com $a \ge 1$, $b > 1$, e seja $E = \log_b a$ (o "expoente crítico").*

1. *Se $f(n) = O(n^{E-\varepsilon})$ para algum $\varepsilon > 0$: $T(n) = \Theta(n^E)$ (as folhas dominam).*
2. *Se $f(n) = \Theta(n^E)$: $T(n) = \Theta(n^E \log n)$ (todos os níveis pesam igual).*
3. *Se $f(n) = \Omega(n^{E+\varepsilon})$ e $af(n/b) \le cf(n)$ para algum $c < 1$: $T(n) = \Theta(f(n))$ (a raiz domina).*

Recorrência	Exemplo	Solução
$T(n) = T(n/2) + O(1)$	busca binária	$\Theta(\log n)$
$T(n) = 2T(n/2) + O(1)$	máximo por D&C	$\Theta(n)$
$T(n) = 2T(n/2) + \Theta(n)$	mergesort, inversões, subvetor máximo	$\Theta(n \log n)$
$T(n) = 8T(n/2) + \Theta(n^2)$	produto de matrizes por blocos	$\Theta(n^3)$
$T(n) = 7T(n/2) + \Theta(n^2)$	Strassen	$\Theta(n^{\log_2 7}) \approx \Theta(n^{2{,}807})$
$T(n) = T(n/5) + T(3n/4) + \Theta(n)$	BFPRT	$\Theta(n)$ (pois $\frac{1}{5} + \frac{3}{4} < 1$)
$T(n) = T(n-1) + \Theta(n)$	quicksort no pior caso	$\Theta(n^2)$
$T(n) = T(9n/10) + T(n/10) + \Theta(n)$	quicksort "desbalanceado fixo"	$\Theta(n \log n)$

Teorema 1.3 (D&C c-balanceado – slide de D&C)**.** *Se um algoritmo divide a instância em k partes de tamanho no máximo n/c ($c > 1$), com partes de tamanhos que diferem por constante, e gasta tempo linear para dividir e combinar, então roda em $O(n \log n)$.*

Ideia. Na árvore de recursão, os subproblemas de um mesmo nível são disjuntos, então cada nível custa $O(n)$; como o tamanho cai por um fator c a cada nível, há $O(\log_c n)$ níveis. □

Somatórios que aparecem. $\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$; $\sum_{k\ge 0} x^k = \frac{1}{1-x}$ ($|x| < 1$); $\sum_{k\ge 0} \frac{k}{2^k} = 2$; $\sum_{k=0}^{\log n} \frac{n}{2^k} \le 2n$; $\sum_{i=1}^{n} \frac{1}{i} = \Theta(\log n)$; $\log(n!) = \sum \log i = \Theta(n \log n)$.

1.5 Complexidade amortizada

A análise amortizada considera uma **sequência** de operações: cada operação individual pode ser cara, mas o custo total da sequência (no pior caso) é pequeno. **Não envolve probabilidade.**

Visão do banqueiro (créditos). Cada operação recebe um número fixo de créditos (seu custo amortizado). Operações baratas "poupam" créditos, que pagam operações caras depois. Se o saldo nunca fica negativo, o custo real total $\leq$ soma dos custos amortizados.

Visão do físico (potencial). Escolhe-se $\Phi(E) \geq 0$ para cada configuração E. Custo amortizado: $a_i = t_i + \Phi(E_i) - \Phi(E_{i-1})$. Somando (telescópica):

$$\sum_{i=1}^{m} t_i = \sum_{i=1}^{m} a_i - \Phi_m + \Phi_0 \;\leq\; \sum_{i=1}^{m} a_i \quad \text{se } \Phi_m \geq \Phi_0.$$

Exemplo 1.4 (Pilha com multi-desempilhamento – Szwarcfiter §8.2)**.** Cada operação faz $k \geq 0$ *retira* e depois um *insere*; no pior caso uma operação custa $\Theta(n)$. Com Φ = número de elementos na pilha: $a_i = (1+k) + (p-k+1) - p = 2$. Logo m operações custam $\leq 2m = O(m)$.

Exemplo 1.5 (Contador binário)**.** Incrementar um contador de k bits pode trocar k bits, mas m incrementos a partir de 0 trocam no total $\leq 2m$ bits: com Φ = número de bits 1, um incremento que zera t bits e liga um tem $a = (t+1) + (1-t) = 2$. É exatamente o argumento da **inserção em heap binomial** (§3.1): inserir = somar 1 em binário.

Pontos-chave

"Amortizado" $\neq$ "caso médio". Amortizado é *garantia de pior caso para a sequência*; uma operação isolada ainda pode ser cara. Isso é decisivo na escolha entre heap binomial e de Fibonacci quando o sistema exige **previsibilidade de cada operação individual** (Teste 1, Q1).

2 Listas de prioridades e heaps binários

2.1 Lista de prioridades

Tabela em que cada dado tem uma **prioridade**; operações: (1) *seleção* do de maior prioridade; (2) *inserção*; (3) *remoção* do de maior prioridade; (4) *alteração* de prioridade; (5) *construção* a partir de um conjunto inicial. (Fusão entra com os heaps avançados.)

Operação	Lista **não** ordenada	Lista ordenada	Heap
seleção	$\Theta(n)$	$\Theta(1)$	$O(1)$
inserção	$\Theta(1)$	$\Theta(n)$	$O(\log n)$
remoção	$\Theta(n)$	$\Theta(1)$	$O(\log n)$
alteração	$\Theta(n)$	$\Theta(n)$	$O(\log n)$
construção	$\Theta(n)$	$\Theta(n \log n)$	$O(n)$

As duas listas têm alguma operação de atualização $\Omega(n)$; o heap equilibra tudo em $O(\log n)$.

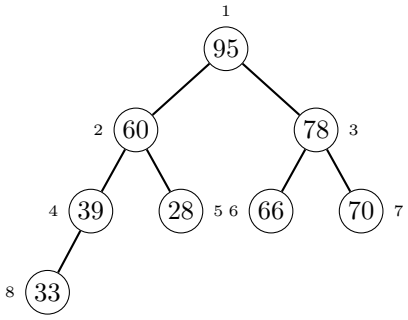
2.2 Definição e representação

Definição 2.1 (Heap – convenção do curso)**.** Um **heap** (de máximo) é uma lista linear $s_1, \ldots, s_n$ tal que

$$s_i \leq s_{\lfloor i/2 \rfloor} \qquad \text{para todo } 1 < i \leq n.$$

(Heap de *mínimo*: $s_i \geq s_{\lfloor i/2 \rfloor}$.)

Visualiza-se como uma **árvore binária completa** numerada por níveis, da esquerda para a direita: o nó i tem pai $\lfloor i/2 \rfloor$, filho esquerdo $2i$ e direito $2i+1$ (se $\leq n$). A condição diz: *cada nó tem prioridade $\geq$ à dos filhos.* Não há ponteiros: é só um vetor.



1	2	3	4	5	6	7	8
95	60	78	39	28	66	70	33

Fatos de índice (usados em todas as provas):

- o nó i está no nível $\lfloor \log_2 i \rfloor + 1$ (raiz no nível 1); a árvore tem $\lfloor \log_2 n \rfloor + 1$ níveis;
- as **folhas** são exatamente as posições $\lfloor n/2 \rfloor + 1, \ldots, n$ (pois i tem filho sse $2i \leq n$); há $\lceil n/2 \rceil$ folhas;
- convenção de altura do Szwarcfiter/slides: **folhas têm altura 1**.

2.3 Subir e descer

subir(i) – **após *aumentar* a prioridade de** i

```
Procedimento SUBIR(i)
    j := ⌊i/2⌋
    se j ≥ 1 então
        se T[i].chave > T[j].chave então
            T[i] ⇔ T[j]
            SUBIR(j)
```

descer(i, n) – **após *diminuir* a prioridade de** i

```
Procedimento DESCER(i, n)
    j := 2i
    se j ≤ n então
        se j < n e T[j + 1].chave > T[j].chave
então
            j := j + 1                          % maior filho
        se T[i].chave < T[j].chave então
            T[i] ⇔ T[j]
            DESCER(j, n)
```

Ambos percorrem um caminho da árvore: $O(\log n)$. **Descer escolhe o maior filho** (se trocasse com o menor, o maior filho ficaria abaixo de um valor menor).

2.4 Inserção, remoção, alteração

Inserção, remoção do máximo e alteração de prioridade

```
Procedimento INSERIR(novo)                      % M = capacidade
    se n < M então
        T[n + 1] := novo; n := n + 1; SUBIR(n)
    senão overflow
Procedimento REMOVER
    se n ≠ 0 então
        agir(T[1]); T[1] := T[n]; n := n − 1; DESCER(1, n)
    senão underflow
Procedimento ALTERAR(i, p)
    antiga := T[i].chave; T[i].chave := p
    se p > antiga então
        SUBIR(i)                                % aumentou ⇒ sobe
    senão
        DESCER(i, n)                            % diminuiu ⇒ desce
```

Erro clássico (monitoria de revisão, Q5)

Um aluno escreveu: "$H[i] := p$; **se** $H[i] > H[\lfloor i/2 \rfloor]$ **então** Descer(i) **senão** Subir(i)". Está **invertido**: se o novo valor ficou maior que o pai, ele viola a propriedade *com o pai* e precisa **subir**; se não ficou maior que o pai, a única violação possível é com os filhos, e ele precisa **descer**. Versão correta: **se** $H[i] > H[\lfloor i/2 \rfloor]$ (com $i > 1$) **então** Subir(i) **senão** Descer(i, n).

Exemplo 2.2 (Traços conferidos – heap $H = [95, 60, 78, 39, 28, 66, 70, 33]$)**.**

- **Aumentar** o nó 6 de 66 para 98: sobe trocando com 78 (pos. 3) e com 95 (pos. 1): $[98, 60, 95, 39, 28, 78, 70, 33]$.
- **Remover a raiz**: coloca-se o último (33) na raiz, $[33, 60, 78, 39, 28, 66, 70]$; desce trocando com 78 (maior filho) e depois com 70: $[78, 60, 70, 39, 28, 66, 33]$.
- **Diminuir** a raiz de 95 para 37: desce trocando com 78 e com 70: $[78, 60, 70, 39, 28, 66, 37, 33]$.
- **Inserir** 73: entra na pos. 9 (filho de 39), sobe trocando com 39 e com 60: $[95, 73, 78, 60, 28, 66, 70, 33, 39]$.

2.5 Construção em tempo linear

1ª tentativa (inserções sucessivas): **para** $i := 2, \ldots, n$ **faça** subir(i) — custa $O(n \log n)$ (e $\Theta(n \log n)$ no pior caso, p.ex. entrada crescente).

2ª tentativa (*arranjar*): as folhas ($i > \lfloor n/2 \rfloor$) já são heaps; corrige-se de baixo para cima.

arranjar(n)

```
Procedimento ARRANJAR(n)
    para i := ⌊n/2⌋ decr. até 1 faça
        DESCER(i, n)
```

Correção: invariante "antes de processar i, as subárvores de raízes $i+1, \ldots, n$ são heaps". Ao chamar descer(i, n), as duas subárvores dos filhos de i já são heaps, e descer transforma a subárvore de i em heap.

Lema 2.3 (Lista de heaps, ex. 6)**.** *Numa árvore binária completa com n nós (folhas com altura 1), no máximo $\lceil n/2^j \rceil$ nós têm altura j.*

Demonstração. Numa árvore completa o caminho mais longo a partir de i é o da esquerda: $i \to 2i \to 4i \to \cdots$. Logo o nó i tem altura $\geq j$ sse seu descendente $j-1$ níveis abaixo existe, isto é, sse $i\,2^{j-1} \leq n$, ou seja, $i \leq \lfloor n/2^{j-1} \rfloor$. Assim, com $x = n/2^{j-1}$ e $m = \lfloor x \rfloor$,

$$\#\{\text{altura} = j\} = \lfloor x \rfloor - \lfloor x/2 \rfloor = m - \lfloor m/2 \rfloor = \lceil m/2 \rceil \leq \lceil x/2 \rceil = \lceil n/2^j \rceil. \qquad \square$$

Teorema 2.4 (Lema 6.1 do Szwarcfiter / slide)**.** *arranjar(n) roda em $O(n)$.*

Demonstração. descer(i, n) num nó de altura j faz no máximo $j-1$ trocas. Pelo Lema 2.3, o total é

$$\sum_{j=2}^{h} (j-1) \left\lceil \frac{n}{2^j} \right\rceil \;\leq\; \sum_{j \geq 2} (j-1)\left(\frac{n}{2^j} + 1\right) \;=\; n \sum_{j \geq 2} \frac{j-1}{2^j} + O(\log^2 n) \;=\; n \cdot 1 + O(\log^2 n) = O(n),$$

pois $\sum_{j \geq 1} \frac{j-1}{2^j} = \sum_{j \geq 1} \frac{j}{2^j} - \sum_{j \geq 1} \frac{1}{2^j} = 2 - 1 = 1$. (O slide faz a mesma conta separando j par e ímpar e usando $\frac{j}{2^{j+1}} \leq \frac{j-1}{2^j}$.) $\square$

Intuição: metade dos nós (folhas) não custa nada, 1/4 custa ≤ 1, 1/8 custa ≤ 2, …; só a raiz custa $\log n$.

Exemplo 2.5 (Traços do *arranjar* (conferidos))**.**

- Lista de heaps, ex. 5: $[18, 25, 41, 34, 14, 10, 52, 50, 48]$ ($n = 9$, processa $i = 4, 3, 2, 1$).
  $i = 4$: $34 \leftrightarrow 50$; $i = 3$: $41 \leftrightarrow 52$; $i = 2$: $25 \leftrightarrow 50$, depois $25 \leftrightarrow 48$; $i = 1$: $18 \leftrightarrow 52$, depois $18 \leftrightarrow 41$.
  Resultado: $[52, 50, 41, 48, 14, 10, 18, 34, 25]$.
- Monitoria (Q4): $[20, 15, 30, 40, 10, 25, 5]$ processa $i = 3, 2, 1$: $i = 3$ nada; $i = 2$: $15 \leftrightarrow 40$; $i = 1$: $20 \leftrightarrow 40$. Resultado $[40, 20, 30, 15, 10, 25, 5]$.
- Monitoria (Q13): $[7, 2, 15, 9, 1, 20, 13]$: $i = 3$: $15 \leftrightarrow 20$; $i = 2$: $2 \leftrightarrow 9$; $i = 1$: $7 \leftrightarrow 20$, $7 \leftrightarrow 15$. Resultado $[20, 9, 15, 2, 1, 7, 13]$ (o mesmo que por inserções sucessivas neste caso).

2.6 Heapsort

Heapsort (Szwarcfiter, Alg. 7.6)

Procedimento HEAPSORT(T, n)
  ARRANJAR(n) % $O(n)$
  para $m := n$ **decrescendo até** 2 **faça**
    $T[1] \Leftrightarrow T[m]$ % o máximo vai para o fim
    DESCER$(1, m-1)$ % $O(\log m)$

Complexidade $O(n) + \sum_m O(\log m) = \Theta(n \log n)$ em *todos* os casos (mesmo com a entrada ordenada). **In-place** (só troca no vetor) e **não estável** (a troca da raiz com o último embaralha iguais). É ótimo entre os algoritmos por comparação. Ex.: $[18, 25, 41, 34, 14, 10, 52, 50, 48] \rightarrow [10, 14, 18, 25, 34, 41, 48, 50, 52]$.

2.7 Propriedades de heap e suas demonstrações

Q3 da AV1: "saber demonstrar propriedades de heap"

Quase toda prova de heap usa só três ferramentas: (i) a definição $s_i \leq s_{\lfloor i/2 \rfloor}$ + **transitividade ao longo de um caminho**; (ii) os **fatos de índice** (pai, filhos, folhas, níveis); (iii) **indução** (na altura, no nível, ou no número de nós). Para "provar ou dar contraexemplo", teste primeiro exemplos pequenos ($n \leq 8$).

Proposição 2.6 (Ancestral domina)**.** *Se j é ancestral de i num heap, então $s_i \leq s_j$. Em particular: todo caminho da raiz a uma folha é não crescente, a* ***raiz é o máximo*** *e* ***cada subárvore é um heap****.*

Demonstração. O caminho de i até j é $i, \lfloor i/2 \rfloor, \lfloor i/4 \rfloor, \ldots, j$; aplicando a definição em cada aresta e a transitividade de $\leq$, $s_i \leq s_{\lfloor i/2 \rfloor} \leq \cdots \leq s_j$. A raiz é ancestral de todos. A condição do heap numa subárvore é a mesma condição restrita a ela. □

Proposição 2.7 (O mínimo está numa folha)**.** *Num heap de máximo com chaves distintas, o menor elemento está numa folha, isto é, numa posição $> \lfloor n/2 \rfloor$.*

Demonstração. Se i não é folha, tem filho $2i$ e $s_{2i} < s_i$ (distintas); logo s_i não é o mínimo. (Com repetições: *algum* mínimo está numa folha.) □

Proposição 2.8 (Onde está o k-ésimo maior)**.** *Com chaves distintas, o k-ésimo maior elemento está num nível $\leq k$, isto é, numa posição $\leq 2^k - 1$. Em particular o 2° maior é filho da raiz (posição 2 ou 3), e o 3° maior está nas posições $2, \ldots, 7$.*

Demonstração. Pela Prop. 2.6, todos os ancestrais de x são maiores que x. Se x é o k-ésimo maior, só existem $k-1$ elementos maiores, então x tem no máximo $k-1$ ancestrais, ou seja, está no nível $\leq k$. As posições dos níveis $1..k$ são $1, \ldots, 2^k - 1$. □

Proposição 2.9 (Altura)**.** *Um heap com n elementos tem altura $\lfloor \log_2 n \rfloor + 1$ (em níveis) $= \Theta(\log n)$; por isso subir/descer/inserir/remover são $O(\log n)$.*

Demonstração. O último nó, n, está no nível $\lfloor \log_2 n \rfloor + 1$, e numa árvore completa todos os níveis anteriores estão cheios: $2^{h-1} \le n \le 2^h - 1$, logo $h = \lfloor \log_2 n \rfloor + 1$. □

Proposição 2.10 (Relação com ordenação)**.** *Toda lista ordenada de forma não crescente é um heap; a recíproca é falsa (ex.:* $[3, 1, 2]$*). Por isso construir um heap é mais fácil que ordenar (*$O(n)$ *vs.* $\Omega(n \log n)$*) — e, como heapsort ordena usando construção +* n *remoções, não existe heap com construção* e *remoção ambas* $o(\log n)$ *amortizado* por comparação *(senão ordenaríamos em* $o(n \log n)$*).*

Proposição 2.11 (Troca de dois elementos – Lista de heaps, ex. 2 e 3)**.** *Seja* S *um heap e* $i < j$*.*

(a) Se $s_i < s_j$*, trocar* s_i *com* s_j ***não*** *produz necessariamente um heap.*

(b) Se $s_i > s_j$*, trocar* s_i *com* s_j ***não*** *produz necessariamente um heap.*

Contraexemplos (verificados). (a) $S = [10, 9, 1, 8, 7, 0, 0, 6]$ é heap; $i = 3$ (1), $j = 4$ (8). Após a troca, $[10, 9, 8, 1, 7, 0, 0, 6]$: a posição 8 (6) tem pai na posição 4 (1) e $6 > 1$. Intuição: o valor pequeno vai para a posição j, cujos filhos podem ser maiores que ele. (b) $S = [10, 9]$, $i = 1$, $j = 2$: após a troca $[9, 10]$, filho maior que o pai. Intuição: o valor pequeno sobe para i e pode ficar abaixo de um descendente maior. □

Proposição 2.12 (Número de heaps – Lista de heaps, ex. 4)**.** *Seja* $H(n)$ *o número de permutações de* $1, \ldots, n$ *que são heaps (vale o mesmo para min-heap). Então* $H(0) = H(1) = 1$ *e*

$$H(n) = \binom{n-1}{L} H(L)\, H(n-1-L),$$

onde L *é o número de nós da subárvore esquerda da raiz: com* $h = \lfloor \log_2 n \rfloor$ *e* $u = n - (2^h - 1)$ *nós no último nível,* $L = (2^{h-1} - 1) + \min\{u, 2^{h-1}\}$*. Valores:* $1, 1, 2, 3, 8, 20, 80, 210, 896, 3360$ *para* $n = 1..10$ *(conferido por força bruta até* $n = 8$*).*

Demonstração. A raiz é necessariamente n (máximo). O formato da árvore é fixo, então L é determinado por n. Escolhem-se quais L dos $n - 1$ valores restantes vão para a esquerda ($\binom{n-1}{L}$ modos); cada lado precisa ser um heap sobre seus valores, e o número de heaps só depende da quantidade de valores ($H(L)$ e $H(n-1-L)$). □

Proposição 2.13 (Correção de subir e descer)**.** *(a) Se* T *era heap e a chave de* i aumentou*, após subir*(i) T *é heap. (b) Se as subárvores dos filhos de* i *são heaps (em particular se* T *era heap e a chave de* i diminuiu*), após descer*(i, n) *a subárvore de* i *é heap.*

Esboço. (a) Invariante: a única violação possível é entre o nó corrente e seu pai. Ao trocar com o pai j (quando $s_i > s_j$), o antigo s_j desce para i e é $\ge$ aos filhos de i (eram seus descendentes); o novo valor em j é maior que o antigo, então não viola com os filhos de j. Repete-se para cima. (b) Invariante: a única violação é entre o nó corrente e seus filhos. Troca-se com o *maior* filho: ele passa a dominar o outro filho e o valor que desceu. A violação desce um nível; termina numa folha ou quando não há violação. □

Pontos-chave

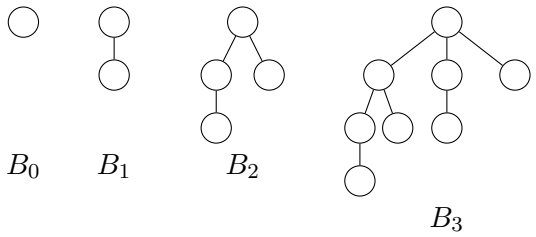
Heap = vetor + "pai $\ge$ filho". Subir após aumento, descer após diminuição (maior filho!). Inserir = pôr no fim + subir; remover = último na raiz + descer. Construção linear pelo *arranjar* (de $\lfloor n/2 \rfloor$ até 1) graças a $\le \lceil n/2^j \rceil$ nós de altura j. Heapsort $\Theta(n \log n)$, in-place, não estável.

3 Heaps avançados: binomiais e de Fibonacci

Motivação. No heap binário (vetor) a **fusão** (união de duas filas) custa $O(n)$: concatena-se os vetores e reconstrói-se com *arranjar*. Em aplicações com muitas fusões (setores de controle, Dijkstra/Prim em grafos grandes) isso é inviável. O heap binomial faz fusão em $O(\log n)$ pior caso; o de Fibonacci, em $O(1)$ amortizado.

3.1 Árvores binomiais

Definição 3.1. B_0 é um único nó. Para $k \geq 1$, B_k é obtida **subordinando** (*link*) uma B_{k-1} à outra: a raiz de uma vira filho (mais à direita/novo primeiro filho) da raiz da outra. Num heap, a raiz de *maior* prioridade fica por cima.



Proposição 3.2 (Propriedades de B_k – provas por indução em k)**.**

1. *B_k tem exatamente 2^k nós* *($2 \cdot 2^{k-1}$).*
2. *O grau da raiz (a **ordem** de B_k) é k, e é o grau máximo da árvore* *(o link dá mais um filho à raiz).*
3. *A altura de B_k é k (medida em arestas)* *(a subordinada fica um nível abaixo:* $\max\{k-1, 1+(k-1)\}$*).*
4. *Há exatamente $\binom{k}{j}$ nós na profundidade j* *($\binom{k-1}{j} + \binom{k-1}{j-1} = \binom{k}{j}$; daí o nome).*
5. *Os filhos da raiz de B_k são raízes de $B_{k-1}, B_{k-2}, \ldots, B_1, B_0$.*

Convenção de altura: aula × livro

Na aula (18/08) e na monitoria "Heaps Binomiais": B_k **tem altura** k (conta arestas). No Szwarcfiter e na monitoria "Heap-fibonacci-binomial": B_h **tem altura** $h+1$ (conta níveis, folha tem altura 1, como nos heaps binários do curso). É a mesma árvore! Ao responder exercícios de "alturas" (lista, ex. 8 e 10), *declare a convenção.* Neste resumo: ordem = grau da raiz = k e altura = k arestas.

3.2 Heap binomial

Definição 3.3. Um **heap binomial** é uma floresta H de árvores binomiais tal que (1) cada árvore satisfaz a condição de heap (filho $\leq$ pai) e (2) as árvores têm **ordens distintas** (no máximo uma de cada ordem). As raízes ficam numa lista ordenada por ordem crescente.

Proposição 3.4. *(a) Para cada n há um único formato de heap binomial com n nós: $B_k \in H$ sse o bit k de n é 1. (b) Logo H tem no máximo $\lfloor \log_2 n \rfloor + 1$ árvores, cada uma de altura $\leq \lfloor \log_2 n \rfloor$.*

Demonstração. B_k contribui 2^k nós e as ordens são distintas, então $n = \sum_{B_k \in H} 2^k$ — é a representação binária de n, que é única. Um número n tem $\lfloor \log_2 n \rfloor + 1$ bits. □

Ex.: $13 = 1101_2 \Rightarrow H = \{B_0, B_2, B_3\}$; $26 = 11010_2 \Rightarrow \{B_1, B_3, B_4\}$.

3.2.1 Operações

Binomial-Link e fusão (Szwarcfiter Alg. 9 / monitoria)

Procedimento BINOMIAL-LINK(y, z) % y e z raízes de B_{k-1}; z fica por cima
  $p[y] \leftarrow z$; $sibling[y] \leftarrow child[z]$; $child[z] \leftarrow y$; $degree[z] \leftarrow degree[z] + 1$
Procedimento FUSÃO(H', H'')
  $H := H' \cup H''$ % intercala as listas de raízes por ordem crescente: $O(\log n)$
  enquanto H contém duas árvores B, B' de mesma ordem **faça** % a menor ordem repetida primeiro
    se $B.raiz.chave \geq B'.raiz.chave$ **então**

```
        subordinar B' a B
    senão
        subordinar B a B'
```

A fusão é exatamente **uma soma binária com "vai um"**: duas B_k viram uma B_{k+1} (o "vai um"). Como há $O(\log n)$ posições e no máximo 3 árvores de mesma ordem ao mesmo tempo (duas originais + um carry), a fusão custa $O(\log n)$ no pior caso. (Na implementação do Cormen, percorre-se a lista com ponteiros *prev-x*, *x*, *next-x* e há 4 casos: ordens diferentes; três iguais seguidas; dois iguais com x vencendo; dois iguais com *next-x* vencendo.)

Exemplo 3.5 (Fusão – Szwarcfiter Fig. 9.10 e exercício da monitoria)**.** $H_1 = \{B_0, B_2, B_4\}$ ($21 = 10101_2$) e $H_2 = \{B_0, B_1\}$ ($3 = 11_2$). Links na ordem: $B_0 + B_0 \to B_1$; esse $B_1 + B_1$ (de H_2) $\to B_2$; esse $B_2 + B_2$ (de H_1) $\to B_3$. Resultado $\{B_3, B_4\}$, isto é, $24 = 11000_2 = 10101_2 + 11_2$.

- **Inserção:** fusão com um heap $\{B_0\}$ contendo só o novo nó. Pior caso $O(\log n)$ (quando $n = 2^k - 1$, todos os bits são 1 e há k links). *Tempo esperado* $O(1)$ (Lema 9.3 do Szwarcfiter: se j é a posição do primeiro 0 de n em binário, lida da direita para a esquerda, a inserção faz j "fusões" no sentido do livro, das quais $j - 1$ são links, um por 1 final de n; com bits equiprováveis, $E[j] = \sum_{j \geq 1} j/2^j = 2$, isto é, 2 passos esperados e 1 link esperado) e *amortizado* $O(1)$ (dá-se 1 crédito a cada árvore; cada link é pago pelo crédito da árvore subordinada, que some — é o contador binário).
- **Seleção do máximo:** o máximo é uma das $\leq \lfloor \log n \rfloor + 1$ raízes: $O(\log n)$, ou $O(1)$ mantendo um ponteiro para a maior raiz (atualizado nas operações).
- **Remoção do máximo:** ache a raiz r máxima, numa árvore B_k; retire B_k de H ($H' = H - B_k$); retirando r, seus filhos $B_{k-1}, \ldots, B_0$ formam um heap binomial H''; faça a fusão $H' \cup H''$. Total $O(\log n)$.
- **Alteração de prioridade:** *aumentar* $\Rightarrow$ sobe dentro da própria árvore trocando com o pai, como no heap binário: $O(\log n)$ (altura $\leq \log n$). *Diminuir* $\Rightarrow$ remover o nó (ver abaixo) e reinseri-lo com a nova prioridade: $O(\log n)$. (Descer trocando com o maior filho também funciona, mas cada nível tem até $O(\log n)$ filhos para comparar: $O(\log^2 n)$.)
- **Remoção de um nó qualquer:** aumente sua prioridade para $+\infty$ (sobe até a raiz) e remova o máximo: $O(\log n)$.
- **Construção:** n inserções custam $O(n)$ no total (contador binário: $\sum_k n/2^k \leq 2n$). É a resposta do ex. 7 da lista.

3.3 Heap de Fibonacci

Ideia: preguiça. O heap binomial é "estrito": toda inserção/fusão já resolve colisões de ordem. O de Fibonacci *adia* toda a arrumação para a remoção do máximo.

Estrutura. Floresta de árvores com condição de heap; as raízes numa **lista circular duplamente encadeada**, e os filhos de cada nó também. Ponteiro para a raiz máxima. Cada nó guarda: chave, pai, um filho, irmãos esquerdo/direito, grau, e um bit **marca**. Listas circulares duplas permitem inserir/remover um nó ou concatenar duas listas em $O(1)$.

Operações (heap de máximo, como no curso; "aumentar prioridade" = *decrease-key* do Cormen, que usa heap de mínimo).

- **Inserção:** cria uma árvore de um nó e a coloca na lista de raízes; atualiza o ponteiro do máximo. $O(1)$ real.
- **Fusão:** concatena as duas listas de raízes; máximo = maior dos dois. $O(1)$ real.
- **Seleção:** ponteiro. $O(1)$.

- **Remoção do máximo:** (1) remove a raiz máxima e promove todos os seus filhos a raízes (desmarcados); (2) **consolida**: percorre as raízes com um vetor $A[0..D]$ indexado pelo grau ($D = O(\log n)$); sempre que duas raízes têm o mesmo grau, subordina a de menor prioridade à de maior (o grau sobe de 1) e repete; (3) reconstrói a lista de raízes a partir de A e acha o novo máximo. Real: $O(D + t)$ com t = número de raízes; **amortizado** $O(\log n)$.
- **Aumento de prioridade de x:** aumenta a chave; se não ficou maior que o pai, fim. Senão: **corta** x (com sua subárvore) e o põe na lista de raízes, desmarcado; então, subindo: se o pai y está *desmarcado*, marque-o (se não for raiz) e pare; se y já estava *marcado* (perdeu um filho antes), corte y também, desmarque-o, leve-o à lista de raízes e continue com o pai de y — **cortes em cascata**. **Amortizado** $O(1)$.
- **Remoção de nó qualquer:** aumenta para $+\infty$ e remove o máximo: $O(\log n)^*$.
- **Diminuição de prioridade:** não é a operação "barata"; faz-se removendo e reinserindo (ou cortando os filhos que passarem a violar): $O(\log n)^*$. Na tabela da aula (convenção de mínimo), isso aparece como "incrementar $O(\log n)^*$, decrementar $O(1)^*$".

Regras das marcas. Todos começam desmarcados; um nó é marcado quando *perde o primeiro filho* por corte; ao perder o segundo, ele próprio é cortado. **Raízes nunca estão marcadas** (e podem perder vários filhos); ao ser subordinado a outro nó, o nó fica desmarcado.

Exemplo 3.6 (Cortes em cascata – exercício da monitoria de 26/08)**.** Nó interno X com pai Y (marcado) e avô Z (marcado); a prioridade de X aumenta violando a ordem. (1) X é cortado e vai para a lista de raízes (desmarcado). (2) Y perdeu seu segundo filho: é cortado, desmarcado, vai para a lista de raízes. (3) Idem para Z. (4) Seja W o pai de Z: se W é raiz, nada muda; se não é raiz e está desmarcado, passa a **marcado** e a cascata para; se estava marcado, a cascata continua. Nós movidos para as raízes: X, Y, Z (e possivelmente mais); flags: X, Y, Z desmarcados, W marcado (se não raiz).

3.3.1 Por que funciona: grau máximo $O(\log n)$

Lema 3.7. *Seja x um nó e $y_1, \ldots, y_k$ seus filhos na ordem em que foram ligados a x. Então* $\text{grau}(y_i) \geq i - 2$.

Demonstração. Quando y_i foi ligado, x já tinha $y_1, \ldots, y_{i-1}$, logo grau $\geq i - 1$; só se ligam nós de mesmo grau, então y_i tinha grau $\geq i - 1$. Desde então y_i perdeu no máximo um filho (senão teria sido cortado). Logo $\text{grau}(y_i) \geq i - 2$. □

Teorema 3.8. *Um nó de grau k tem subárvore com pelo menos $F_{k+2} \geq \varphi^k$ nós (F = Fibonacci, $\varphi = \frac{1+\sqrt{5}}{2}$). Logo o grau máximo é $D(n) \leq \log_\varphi n = O(\log n)$, e um heap de Fibonacci tem $O(\log n)$ árvores após a consolidação.*

Esboço. Seja s_k o tamanho mínimo de uma subárvore de raiz com grau k. Pelo lema, $s_k \geq 2 + \sum_{i=2}^{k} s_{i-2}$, e por indução $s_k \geq F_{k+2}$ (pois $F_{k+2} = 1 + \sum_{i=0}^{k} F_i$). É daí que vem o nome: as menores árvores de posto k ($F_0, \ldots, F_5$ desenhadas em aula) têm $1, 2, 3, 5, 8, 13$ nós. □

Sem marcas/cascata, poderíamos cortar todos os netos de uma raiz: uma árvore de posto k com só $k + 1$ nós; o posto deixaria de ser $O(\log n)$ e a consolidação ficaria cara (resposta ao "Exercício 3" da monitoria).

3.3.2 Análise amortizada

Potencial (Cormen, cap. 19): $\Phi(H) = t(H) + 2\,m(H)$, com t = nº de raízes e m = nº de nós marcados.

- Inserção: real $O(1)$, $\Delta\Phi = +1 \Rightarrow O(1)$. Fusão: $\Delta\Phi = 0 \Rightarrow O(1)$.

- Remoção do máximo: real $O(D+t)$; depois há $\leq D+1$ raízes, então $\Delta\Phi \leq (D+1)-t \Rightarrow$ amortizado $O(D) = O(\log n)$. *O custo de consolidar as t raízes acumuladas foi "pago" pelas inserções/fusões preguiçosas.*
- Aumento de prioridade com c cortes em cascata: real $O(c)$; t cresce c, m cai $\geq c-2 \Rightarrow \Delta\Phi \leq c+2(2-c) = 4-c \Rightarrow$ amortizado $O(1)$. (Visão do banqueiro do Szwarcfiter: o corte de um nó marcado é debitado à operação que o marcou.)

Teorema 3.9 (Aula de 21/08)**.** *A partir de um heap de Fibonacci vazio, qualquer sequência de a_1 inserções, a_2 remoções, a_3 aumentos de prioridade e a_4 uniões custa $O(a_1 + a_3 + a_4 + a_2 \log n)$.*

Armadilhas comuns

- Os $O(1)$ do Fibonacci são **amortizados**: *uma* remoção do máximo após muitas inserções/-fusões pode custar $\Theta(n)$ (consolida tudo de uma vez). Se o requisito é previsibilidade de cada operação, Fibonacci é ruim.
- Constantes grandes e muita memória por nó (4 ponteiros + grau + marca): na prática só compensa com MUITAS alterações de prioridade (Dijkstra/Prim em grafos densos).
- Uma árvore de um heap de Fibonacci *pode* ter altura grande (até $\Theta(n)$ em sequências adversárias; Cormen, ex. 19.4-1): o que é logarítmico é o **grau**, não a altura.

3.4 Comparação e escolha da estrutura (Q4 e Teste 1)

	Heap binário	Heap binomial	Heap de Fibonacci
Seleção	$O(1)$	$O(\log n)$ / $O(1)$ c/ ponteiro	$O(1)$
Inserção	$O(\log n)$	$O(\log n)$ pior; $O(1)$ esp./amort.	$O(1)^*$
Remoção do máx.	$O(\log n)$	$O(\log n)$	$O(\log n)^*$
Aumentar prioridade	$O(\log n)$	$O(\log n)$	$O(1)^*$
Fusão	$O(n)$	$O(\log n)$ **pior caso**	$O(1)^*$
Construção	$O(n)$	$O(n)$	$O(n)$
Memória	vetor, sem ponteiros	3 ponteiros + grau/nó	4 ponteiros + grau + marca/nó
Garantia	pior caso	pior caso	amortizada
Simplicidade	muito simples	média	complexa

Roteiro de escolha

1. Precisa de **fusão**? Não ⇒ **heap binário** (mais simples, menos memória, ótimo em cache).
2. Fusão frequente e **cada operação precisa ser previsível** (limite de tempo por operação, sistema de tempo real) ⇒ **heap binomial** (fusão $O(\log n)$ *pior caso*).
3. Muitíssimos **aumentos de prioridade** (ou fusões) e o que importa é o **custo total** da execução ⇒ **heap de Fibonacci**.
4. Restrição de **memória** forte ⇒ binário (as outras têm overhead de ponteiros por nó).

Teste 1 – respostas esperadas.

- **Q1 (aeroporto):** fusões muito frequentes com filas de centenas de milhares de aeronaves; alterações raras; exigência de tempo *previsível por operação individual.* ⇒ **Heap binomial**: fusão em $O(\log n)$ no pior caso de *cada* execução, crescendo só logaritmicamente. Binário: fusão $O(n)$ (reconstrução) — milhares de fusões com 10^5 elementos estouram a janela. Fibonacci: fusão $O(1)$ só *amortizada*; o custo acumulado aparece de uma vez numa remoção (consolidação

$\Theta(n)$ possível) — imprevisível; e sua vantagem (aumento de prioridade $O(1)$) não é usada, pois alterações são raras.

- **Q2 (central médica, 40 MB):** $2 \cdot 10^6$ chamadas $\times$ 16 bytes = 32 000 000 bytes $\approx$ 30,5 MB $<$ 41 943 040. Heap binomial precisaria de ≥ 3 ponteiros (pai, filho, irmão = 24 bytes) + grau: ≥ 40 bytes/nó $\Rightarrow \geq 80 \cdot 10^6$ bytes; Fibonacci (≥ 4 ponteiros + grau + marca) ainda mais. Não há fusão, alterações são poucas, inserção/remoção $O(\log n)$ bastam. $\Rightarrow$ **Heap binário em vetor** (único que cabe, e o mais simples).
- **Q3 (supercomputador):** $5 \cdot 10^6$ inserções, $5 \cdot 10^6$ remoções, $80 \cdot 10^6$ aumentos de urgência, poucas uniões; importa o custo total. Fibonacci: $\approx 5 \cdot 10^6 + 80 \cdot 10^6 + 5 \cdot 10^6 \log_2(5 \cdot 10^6) \approx 2 \cdot 10^8$ passos; binário/binomial: $\approx 90 \cdot 10^6 \cdot 22 \approx 2 \cdot 10^9$. $\Rightarrow$ **Heap de Fibonacci** (aumento $O(1)^*$ sem remover/reinserir). Binário/binomial pagam $O(\log n)$ em cada um dos 80 milhões de aumentos.

4 Conjuntos disjuntos: Union-Find

4.1 O problema

Manter uma partição de um universo de n elementos em conjuntos disjuntos, cada um identificado por um **representante**, com:

- MAKE-SET(x): cria o conjunto $\{x\}$ (Szwarcfiter: *gerar*);
- FIND(x): devolve o representante do conjunto de x (*buscar*); "x e y estão juntos?" $\iff$ FIND(x) = FIND(y);
- UNION(x, y): une os conjuntos de x e y (*fundir*).

Conjuntos **nunca se separam** (não há "split"). Uma sequência típica: n MAKE-SET e m FIND/UNION intercalados; no máximo $n - 1$ uniões efetivas. O número de conjuntos = número de raízes, que nunca aumenta. **Aplicações:** componentes conexas dinâmicas, algoritmo de Kruskal (AGM), equivalência de símbolos em compiladores, grupos em redes sociais, percolação.

4.2 Implementações

(a) Vetor de representantes ("quick-find")

$rep[x]$ guarda o representante. FIND= $O(1)$; UNION precisa trocar rep de todos os elementos de um conjunto: $O(n)$ por união.

(b) Listas encadeadas com ponteiro direto ao representante

Cada conjunto é uma lista; um objeto-conjunto guarda *head*, *tail* e *size*; todo nó tem ponteiro para o objeto-conjunto (o representante).

- MAKE-SET: $O(1)$. FIND: $O(1)$ (segue um ponteiro).
- UNION: concatena as listas em $O(1)$ (tail de uma $\rightarrow$ head da outra) *mas* precisa atualizar o ponteiro ao representante de todos os nós da lista anexada.
- Sem cuidado: $\Theta(n)$ por união, $\Theta(n^2)$ numa sequência ruim (sempre anexar a lista grande à unitária).
- **União ponderada** (anexa a *menor* à maior): cada vez que o ponteiro de um elemento x é atualizado, o conjunto de x pelo menos **dobra**; como tamanhos $\leq n$, x é atualizado $\leq \log_2 n$ vezes. Total de n MAKE-SET e m operações: $O(m + n \log n)$, isto é, UNION em $O(\log n)$ **amortizado**.
- Vantagem exclusiva: **listar os membros** de um conjunto em $O(k)$ (k = tamanho do conjunto), percorrendo a lista.

(c) Floresta ("quick-union")

Cada conjunto é uma árvore; cada nó aponta para o pai ($pai[x]$); a raiz é o representante ($pai[r] = r$ ou 0). MAKE-SET: $pai[x] := x$. FIND: sobe até a raiz, $O(h)$. UNION: acha as duas raízes e faz uma apontar para a outra, $O(1)$ *dadas as raízes*. Problema: sem critério, uma sequência de uniões gera uma **cadeia de altura** $n-1$ (Find $\Theta(n)$).

(d) União por tamanho ou por rank

Liga-se a raiz da árvore *menor* (em tamanho, ou em *rank*) à raiz da maior. **Rank** (aula de 25/08):

- todo nó começa com rank 0;
- só o rank de raízes é atualizado: na união de raízes de ranks *iguais* r, a nova raiz passa a ter rank $r+1$; com ranks diferentes, nada muda;
- o rank nunca decresce; quando um nó deixa de ser raiz, seu rank nunca mais muda;
- sem compressão de caminhos, rank = altura; com compressão, rank é só uma cota superior da altura.

Teorema 4.1 (Altura logarítmica)**.** *Com união por rank (ou por tamanho), toda árvore de rank/altura h tem pelo menos 2^h nós. Logo $h \le \lfloor \log_2 n \rfloor$ e* FIND *custa $O(\log n)$.*

Demonstração. Indução no número de operações. Uma árvore de um nó tem $h = 0$ e $2^0 = 1$ nó. A altura (rank) de uma árvore só aumenta quando se unem duas de mesmo rank h: pela hipótese cada uma tem $\ge 2^h$ nós, e a nova tem $\ge 2 \cdot 2^h = 2^{h+1}$ nós e rank $h+1$. Ligando a menor sob a maior, a altura não aumenta. Como $2^h \le n$, $h \le \log_2 n$. (Szwarcfiter, Teo. 8.2, com união por tamanho e altura contada em níveis: $2^{h(T)-1} \le |T|$, usando $h(T) = \max\{1 + h(T_v), h(T_w)\}$ com $|T_v| \le |T_w|$.) □

Corolário 4.2. *(i) O número de nós com rank exatamente k é $\le n/2^k$ (cada um é raiz, em algum momento, de uma subárvore com $\ge 2^k$ nós, e essas subárvores são disjuntas). (ii) Todo rank é $\le \lfloor \log_2 n \rfloor$. (iii)* $\mathrm{rank}(x) < \mathrm{rank}(pai[x])$ *para todo x não raiz.*

(e) Compressão de caminhos

Durante FIND(x), depois de achar a raiz, faz-se *todo nó do caminho* apontar diretamente para a raiz. As próximas buscas desses nós custam $O(1)$. Variantes (Szwarcfiter): *separação* de caminhos (cada nó passa a apontar para o avô) e *divisão* (nós alternados apontam para o avô) — mesma complexidade assintótica, uma só passada.

Floresta com união por rank e compressão de caminhos

```
Procedimento MAKE-SET(x)
    pai[x] := x; rank[x] := 0
Função FIND(x)
    se pai[x] ≠ x então
        pai[x] := FIND(pai[x])                    % compressão
    retorna pai[x]
Procedimento UNION(x, y)
    r := FIND(x); s := FIND(y)
    se r = s então retorna
    se rank[r] < rank[s] então
        pai[r] := s
    senão se rank[r] > rank[s] então
        pai[s] := r
    senão
        pai[s] := r; rank[r] := rank[r] + 1
```

Versão iterativa da compressão (Szwarcfiter, *comp-buscar*): uma passada acha a raiz y; uma segunda passada percorre de novo o caminho fazendo $pai[\cdot] := y$.

4.3 Rank + compressão: $O(m \log^* n)$ (demonstração da aula de 25/08)

$\log^* n$ = número de vezes que se aplica $\log_2$ até chegar a ≤ 1. Defina $F(0) = 1$, $F(k) = 2^{F(k-1)}$: $F = 1, 2, 4, 16, 65536, 2^{65536}$. Então $\log^* n$ é o menor k com $F(k) \geq n$ ($\log^* n \leq 5$ para qualquer n prático).

Grupos de ranks: o grupo $B \geq 1$ contém os ranks em $(F(B-1), F(B)]$, e o grupo 0 os ranks $\leq F(0) = 1$: grupo 0 $= \{0, 1\}$, 1 $= \{2\}$, 2 $= \{3, 4\}$, 3 $= \{5, \ldots, 16\}$, 4 $= \{17, \ldots, 65536\}$.

- **Propriedade A:** como ranks $\leq \log n$, há no máximo $\log^* n$ grupos (ranks até $\log n$ caem no grupo $\leq \log^* n - 1$).
- **Propriedade B:** o número de nós no grupo B é $\leq \sum_{r=F(B-1)+1}^{F(B)} \frac{n}{2^r} \leq \frac{n}{2^{F(B-1)}} = \frac{n}{F(B)}$ (para $B \geq 1$; o grupo 0 tem trivialmente $\leq n = n/F(0)$ nós).

Num FIND, cobre-se cada nó x do caminho: se x é raiz, filho da raiz, ou se $pai[x]$ está num *grupo diferente* do de x, cobra-se do FIND — como os ranks crescem ao longo do caminho, há $O(\log^* n)$ mudanças de grupo por FIND: total $O(m \log^* n)$. Caso contrário ($pai[x]$ no mesmo grupo), cobra-se de x: a compressão dá a x um novo pai de rank *estritamente maior*; um nó do grupo B só pode receber isso $\leq F(B)$ vezes antes que seu pai vá para outro grupo (e aí nunca mais é cobrado). Total por grupo: $\leq \frac{n}{F(B)} \cdot F(B) = n$; somando os $\log^* n$ grupos, $O(n \log^* n)$. **Total:** $O(m \log^* n)$ (com $m \geq n$).

Deslocamento na lista de grupos das notas

Nas notas de 25/08 a lista explícita "grupo 0: $\{0\}$, grupo 1: $\{1\}$, grupo 2: $\{2\}$, grupo 3: $\{3, 4\}$, grupo 4: $\{5, \ldots, 16\}$" está deslocada de uma posição em relação à fórmula $(F(B-1), F(B)]$ escrita logo acima, que dá grupo 1 $= (1, 2] = \{2\}$, grupo 2 $= (2, 4] = \{3, 4\}$ etc. Nada muda na análise: continua havendo $O(\log^* n)$ grupos e a Propriedade B continua valendo. Na prova, use a fórmula e seja consistente. As notas escrevem m tanto para o número de elementos quanto para o de operações; aqui usamos n para elementos e m para operações.

Observação 4.3. O limite justo (Tarjan) é $O(m\,\alpha(m, n))$, com α a inversa da função de Ackermann ($\alpha \leq 4$ na prática; Szwarcfiter §8.4: $O(n + m\,\alpha(m + n, n))$). Para a prova, o resultado demonstrado em aula é o $O(m \log^* n)$; ambos são "quase lineares".

4.4 Qual implementação usar? (Q2 da AV1)

Tabela 1: Implementações de Union-Find e quando usar cada uma.

Implementação	MAKE-SET	FIND	UNION	Quando é a melhor escolha
Vetor de representantes	$O(1)$	$O(1)$	$O(n)$	Quase só consultas e pouquíssimas uniões, n pequeno.
Listas encadeadas + ponteiro ao representante + união ponderada	$O(1)$	$O(1)$ **pior caso**	$O(\log n)^*$	Consultas dominam e cada uma precisa ser $O(1)$ *garantido*; é preciso **listar os membros** de um grupo em $O(k)$; memória extra aceitável (head/tail/size + ponteiros).
Floresta ingênua	$O(1)$	$O(n)$	$O(1) + \text{Finds}$	Nunca (só didática).
Floresta + união por rank/tamanho (sem compressão)	$O(1)$	$O(\log n)$	$O(\log n)$	Quando é preciso **desfazer uniões** (rollback/"split" offline): sem compressão, cada união muda só um ponteiro (e talvez um rank), que se empilha e se desfaz em $O(1)$.
Floresta + rank + compressão de caminhos	$O(1)$	$O(\log^* n)^*$ (α)		Caso geral, muitas operações misturadas, só *Find* e *Union* (Kruskal, conectividade, redes sociais): praticamente $O(1)$ por operação, memória mínima (um vetor *pai* + *rank*).

Cenários típicos (documento "Conjuntos Disjuntos" do Classroom)

- **"Listar os membros do grupo de U"**: a floresta só tem ponteiros filho→pai; listar exigiria testar $\text{FIND}(i) = \text{FIND}(U)$ para todos os N elementos ($O(N)$). Com **listas encadeadas**: percorre-se a lista, $O(k)$; *MesmoGrupo* em $O(1)$; *Juntar* em $O(\log n)^*$ (menor na maior).
- **Remoção de arestas / conectividade após destruições** ("A queda do império"): Union-Find não desfaz uniões (e a compressão destrói a topologia). Solução *offline*: leia todas as consultas, construa o estado final (sem as arestas que serão destruídas) e processe **de trás para frente**, transformando cada destruição numa UNION; guarde as respostas e imprima invertido.
- **Operações online com desfazer (snapshot/rollback)**: **sem compressão**, só união por rank/tamanho (altura $O(\log n)$, FIND $O(\log n)$); cada UNION empilha o ponteiro/rank alterado; desfazer = desempilhar e restaurar em $O(1)$.
- **Kruskal**: ordena arestas; para cada aresta (u, v), se $\text{FIND}(u) \neq \text{FIND}(v)$ inclui e faz UNION. Floresta com rank + compressão.

Teste 1, Q4 – plataforma com 10^7 usuários

10^5 uniões e 10^8 consultas "mesmo grupo?"; grupos nunca se separam; não importa o histórico. **(a)** Estrutura: **Union-Find**. Entre "árvore apontando até o representante" e "lista com ponteiro direto ao representante": as consultas dominam ($10^8 \gg 10^5$) e precisam ser rápidas; na **lista encadeada** cada FIND é $O(1)$ *no pior caso* (um acesso), e o custo fica nas raras uniões ($O(\log n)^*$ com união ponderada: no total $\leq n \log n \approx 10^7 \cdot 23$ atualizações, diluídas). **Resposta recomendada: a lista encadeada**, porque com razão consultas:uniões de $10^8 : 10^5$ o que importa é o custo de cada FIND, e a lista o garante em $O(1)$ *no pior caso.* A floresta com rank + compressão também é defensável ($O(\alpha)^*$ por operação e menos memória), desde que se explique que a garantia é amortizada; o que *não* se deve propor é floresta sem balanceamento. **(b)** Estratégia de união: **anexar o menor ao maior** (união por tamanho na lista; por rank/tamanho na árvore). Na lista, garante que cada elemento muda de representante $\leq \log_2 n$ vezes (o tamanho do seu grupo dobra a cada mudança). Na árvore, garante altura $\leq \log_2 n$ (Teo. 4.1), evitando

cadeias de altura $\Theta(n)$ que tornariam cada consulta $\Theta(n)$.

4.5 Exercícios resolvidos (monitoria de revisão e lista)

Exemplo 4.4 (Revisão, Q2)**.** Partindo de $\{A\}, \ldots, \{F\}$ e as uniões $\textsc{Union}(A, B)$, $\textsc{Union}(C, D)$, $\textsc{Union}(A, C)$, $\textsc{Union}(E, F)$, com união por rank e, em empate, a primeira raiz vence. $\textsc{Union}(A, B)$: ranks $0 = 0 \Rightarrow$ raiz A, $\text{rank}(A) = 1$. $\textsc{Union}(C, D)$: raiz C, $\text{rank}(C) = 1$. $\textsc{Union}(A, C)$: $1 = 1 \Rightarrow$ raiz A, $\text{rank}(A) = 2$ (árvore $A \to \{B, C\}$, $C \to D$). $\textsc{Union}(E, F)$: raiz E, $\text{rank}(E) = 1$. Representantes finais: A (de $\{A, B, C, D\}$) e E (de $\{E, F\}$): **2 conjuntos**.

Exemplo 4.5 (Revisão, Q3 e Q4)**.** Cadeia $pai[B] = A$, $pai[C] = B$, $pai[D] = C$, $pai[E] = D$. $\textsc{Find}(E)$ com compressão devolve A e deixa B, C, D, E todos apontando para A (árvore de altura 1). Vantagem: consultas futuras a esses nós custam $O(1)$. Se $\text{rank}(A) = \text{rank}(D) = 1$ e $\textsc{Union}(A, D)$: empate, a nova raiz é A (pela convenção) e $\text{rank}(A) = 2$. O rank só sobe no empate porque, ligando a menor sob a maior, a altura da maior não muda; só com alturas iguais a nova árvore fica 1 mais alta.

Exemplo 4.6 (Revisão, Q5 – por que usar as duas técnicas juntas?)**.** Só rank, sem compressão: $\textsc{Find}$ fica $\Theta(\log n)$ (há sequências que atingem essa altura). Só compressão, sem rank: uma única árvore pode ficar alta (cadeia) antes de ser comprimida; custo total $O(m \log n)$ (Tarjan–van Leeuwen), e $\textsc{Find}$ individual pode custar $\Theta(n)$. Juntas: $O(m \log^* n)$ (ou $O(m\,\alpha)$) no total — praticamente constante por operação.

Exemplo 4.7 (Lista UF/AVL, ex. 1 – a cota $\lfloor \log n \rfloor$ é justa)**.** A prova é o Teo. 4.1. Sequência que atinge altura $\log_2 n$ ($n = 2^k$): una os elementos aos pares (altura 1), depois as árvores aos pares (altura 2), etc. Após k rodadas há uma árvore de rank/altura $k = \log_2 n$ (é uma árvore binomial B_k!).

Exemplo 4.8 (Lista UF/AVL, ex. 2 – só compressão)**.** Árvore alta antes da compressão: $\textsc{Union}(1, 2)$, $\textsc{Union}(1, 3)$, …, sempre pendurando a raiz da árvore grande sob o novo elemento unitário ($pai[\text{raiz}] := i$): forma uma cadeia $1 \to 2 \to \cdots \to n$ de altura $n - 1$, sem nenhum $\textsc{Find}$ caro no caminho. Um $\textsc{Find}(1)$ custa $\Theta(n)$ e achata tudo. Resultado conhecido: com compressão e união arbitrária, m operações custam $O(m \log n)$ (na verdade $\Theta(m \log_{1+m/n} n)$); a ideia é que cada compressão que passa por um nó "leve" (cuja subárvore é menos da metade da do pai) só pode acontecer $O(\log n)$ vezes por nó.

Pontos-chave

Q2 em uma frase: **listas encadeadas** dão $\textsc{Find}$ $O(1)$ pior caso, $\textsc{Union}$ $O(\log n)^*$ com "menor na maior" e listagem de membros em $O(k)$; **florestas** com **rank + compressão** dão $O(\log^* n)$ (ou α) amortizado para tudo com memória mínima; **sem compressão** se for preciso desfazer uniões. Sempre **menor no maior**, senão árvores/listas degeneram.

5 Árvores binárias de busca e árvores AVL

5.1 Árvore binária de busca (ABB)

Motivação: lista sequencial ordenada dá busca binária $\Theta(\log n)$, mas inserção/remoção $\Omega(n)$; lista encadeada insere rápido mas busca em $\Omega(n)$. A ABB combina busca eficiente com estrutura dinâmica.

Definição 5.1. Árvore binária em que, para todo nó p: a chave de p é $\geq$ às chaves da subárvore esquerda e $\leq$ às da subárvore direita. (O percurso *em ordem* lista as chaves ordenadas.)

- **Busca:** desce comparando; custo $\Theta(h)$, h = altura.

- **Inserção:** busca a posição e insere como **folha**: $\Theta(h)$.
- **Remoção** de p: (i) folha: remove; (ii) um filho: o filho ocupa o lugar de p; (iii) dois filhos: o **maior da subárvore esquerda** (predecessor: desce à esquerda uma vez e depois sempre à direita) assume o lugar de p (alternativa simétrica: menor da direita). $\Theta(h)$.
- $\lfloor \log_2 n \rfloor + 1 \le h \le n$ (em níveis). Inserções em ordem crescente/decrescente geram uma "lista" de altura $n \Rightarrow$ tudo vira $\Theta(n)$.

Árvores completas minimizam a altura, mas mantê-las completas após inserções custa $\Omega(n)$ (reorganização global). Solução: exigir só **balanceamento**: altura $O(\log n)$, e o mesmo para toda subárvore (m nós $\Rightarrow$ altura $O(\log m)$), com rebalanceamento em $O(\log n)$.

5.2 Árvores AVL (Adel'son-Vel'skii e Landis, 1962)

Definição 5.2. Uma árvore binária é **AVL** se, para todo nó v, as alturas das subárvores esquerda e direita diferem de no máximo 1 (v é *regulado*; senão, *desregulado*). Define-se

$$\mathrm{bal}(v) = h_D(v) - h_E(v) \qquad \text{(direita menos esquerda; bal} = -1 \text{ "pesa à esquerda"),}$$

e v é regulado sse $\mathrm{bal}(v) \in \{-1, 0, 1\}$. Folhas têm bal $= 0$. Toda árvore completa é AVL (a recíproca é falsa).

Teorema 5.3 (AVL é balanceada)**.** *Toda árvore AVL com n nós tem altura $O(\log n)$ (precisamente $h < 1{,}44 \log_2(n+2)$).*

Demonstração. Seja $N(h)$ o número mínimo de nós de uma AVL de altura h (em níveis). $N(1) = 1$, $N(2) = 2$ e, para $h \ge 3$, uma AVL mínima de altura h tem raiz, uma subárvore de altura $h-1$ e a outra de altura $h-2$ (a menor possível), ambas mínimas:

$$N(h) = 1 + N(h-1) + N(h-2).$$

(Essas árvores mínimas são as **árvores de Fibonacci**; $N(h) = F_{h+2} - 1$.) Como N é crescente, $N(h) > 2N(h-2)$, donde $N(h) \ge 2^{\lfloor h/2 \rfloor}$, isto é, $h \le 2 \log_2 N(h) + 1$. Como qualquer AVL de altura h com n nós tem $n \ge N(h)$, $h = O(\log n)$. □

Consequência: busca em $O(\log n)$. Falta mostrar que inserção e remoção, com o rebalanceamento, também são $O(\log n)$.

5.3 Inserção: atualização dos balanços

Insere-se como numa ABB (a nova chave vira folha; se já existe, para). Depois percorre-se o caminho de volta, do pai w do novo nó até a raiz, atualizando bal. Suponha que a inserção foi na subárvore **esquerda** de p (o caso direito é simétrico, trocando os sinais):

bal(p) antes	depois	altura de $T(p)$	ação
$+1$	0	não mudou	**para** (ancestrais não mudam)
0	-1	aumentou 1	continua com o pai de p (se p é a raiz, para)
-1	-2	—	p desregulado: **rotação**; depois, **para**

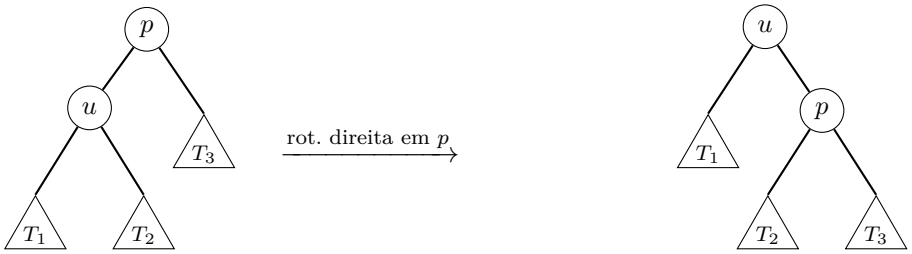
Uma inserção aumenta alturas em no máximo 1, logo o desregulado tem $|\,\mathrm{bal}\,| = 2$. O p a rotacionar é o **primeiro nó desregulado encontrado ao subir da folha para a raiz** (o mais profundo). Como a rotação devolve à subárvore a altura que ela tinha *antes* da inserção, **uma inserção faz no máximo uma rotação** (simples ou dupla).

5.4 As rotações

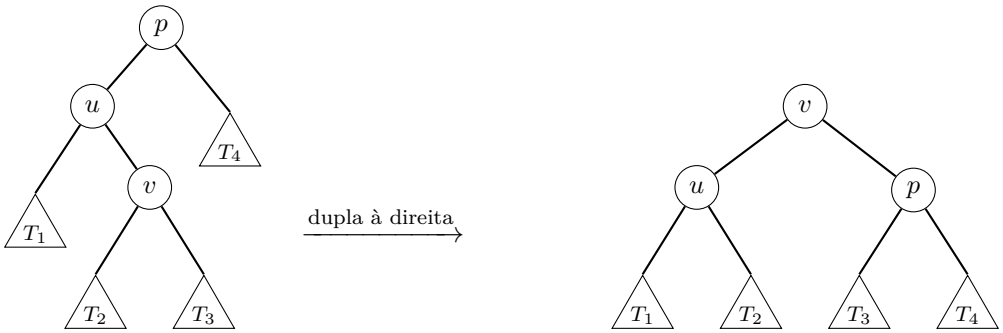
Seja p o nó desregulado mais profundo.

Caso	bal(p)	filho do lado pesado	sinais	rotação
1.1 (“LL”)	-2	u = esq., bal(u) $= -1$	iguais	**simples à direita** em p
1.2 (“LR”)	-2	u = esq., bal(u) $= +1$	opostos	**dupla à direita**: esq. em u, dir. em p
2.1 (“RR”)	$+2$	z = dir., bal(z) $= +1$	iguais	**simples à esquerda** em p
2.2 (“RL”)	$+2$	z = dir., bal(z) $= -1$	opostos	**dupla à esquerda**: dir. em z, esq. em p

Regra mnemônica (monitoria): **FB positivo ⇒ rotações à esquerda; negativo ⇒ à direita. Mesmo sinal: simples; sinais opostos: dupla.** (Filho com bal $= 0$ só ocorre na *remoção*, e aí usa-se rotação simples.)



Caso 1.1: $h(T_1) = h + 1$, $h(T_2) = h(T_3) = h$. Depois: bal(u) = bal(p) $= 0$; altura volta a $h + 2$.



Caso 1.2: v sobe para a raiz. Depois bal(v) $= 0$; se bal(v) era -1: bal(u) $= 0$, bal(p) $= +1$; se era $+1$: bal(u) $= -1$, bal(p) $= 0$; se era 0 (v é o novo nó): ambos 0.

Os casos 2.1 e 2.2 são espelhados (rotação simples à esquerda: z sobe, p vira filho esquerdo de z e o antigo filho esquerdo de z vira filho direito de p). **Rotações preservam a ordem da ABB** (o percurso em ordem T_1, u, T_2, p, T_3 é o mesmo) e custam $O(1)$ (só ponteiros).

5.5 Pseudocódigo (slides do professor)

Cada nó tem campos `esq`, `dir`, `chave`, `bal`; λ é o ponteiro nulo; $pt\uparrow.campo$ acessa o campo do nó apontado. Chamada externa: Ins-AVL($x, ptraiz$). O booleano h devolvido indica se **a altura da subárvore aumentou** (só então o pai precisa atualizar seu balanço).

Inserção

```
Procedimento Iniciar-no(pt, x)
    alocar(pt); pt↑.esq := λ; pt↑.dir := λ; pt↑.chave := x; pt↑.bal := 0
Função Ins-AVL(x, ref pt)                    % retorna true se a altura de T(pt) aumentou
    se pt = λ então
        Iniciar-no(pt, x); retorna true
    se x = pt↑.chave então
        retorna false                                  % chave repetida
    se x < pt↑.chave então
```

```
        h := Ins-AVL(x, pt↑.esq)
        se h então
            pt, h := AtualizaBalanço(pt, "esquerda")
    senão
        h := Ins-AVL(x, pt↑.dir)
        se h então
            pt, h := AtualizaBalanço(pt, "direita")
    retorna h
Função AtualizaBalanço(pt, a)
    h := false
    se a = "esquerda" então
        se pt↑.bal = 1 então
            pt↑.bal := 0
        senão se pt↑.bal = 0 então
            pt↑.bal := −1; h := true                    % altura aumentou: o pai verifica
        senão
            pt := RotaçõesDireita(pt)                   % era −1: ficaria −2
    senão
        se pt↑.bal = −1 então
            pt↑.bal := 0
        senão se pt↑.bal = 0 então
            pt↑.bal := 1; h := true
        senão
            pt := RotaçõesEsquerda(pt)
    retorna (pt, h)          % se houve rotação, pt aponta para a nova raiz da subárvore
```

Rotações (slides)

```
Função RotaçõesDireita(pt)
    ptu := pt↑.esq
    se ptu↑.bal = −1 então                                  % simples à direita
        pt↑.esq := ptu↑.dir; ptu↑.dir := pt; pt↑.bal := 0; ptu↑.bal := 0; pt := ptu
    senão                                                   % dupla à direita
        ptv := ptu↑.dir
        ptu↑.dir := ptv↑.esq; ptv↑.esq := ptu; pt↑.esq := ptv↑.dir; ptv↑.dir := pt
        se ptv↑.bal = −1 então pt↑.bal := 1
        senão pt↑.bal := 0
        se ptv↑.bal = 1 então ptu↑.bal := −1
        senão ptu↑.bal := 0
        ptv↑.bal := 0; pt := ptv
    retorna pt
Função RotaçõesEsquerda(pt)
    ptz := pt↑.dir
    se ptz↑.bal = 1 então                                   % simples à esquerda
        pt↑.dir := ptz↑.esq; ptz↑.esq := pt; pt↑.bal := 0; ptz↑.bal := 0; pt := ptz
    senão                                                   % dupla à esquerda
        pty := ptz↑.esq
        ptz↑.esq := pty↑.dir; pty↑.dir := ptz; pt↑.dir := pty↑.esq; pty↑.esq := pt
        se pty↑.bal = 1 então pt↑.bal := −1
        senão pt↑.bal := 0
        se pty↑.bal = −1 então ptz↑.bal := 1
        senão ptz↑.bal := 0
        pty↑.bal := 0; pt := pty
    retorna pt
```

Detalhes do pseudocódigo que costumam ser perguntados

- **Para que serve h?** Indica se a altura da subárvore cresceu; só nesse caso o pai precisa atualizar o balanço. Por isso AtualizaBalanço só é chamada se $h =$ true: se a altura não mudou, nenhum ancestral muda de balanço.
- **Por que atualizar pt após a rotação?** A raiz da subárvore mudou (u, v, z ou y sobe); como pt é passado *por referência*, a atribuição religa a nova raiz ao pai (ou atualiza $ptraiz$).
- Nos slides, os três testes de AtualizaBalanço aparecem como "se" consecutivos; eles precisam ser **mutuamente exclusivos** ("senão se"), como acima. Com "se" em sequência, um nó com bal $= 1$ viraria 0 e, logo em seguida, o teste "bal $= 0$" dispararia de novo (bug do "aluno" da monitoria).
- Após a rotação na inserção, $h :=$ false (a altura voltou ao valor antigo).

5.6 Remoção em AVL

Remove-se como numa ABB (se p tem dois filhos, o maior da subárvore esquerda o substitui) e sobe-se a partir do nó *fisicamente removido* atualizando balanços. Agora a pergunta é se a altura **diminuiu**. Remoção na subárvore **direita** de p (esquerda é simétrico):

bal(p) antes	depois	altura de $T(p)$	ação
$+1$	0	**diminuiu**	continua com o pai
0	-1	não mudou	**para**
-1	-2	—	rotação à direita (ver abaixo)

Na rotação, olha-se o filho esquerdo u:

- bal(u) $= -1$: simples à direita; ambos ficam 0; a altura **diminuiu** $\Rightarrow$ continua subindo.
- bal(u) $= 0$ (*caso exclusivo da remoção*): simples à direita; fica bal(p) $= -1$, bal(u) $= +1$; a altura **não muda** $\Rightarrow$ para.
- bal(u) $= +1$: dupla à direita; altura **diminuiu** $\Rightarrow$ continua subindo.

Por isso a remoção pode precisar de uma rotação em **cada** ancestral: $O(\log n)$ rotações, cada uma $O(1) \Rightarrow O(\log n)$ no total.

Armadilhas comuns

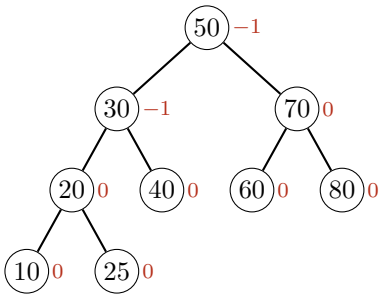
Os slides implementam a remoção em dois passos (Remover-AVL-Passo1: remoção de ABB que devolve o nó mais profundo afetado e o ajuste $+1/0/-1$; Remover-AVL-Passo2: sobe atualizando com AtualizaBalançoAposRemoção). Ao reutilizar RotaçõesDireita na remoção, o caso $ptu\uparrow.bal = 0$ cairia na rotação *dupla* (o teste é só "$= -1$"), o que está errado: com bal(u) $= 0$ deve-se fazer a **simples** (teste "≤ 0") e ajustar bal(p) $= -1$, bal(u) $= +1$, devolvendo "altura não diminuiu".

Exemplo 5.4 (Remoções pequenas (conferidas))**.**

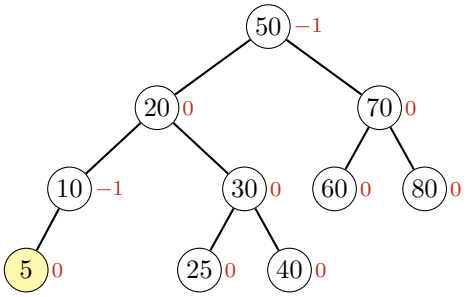
- $20^{+1}(10,\ 40^{0}(30, 50))$, remover 10: bal(20) $= +2$, filho 40 com bal $= 0 \Rightarrow$ simples à esquerda em 20: $40^{-1}(20^{+1}(\cdot, 30),\ 50)$; altura inalterada.
- $20^{-1}(10^{+1}(\cdot, 15),\ 30)$, remover 30: bal(20) $= -2$, filho 10 com bal $= +1 \Rightarrow$ dupla à direita: $15(10, 20)$.

5.7 Exemplos resolvidos (conferidos por simulação)

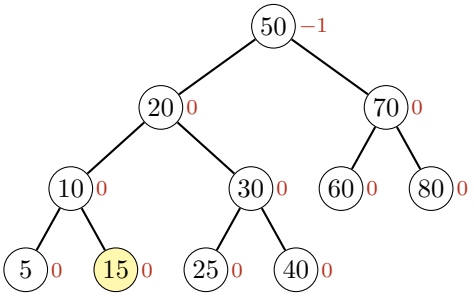
Exemplo 5.5 (Teste 1, Q5)**.** Árvore inicial e inserções $5, 15, 35, 45$ (balanços em vermelho, bal $= h_D - h_E$):



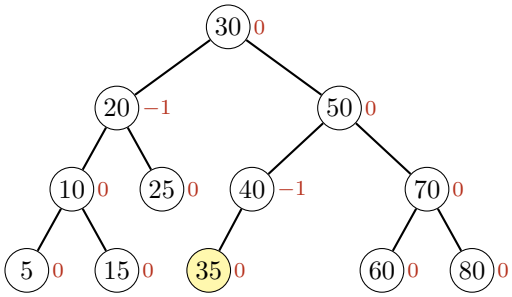
Inserir 5 (filho esq. de 10). Subindo: bal(10) = −1, bal(20) = −1, bal(30) = −2. **Primeiro desregulado: 30**; filho esq. 20 com bal = −1 (mesmo sinal) ⇒ caso 1.1, **rotação simples à direita em 30**. A altura da subárvore volta a 3, então 50 fica com −1.



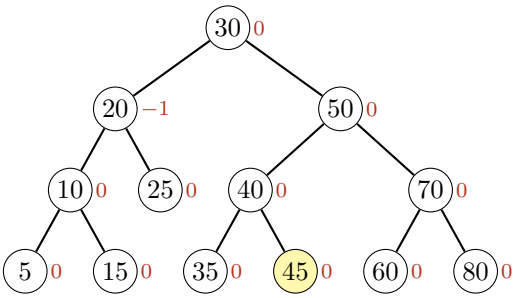
Inserir 15 (filho dir. de 10): bal(10) vai de −1 a 0 e a altura de $T(10)$ não muda: **para**. Sem rotação.



Inserir 35 (filho esq. de 40). Subindo: bal(40) = −1, bal(30) = +1, bal(20) = +1, bal(50) = −2. **Primeiro desregulado: 50**; filho esq. 20 com bal = +1 (sinais opostos) ⇒ caso 1.2, **rotação dupla à direita**: esquerda em 20, depois direita em 50. O nó $v = 30$ sobe para a raiz; como bal(30) era +1: bal(20) = −1, bal(50) = 0.



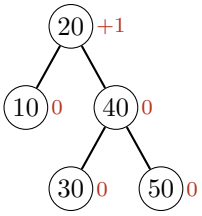
Inserir 45 (filho dir. de 40): bal(40) vai de −1 a 0, altura de $T(40)$ inalterada: para. Árvore final:



(c) Numa ABB comum, inserir códigos em ordem desfavorável (crescente, decrescente, "zigue-zague") gera altura $\Theta(n)$ e busca/inserção/remoção em $\Theta(n)$; a AVL garante altura $\leq 1{,}44 \log_2 n$ (Teo. 5.3), logo $O(\log n)$ *independentemente da ordem de chegada*, pagando só $O(1)$ rotações por inserção.

Exemplo 5.6 (Monitoria de revisão, Q5: $40, 20, 10, 30, 35$)**.** 40; $40^{-1}(20)$; inserir 10: bal(40) $= -2$, filho 20 com $-1 \Rightarrow$ simples à direita em 40: $20^{0}(10, 40)$. Inserir 30: $20^{+1}(10,\ 40^{-1}(30, \cdot))$. Inserir 35 (dir. de 30): bal(30) $= +1$, bal(40) $= -2 \Rightarrow$ primeiro desregulado é **40**, filho esq. 30 com $+1 \Rightarrow$ **dupla à direita** (esq. em 30, dir. em 40). Final: $20^{+1}(10^{0},\ 35^{0}(30^{0}, 40^{0}))$. Rotações: na 3ª inserção (simples, direita) e na 5ª (dupla, direita).

Exemplo 5.7 (Monitoria de revisão, Q7: $10, 20, 30, 40, 50$ — e um erro no slide)**.** Inserir 30: bal(10) $=$ $+2$, filho 20 com $+1 \Rightarrow$ simples à esquerda em 10: $20(10, 30)$. Inserir 40: $20^{+1}(10,\ 30^{+1}(\cdot, 40))$. Inserir 50: subindo, bal(40) $= +1$, bal(30) $= +2 \Rightarrow$ o **primeiro desregulado é 30** (e não 20!): simples à esquerda em 30. **Final correto:** $20^{+1}(10^{0},\ 40^{0}(30^{0}, 50^{0}))$.



Atenção: o slide da monitoria rotaciona em 20 e chega a uma árvore com raiz 30 — está errado: a rotação é feita no desregulado *mais profundo*; após rotacionar em 30, o nó 20 fica com bal $= +1$ e nem chega a desregular.

Inserir 49 na árvore correta: vai para a esquerda de 50; bal(50) $= -1$, bal(40) $= +1$, bal(20) $=$ $+2 \Rightarrow$ desregulado 20, filho dir. 40 com $+1 \Rightarrow$ **simples à esquerda em 20**: $40^{0}(20^{0}(10, 30),$ $50^{-1}(49, \cdot))$. (Partindo da árvore errada do slide, $30(20(10), 40(\cdot, 50))$, o desregulado seria 40 com filho 50 de bal $= -1$, e aí sim seria a **dupla à esquerda** mostrada no slide, com resultado $30(20(10),$ $49(40, 50))$.)

5.8 Exercícios da lista (Union-Find e AVL)

Exemplo 5.8 (Ex. 3 – remoção com rotação em *todos* os ancestrais)**.** Use uma **árvore de Fibonacci** (AVL mínima) em que todo nó interno tem bal $= -1$ (esquerda mais alta): T_h tem raiz, T_{h-1} à esquerda e T_{h-2} à direita. Removendo a **folha mais à direita** (caminho raiz $\to T_{h-2} \to T_{h-4} \to \cdots$), cada ancestral perde altura do lado já mais baixo, fica com bal $= -2$ e tem filho esquerdo com bal $= -1 \Rightarrow$ rotação simples que *diminui* a altura, propagando o problema. Exemplo conferido: T_7 com 33 nós (chaves 1..33 em ordem), raiz 21; remover a folha 33 (ancestrais $32, 29, 21$) causa rotações em **32, 29 e 21**. Em geral, h ímpar dá $(h-1)/2$ rotações $= \Theta(\log n)$.

Exemplo 5.9 (Ex. 4 – fusão de duas AVL (alto nível))**.** *Caso "junção" (todas as chaves de $T_1 <$ todas as de T_2):* retire o mínimo x de T_2 (ou o máximo de T_1) em $O(\log n)$. Suponha $h(T_1) \geq h(T_2)$: desça pela espinha direita de T_1 até um nó v com $h(v) \in \{h(T_2), h(T_2)+1\}$; crie o nó x com filhos $T(v)$ e T_2 e coloque-o no lugar de v; suba rebalanceando como numa inserção (no máximo uma rotação ajusta tudo). Custo $O(|h_1 - h_2| + 1) = O(\log n)$. *Caso geral (chaves intercaladas):* percursos

em ordem de ambas ($O(n+m)$), intercalação das duas listas ordenadas ($O(n+m)$) e construção de uma árvore completa a partir da lista ordenada (sempre a mediana como raiz, $O(n+m)$) — total linear. Alternativa: inserir cada nó da menor na maior, $O(m \log(n+m))$.

Exemplo 5.10 (Ex. 5 – AVL como estrutura de conjuntos disjuntos?)**.** Possível: cada conjunto é uma AVL e cada elemento guarda ponteiro para o pai. FIND(x) sobe até a raiz da sua AVL (que identifica o conjunto): $O(\log n)$. UNION: inserir os elementos da menor AVL na maior, $O(k \log n)$ (k = tamanho da menor); com "menor na maior", cada elemento é reinserido $\leq \log n$ vezes, total $O(n \log^2 n)$. Ganha-se ordem (percurso ordenado, busca por chave), mas é pior que o Union-Find ($\approx O(1)$ amortizado) para FIND/UNION.

Heap × AVL × ABB (revisão, Q6 e Q14)

- **Fila de prioridade pura** (sempre atender o de maior prioridade): **heap** — máximo em $O(1)$, inserir/remover $O(\log n)$, vetor simples. A AVL também faria tudo em $O(\log n)$, mas não dá o máximo em $O(1)$ sem ponteiro extra e é mais pesada.
- **Busca por chave arbitrária, remoção de elemento específico, percurso em ordem, sucessor/predecessor, intervalos**: **AVL** (o heap não tem ordem entre irmãos: buscar uma chave é $O(n)$).
- **ABB simples × AVL**: com inserções/remoções frequentes e ordem de chegada imprevisível, **AVL** (ABB pode degenerar para $\Theta(n)$).

5.9 Árvores rubro-negras (pesquisa)

Complemento – pesquisa proposta em 02/09, entrega 19/09 (Cormen, cap. 13)

Definição: ABB em que cada nó é *vermelho* ou *preto* (1 bit extra) e: (1) a raiz é preta; (2) as folhas nulas (λ) são pretas; (3) um nó vermelho tem os dois filhos pretos (não há dois vermelhos seguidos); (4) todo caminho de um nó até as folhas nulas descendentes tem o mesmo número de nós pretos (*altura negra* bh).
Por que é balanceada: por (4), a subárvore de um nó x tem $\geq 2^{bh(x)} - 1$ nós internos (indução); por (3), pelo menos metade dos nós de qualquer caminho raiz–folha é preta, então $bh(\text{raiz}) \geq h/2$. Logo $n \geq 2^{h/2} - 1$, isto é, $h \leq 2 \log_2(n+1)$.
Inserção: insere como ABB e pinta o nó z de *vermelho* (não altera alturas negras); se o pai é vermelho, viola (3). Correção olhando o *tio* y: (caso 1) tio vermelho $\Rightarrow$ recolore pai e tio de preto e avô de vermelho, e continua do avô; (caso 2) tio preto e z em "zigue-zague" $\Rightarrow$ rotação no pai para virar caso 3; (caso 3) tio preto e z "em linha" $\Rightarrow$ rotação no avô e troca de cores pai/avô. No fim, raiz preta. No máximo **2 rotações** por inserção; recolorações $O(\log n)$.
Remoção: remoção de ABB; se o nó retirado era preto, surge um "preto duplo" que é resolvido olhando o irmão (4 casos), com no máximo **3 rotações**.
AVL × rubro-negra: AVL é mais rigidamente balanceada ($h \leq 1{,}44 \log n$ vs. $2 \log n$): buscas um pouco mais rápidas. Rubro-negra faz menos rotações em atualizações (remoção $O(1)$ rotações vs. $O(\log n)$ na AVL): preferida em bibliotecas (`std::map`, `TreeMap`, kernel Linux).

Pontos-chave

AVL: bal $= h_D - h_E \in \{-1, 0, 1\}$; altura $\leq 1{,}44 \log n$ (árvores de Fibonacci). Inserção: sobe atualizando; $+1 \to 0$ para; $0 \to \pm 1$ continua; $\pm 1 \to \pm 2$ rotaciona no **mais profundo** e para (1 rotação). Sinais iguais: simples; opostos: dupla. Remoção: pode rotacionar em todos os ancestrais; filho com bal $= 0 \Rightarrow$ simples e para.

6 Ordenação

Problema: dado S, obter uma permutação $s_1, \ldots, s_n$ com $s_i \leq s_{i+1}$.

- **Estável:** elementos de chaves iguais aparecem na saída na mesma ordem relativa da entrada (essencial para o radix sort e para ordenar por várias chaves).
- **In-place:** memória adicional independente de n (ou $O(\log n)$ de pilha). Quicksort e heapsort são in-place; mergesort e counting sort não.

6.1 Ordenação por comparação

Mergesort (intercalação)

MergeSort e Intercalação

```
Procedimento MergeSort(V, ini, fim)
    se ini < fim então
        meio := ⌊(ini + fim)/2⌋
        MergeSort(V, ini, meio); MergeSort(V, meio + 1, fim)
        Intercala(V, ini, meio, fim)
Procedimento Intercala(V, ini, meio, fim)
    i := ini; j := meio + 1; k := 1                          % W = vetor auxiliar
    enquanto i ≤ meio e j ≤ fim faça
        se V[i] ≤ V[j] então
            W[k] := V[i]; i := i + 1                         % "≤" mantém a estabilidade
        senão
            W[k] := V[j]; j := j + 1
        k := k + 1
    copie o que sobrou de V[i..meio] ou de V[j..fim] para o fim de W
    copie W[1..fim − ini + 1] de volta para V[ini..fim]
```

$T(n) = 2T(n/2) + \Theta(n) \Rightarrow \Theta(n \log n)$ em todos os casos. **Estável** (com "$\leq$"), **não in-place** ($\Theta(n)$ extra). Versão iterativa (*bottom-up*): intercale blocos de tamanho $1, 2, 4, \ldots$ ao longo do vetor.

Quicksort

Se $n \leq 1$, pronto; escolha um **pivô** x; particione em $S_1 = \{w < x\}$ e $S_2 = \{w \geq x\}$; ordene recursivamente e concatene.

Quicksort com partição de Lomuto

```
Procedimento QuickSort(V, ini, fim)
    se ini < fim então
        q := Particiona(V, ini, fim)
        QuickSort(V, ini, q − 1); QuickSort(V, q + 1, fim)
Função Particiona(V, ini, fim)
    x := V[fim]; i := ini − 1                    % pivô no fim (troque antes se escolher outro)
    para j := ini até fim − 1 faça
        se V[j] < x então
            i := i + 1; V[i] ⇔ V[j]
    V[i + 1] ⇔ V[fim]; retorna i + 1
```

- **Pior caso** $\Theta(n^2)$: partições sempre 0 e $n-1$ (ex.: pivô = primeiro elemento e vetor já ordenado ou invertido): $T(n) = T(n-1) + \Theta(n)$.
- **Melhor/médio** $\Theta(n \log n)$. Argumento do slide: com pivô "aleatório", em metade dos casos a parte maior tem tamanho $\leq 3n/4$; se isso sempre ocorresse, no nível t os subproblemas teriam tamanho $\leq (3/4)^t n$, $O(\log n)$ níveis de custo $O(n)$; como ocorre com probabilidade $1/2$, em média a cada 2 níveis o tamanho cai por $3/4$: ainda $O(n \log n)$ esperado.

- **Escolha do pivô:** primeiro elemento (ruim se já ordenado), aleatório, **mediana de três** (primeiro, meio, último). Usando a **mediana exata via BFPRT** (§7) obtém-se $O(n \log n)$ no *pior* caso, mas a constante é alta — na prática não compensa (resposta a um exercício dos slides).
- **In-place** (partição por trocas; recursão $O(\log n)$ de pilha se recursar primeiro na parte menor). **Não estável** (a troca de longa distância embaralha iguais); dá para torná-lo estável com partição estável usando $O(n)$ de memória extra.

Heapsort

Ver §2: $\Theta(n \log n)$ sempre, in-place, não estável.

Algoritmos quadráticos (para referência)

Inserção: insere $V[i]$ na parte já ordenada $V[1..i-1]$ deslocando os maiores; $\Theta(n^2)$ pior, $\Theta(n)$ se já ordenado, estável, in-place, ótimo para n pequeno ou quase ordenado (usado dentro do bucket sort). **Bolha**: $\Theta(n^2)$, estável.

6.2 Limite inferior $\Omega(n \log n)$ para ordenação por comparação

Teorema 6.1. *Todo algoritmo de ordenação baseado em comparações faz $\Omega(n \log n)$ comparações no pior caso.*

Demonstração. Modela-se o algoritmo (para cada n) por uma **árvore de decisão**: cada nó interno é uma comparação $a_i \le a_j?$, com dois filhos (os dois resultados); cada folha é um estado final (uma permutação que ordena a entrada). A altura h é o número de comparações no pior caso. Como qualquer uma das $n!$ permutações pode ser a resposta, há pelo menos $n!$ folhas; uma árvore binária de altura h tem no máximo 2^h folhas. Logo

$$2^h \ge n! \ge n(n-1)\cdots\left(\tfrac{n}{2}\right) \ge \left(\tfrac{n}{2}\right)^{n/2} \;\Rightarrow\; h \ge \tfrac{n}{2}\log_2 \tfrac{n}{2} = \tfrac{n}{2}\log n - \tfrac{n}{2} = \Omega(n \log n). \qquad \square$$

Corolário 6.2. *Heapsort e mergesort são **ótimos** entre os algoritmos por comparação (supondo comparação $O(1)$). Para ordenar em $o(n \log n)$ é preciso* não *se basear em comparações: usar a estrutura das chaves (inteiros pequenos, dígitos, distribuição).*

6.3 Ordenação em tempo linear

Resposta ao desafio "dá para ordenar em tempo linear?": depende da natureza dos elementos.

Counting sort (inteiros em $[0, k]$)

COUNTING-SORT(A, n, k) **(slides)**

```
para i := 0 até k faça
    C[i] := 0
para j := 1 até n faça
    C[A[j]] := C[A[j]] + 1                          % C[i] = #{j : A[j] = i}
para i := 1 até k faça
    C[i] := C[i] + C[i − 1]                         % C[i] = #{j : A[j] ≤ i}
para j := n decrescendo até 1 faça                  % de trás para frente ⇒ estável
    B[C[A[j]]] := A[j]; C[A[j]] := C[A[j]] − 1
retorna B
```

- Tempo e espaço $\Theta(n + k)$; **linear se** $k = O(n)$. Se $k = \Theta(2^n)$ fica exponencial: é **pseudopolinomial**.

- **Estável**: a última ocorrência de cada valor vai para a última posição do seu bloco. **Não in-place** (vetores B e C). Não faz nenhuma comparação entre elementos.

Radix sort (inteiros com d dígitos na base b)

Ordena **dígito a dígito, do menos significativo (LSD) para o mais significativo**, usando em cada passada um método **estável**.

Radix sort com filas (slides)

para $i := 1$ **até** d **faça**
  para $j := 1$ **até** n **faça**
    $k :=$ i-ésimo dígito menos significativo de $L[j]$ na base b; insere $L[j]$ no fim da fila $F[k]$
  $j := 1$
  para $k := 0$ **até** $b - 1$ **faça**
    enquanto $F[k]$ não vazia **faça**
      remove o início de $F[k]$ e põe em $L[j]$; $j := j + 1$

- **Por que precisa ser estável?** Depois de ordenar pelo dígito i, elementos com o mesmo dígito i devem manter a ordem obtida pelos dígitos $1..i-1$; é o que garante (por indução) que após a passada i a lista está ordenada pelos i últimos dígitos. As filas preservam a ordem naturalmente (alternativa: counting sort por dígito).
- Tempo $\Theta(d(n+b))$; com b e d constantes, $\Theta(n)$. Truque: inteiros até n^c na base $b = n$ têm $d = c$ dígitos $\Rightarrow \Theta(cn)$.
- **Exemplo da aula (conferido):** $102, 313, 690, 131, 222, 670, 690, 130, 079, 102$.
  Unidades: 690 670 690 130 131 102 222 102 313 079;
  dezenas: 102 102 313 222 130 131 670 079 690 690;
  centenas: 079 102 102 130 131 222 313 670 690 690.
- Chaves de tamanhos variáveis d_i (exercício): agrupe por número de dígitos (counting por tamanho) e aplique o radix de modo que cada número só participe das passadas dos seus próprios dígitos, ou use o MSD/American Flag recursivo: $O(\sum d_i)$. Para **strings**: ordenar da direita para a esquerda completando as menores com um símbolo "menor que todos", ou MSD (primeiro caractere) recursivo.

Bucket sort (reais uniformes em $[0, 1)$)

Divide $[0, 1)$ em r baldes iguais, distribui ($A[i]$ vai para o balde $\lfloor r\, A[i] \rfloor$), ordena cada balde (ex.: inserção) e concatena. Com entrada uniforme, cada balde tem em média n/r elementos; com $r = \Theta(n)$ o tempo **esperado** é $\Theta(n)$ (Cormen: $E[n_i^2] = 2 - 1/n$, então $\sum E[O(n_i^2)] = O(n)$). Pior caso: o do algoritmo usado nos baldes (tudo num balde só).

American Flag sort

Radix **MSD in-place**: conta quantos elementos há de cada dígito (vetor C de tamanho b), calcula onde começa cada bloco e *permuta no próprio vetor* cada elemento para o seu bloco (trocas); depois recursão em cada bloco no dígito seguinte. **Memória auxiliar:** só os contadores, $O(b)$ por nível de recursão, $O(b \cdot d)$ no total (independe de n) $\Rightarrow$ **in-place**. **Não é estável** (as trocas mudam a ordem de iguais).

6.4 Qual algoritmo de ordenação usar?

Situação	Escolha
Inteiros num intervalo pequeno ($k = O(n)$)	**Counting sort** $\Theta(n+k)$
Inteiros/IDs com poucos dígitos (d fixo), k enorme	**Radix sort** $\Theta(d(n+b))$
Reais uniformemente distribuídos em um intervalo	**Bucket sort** $\Theta(n)$ esperado
Chaves arbitrárias, precisa de garantia e/ou estabilidade	**Mergesort** $\Theta(n \log n)$ (memória $\Theta(n)$)
Chaves arbitrárias, memória escassa, garantia	**Heapsort** $\Theta(n \log n)$ in-place
Chaves arbitrárias, desempenho médio na prática	**Quicksort** (pivô aleatório/mediana de 3)
Quase ordenado ou n pequeno	**Inserção**
Dados em lista encadeada / arquivos externos	**Mergesort**

Teste 2, Exercício 3 – respostas esperadas

- **A** (10^7 notas entre 0 e 100): **Counting sort**. $k = 101 \ll n$: $\Theta(n+k) = \Theta(n)$, vetor de contagem de 101 posições; comparação daria $\Omega(n \log n) \approx 2{,}3 \cdot 10^8$ comparações.
- **B** (10^6 identificadores de 12 dígitos): **Radix sort**. Counting é inviável ($k = 10^{12}$); radix com $d = 12$ e $b = 10$ faz 12 passadas lineares: $\Theta(d(n+b)) \approx 1{,}2 \cdot 10^7$ (ou base $10^3/10^4$: 4 ou 3 passadas).
- **C** (10^5 reais uniformes em $[0, 1)$): **Bucket sort**, n baldes, tempo esperado $\Theta(n)$ (radix/-counting não se aplicam a reais).
- **D** (10^6 inteiros em $[-10^9, 10^9]$): **Radix sort** após somar 10^9 (chaves em $[0, 2 \cdot 10^9]$, ≈ 31 bits: 2 passadas na base 2^{16}, ou 10 na base 10). Counting inviável ($k = 2 \cdot 10^9 \gg n$ de memória). Mergesort/quicksort $\Theta(n \log n)$ também seriam aceitáveis, mas o radix é linear.
- **E** (10^5 registros, chaves arbitrárias, sem restrição de domínio): só **comparação**: **Mergesort** (garantia $\Theta(n \log n)$ e estabilidade, ideal para registros) ou Quicksort (mais rápido na média, in-place, mas $\Theta(n^2)$ no pior caso e não estável).

6.5 Q4: dado um código, qual estrutura e qual ordenação?

A Q4 dará um código genérico; o roteiro é: **(1)** liste as operações que o código faz repetidamente; **(2)** estime quantas vezes cada uma ocorre; **(3)** escolha a estrutura que torna barata a operação dominante; **(4)** olhe o tipo das chaves para escolher a ordenação; **(5)** justifique com complexidades (e memória/estabilidade quando relevante).

O código faz repetidamente...	Estrutura
pega o maior/menor, insere, (às vezes) altera prioridade	**heap binário**
... e une filas com frequência (previsível)	**heap binomial**
... com muitíssimos aumentos de prioridade (custo total)	**heap de Fibonacci**
"x e y estão no mesmo grupo?" / une grupos	**Union-Find** (rank + compressão)
busca por chave, inserção, remoção, em ordem, sucessor, mínimo/máximo, intervalos	**AVL** (ou rubro-negra)
só busca/inserção/remoção por chave exata, sem ordem	**tabela hash** ($O(1)$ esperado)
muitas buscas, quase nenhuma atualização	**vetor ordenado** + busca binária
k-ésimo menor / mediana uma vez	**seleção linear** (BFPRT/quickselect), sem ordenar
os k maiores de um fluxo	heap de mínimo de tamanho k: $O(n \log k)$

Padrões a reconhecer em código: laço que procura o máximo de uma lista e o remove ($\Theta(n)$ por iteração $\Rightarrow$ heap); laço que ordena a lista inteira a cada inserção ($\Rightarrow$ heap ou AVL); laço que

percorre todos os elementos para saber se dois estão no mesmo grupo (⇒ Union-Find); ordenar para depois pegar só o k-ésimo (⇒ seleção $O(n)$); ordenar inteiros pequenos com `sort` genérico (⇒ counting/radix).

Pontos-chave

Comparação ⇒ $\Omega(n \log n)$ (árvore de decisão, $2^h \geq n!$). Merge: estável, $\Theta(n)$ extra. Quick: in-place, médio $n \log n$, pior n^2. Heap: in-place, sempre $n \log n$, não estável. Counting $\Theta(n+k)$ estável; Radix $\Theta(d(n+b))$ precisa de estável por dígito (LSD); Bucket $\Theta(n)$ esperado com uniforme.

7 Seleção: o algoritmo BFPRT (mediana das medianas)

Problema da seleção: dado um conjunto A de n números e k, achar o k-ésimo menor. Casos: mínimo ($k = 1$), máximo ($k = n$), mediana inferior $\lfloor (n+1)/2 \rfloor$, superior $\lceil (n+1)/2 \rceil$.

Possibilidades. (1) Ordenar: $O(n \log n)$. (2) Retirar o mínimo k vezes: $O(kn)$ ($O(n^2)$ para a mediana). (3) **Quickselect** (pivoteamento como no quicksort, mas recursão em *um* lado só): $O(n)$ esperado, $O(n^2)$ no pior caso. (4) **BFPRT** (Blum, Floyd, Pratt, Rivest, Tarjan): $O(n)$ no **pior caso**.

Quickselect (exercício do slide: tempo esperado $O(n)$, mais simples)

Função QuickSelect(S, k)
  escolha um pivô m **aleatório** em S
  $S_1 := \{s < m\}$; $S_2 := \{s = m\}$; $S_3 := \{s > m\}$
  se $|S_1| \geq k$ **então**
    retorna QuickSelect(S_1, k)
  senão se $|S_1| + |S_2| \geq k$ **então**
    retorna m
  senão
    retorna QuickSelect$(S_3,\ k - |S_1| - |S_2|)$

Esperado: com probabilidade $\geq 1/2$ o pivô cai no "meio" (entre os quartis) e o lado mantido tem $\leq 3n/4$. Agrupe as chamadas em *fases*: a fase termina quando o tamanho cai para $\leq 3n/4$. O número de chamadas numa fase é dominado por uma geométrica de parâmetro $1/2$, logo tem esperança ≤ 2, e cada chamada custa $\leq cn$. Assim $E[T(n)] \leq E[T(3n/4)] + 2cn$, e somando a série geométrica, $E[T(n)] \leq 2cn \sum_{t \geq 0} (3/4)^t = 8cn = O(n)$.

7.1 O algoritmo

A ideia é garantir um pivô *sempre* bom: a **mediana das medianas** de grupos de 5.

Seleção(S, k) – BFPRT (slides)

Função Seleção(S, k)
  se $|S| \leq 49$ **então**
    ordene S (mergesort) e **retorna** o k-ésimo menor
  divida S em $\lceil |S|/5 \rceil$ grupos de (até) 5 elementos
  determine a mediana de cada grupo (ordenar 5 elementos: $O(1)$ cada)
  $M :=$ conjunto das medianas; $m :=$ Seleção$(M, \lceil |M|/2 \rceil)$ % recursão 1: tamanho $n/5$
  $S_1 := \{s \in S : s < m\}$; $S_2 := \{s \in S : s = m\}$; $S_3 := \{s \in S : s > m\}$
  se $|S_1| \geq k$ **então**
    retorna Seleção(S_1, k) % recursão 2: tamanho $\leq 3n/4$
  senão se $|S_1| + |S_2| \geq k$ **então**
    retorna m
  senão

retorna SELEÇÃO($S_3,\ k - |S_1| - |S_2|$)

Armadilhas comuns

Num dos slides o teste aparece como "se $|S_1| = k$ então o k-ésimo está em S_1"; o correto é $|S_1| \geq k$ (como no pseudocódigo da página seguinte). E, ao ir para S_3, o índice muda para $k - |S_1| - |S_2|$.

7.2 Análise

Lema 7.1. *$|S_1| \leq n - 3\lfloor n/10 \rfloor$ e $|S_3| \leq n - 3\lfloor n/10 \rfloor$; para $n \geq 50$, isso é $\leq 3n/4$.*

Demonstração. Das $\lfloor n/5 \rfloor$ medianas, pelo menos metade, $\lfloor \lfloor n/5 \rfloor / 2 \rfloor = \lfloor n/10 \rfloor$, são $\geq m$. Cada uma dessas vem de um grupo que tem mais 2 elementos $\geq$ ela (logo $\geq m$). Assim, pelo menos $3\lfloor n/10 \rfloor$ elementos são $\geq m$, ou seja, $|S_2| + |S_3| \geq 3\lfloor n/10 \rfloor$ e $|S_1| \leq n - 3\lfloor n/10 \rfloor$. Simetricamente para S_3. Para $n - 3\lfloor n/10 \rfloor \leq 3n/4$ basta $12\lfloor n/10 \rfloor \geq n$, isto é, $2\lfloor n/10 \rfloor \geq n - 10\lfloor n/10 \rfloor$; como o lado direito é ≤ 9, basta $\lfloor n/10 \rfloor \geq 5$, ou seja, $n \geq 50$ (por isso o caso base é $n \leq 49$). □

Teorema 7.2. *$T(n) \leq 20cn$, logo a BFPRT é $O(n)$ no pior caso.*

Demonstração. $T(n) = O(1)$ para $n \leq 49$ e $T(n) \leq T(n/5) + T(3n/4) + cn$ para $n > 49$ (a partição e as medianas dos grupos são lineares). Por indução forte (com a base ajustada, p.ex. $T(n) \leq 6n \leq 20cn$ pelo mergesort para $n < 50$):

$$T(n) \leq 20c\,\frac{n}{5} + 20c\,\frac{3n}{4} + cn = 4cn + 15cn + cn = 20cn.$$ □

Por que grupos de 5? Para linearidade, a soma dos tamanhos dos subproblemas precisa ser uma fração < 1 de n: $\frac{n}{5} + \frac{3n}{4} = \frac{19n}{20} < n$. Com grupos de 3: pivô garante só $\approx n/3$ de cada lado, $T(n) = T(n/3) + T(2n/3) + cn$ e $\frac{1}{3} + \frac{2}{3} = 1 \Rightarrow \Theta(n \log n)$. Grupos de 7, 9, ... também funcionam (constantes diferentes).

Aplicação. Quicksort com a mediana (BFPRT) como pivô: $T(n) = 2T(n/2) + O(n) = O(n \log n)$ no **pior** caso; na prática é mais lento que o quicksort aleatório por causa das constantes.

Pontos-chave

Seleção em $O(n)$: grupos de 5, mediana das medianas como pivô, $\geq 3\lfloor n/10 \rfloor$ de cada lado, $T(n) \leq T(n/5) + T(3n/4) + cn \leq 20cn$. Não é preciso ordenar para achar o k-ésimo!

8 Divisão e conquista

Paradigma: (1) *Divisão*: quebrar o problema Π em subproblemas menores **do mesmo tipo** $\Pi_1, \ldots, \Pi_k$; (2) *Conquista*: resolvê-los recursivamente (casos pequenos diretamente); (3) *Combinação*: juntar as soluções parciais com um procedimento "simples". Busca-se **bom balanceamento** (partes de tamanhos parecidos). Funciona bem quando os subproblemas são **disjuntos**; quando eles se repetem muito, a técnica certa é programação dinâmica (§11). Exemplos do curso: mergesort, quicksort, BFPRT, contagem de inversões, Strassen, subvetor máximo.

Q5: modelo de resposta para "escreva o pseudocódigo por divisão e conquista"

1. **Assinatura**: RESOLVE(V, ini, fim) trabalhando num intervalo (evita copiar vetores). Diga o que a função **retorna** (às vezes mais de um valor: soma *e* índices; contagem *e* vetor ordenado).
2. **Caso base**: $ini = fim$ (ou $n \leq$ constante): resposta direta.

3. **Divisão**: $meio := \lfloor (ini + fim)/2 \rfloor$.
4. **Conquista**: duas chamadas recursivas, $[ini, meio]$ e $[meio + 1, fim]$.
5. **Combinação**: o que "atravessa" o meio (a parte criativa); geralmente $O(n)$.
6. **Complexidade**: escreva a recorrência e resolva (árvore de recursão ou Teorema Mestre, §1.4): $2T(n/2) + \Theta(n) = \Theta(n \log n)$.
7. **Correção** (uma frase): toda solução está inteira à esquerda, inteira à direita ou cruza o meio; cada caso é tratado.

8.1 Contagem de inversões (aula de 11/09)

Dada $a_1, \ldots, a_n$, um par $i < j$ é uma **inversão** se $a_i > a_j$ (métrica de similaridade entre rankings: minha ordem $1, \ldots, n$ vs. a sua). Força bruta: $\Theta(n^2)$. **D&C:** divide em metades A e B; conta recursivamente as inversões dentro de cada uma; **combina** contando os pares (a, b) com $a \in A$, $b \in B$, $a > b$. Se A e B estão *ordenadas*, isso é fácil durante a intercalação: ao comparar a_i e b_j, se $a_i \leq b_j$, a_i não é invertido com nenhum elemento restante de B; se $a_i > b_j$, então b_j é invertido com **todos os elementos restantes de** A. Por isso a função também devolve o vetor ordenado (é um mergesort "instrumentado").

Contagem de inversões (aula + completado)

```
Função CalculaInversões(V, ini, fim)                    % conta e deixa V[ini..fim] ordenado
    se ini = fim então
        retorna 0
    meio := ⌊(ini + fim)/2⌋
    a := CalculaInversões(V, ini, meio)
    b := CalculaInversões(V, meio + 1, fim)
    c := MergeCalculandoInversões(V, ini, meio, fim)
    retorna a + b + c
Função MergeCalculandoInversões(V, ini, meio, fim)
    i := ini; j := meio + 1; k := 1; cont := 0
    enquanto i ≤ meio e j ≤ fim faça
        se V[i] ≤ V[j] então
            W[k] := V[i]; i := i + 1
        senão
            W[k] := V[j]; j := j + 1
            cont := cont + (meio − i + 1)               % V[j] é menor que todos os restantes da esquerda
        k := k + 1
    copie os restantes de V[i..meio] e de V[j..fim] para W; copie W para V[ini..fim]
    retorna cont
```

$T(n) = 2T(n/2) + \Theta(n) \Rightarrow \Theta(n \log n)$. *Cuidado:* usar "$\leq$" (iguais não são inversão) e somar $meio - i + 1$ (quantos restam em A), não 1. Ex. conferido: $1, 5, 4, 8, 10, 2, 6, 9, 12, 11, 3, 7$ tem **22** inversões.

8.2 Subvetor contíguo de soma máxima (Teste 2, Exercício 1)

Dado $A[1..n]$ (inteiros, podem ser negativos), achar $i \leq j$ maximizando $A[i] + \cdots + A[j]$ (subvetor **não vazio**). **Ideia:** o melhor subvetor de $A[ini..fim]$ está (i) inteiro na metade esquerda, (ii) inteiro na direita, ou (iii) **cruza o meio**, i.e., é $A[i..meio] \cup A[meio + 1..j]$. O caso (iii) se resolve em tempo linear: melhor sufixo da esquerda terminando em $meio$ + melhor prefixo da direita começando em $meio + 1$.

Subvetor de soma máxima por D&C (retorna soma, início, fim)

Função MaxSub(A, ini, fim)
  se $ini = fim$ **então**
    retorna $(A[ini],\ ini,\ ini)$ % não vazio: um elemento
  $meio := \lfloor (ini + fim)/2 \rfloor$
  $(s_E, i_E, j_E) :=$ MaxSub$(A, ini, meio)$
  $(s_D, i_D, j_D) :=$ MaxSub$(A, meio + 1, fim)$
  $(s_C, i_C, j_C) :=$ MaxCruzado$(A, ini, meio, fim)$
  retorna a tripla de maior soma entre (s_E, i_E, j_E), (s_D, i_D, j_D), (s_C, i_C, j_C)
Função MaxCruzado$(A, ini, meio, fim)$
  $soma := 0$; $melhorE := -\infty$
  para $i := meio$ **decrescendo até** ini **faça**
    $soma := soma + A[i]$
    se $soma > melhorE$ **então**
      $melhorE := soma$; $iMax := i$
  $soma := 0$; $melhorD := -\infty$
  para $j := meio + 1$ **até** fim **faça**
    $soma := soma + A[j]$
    se $soma > melhorD$ **então**
      $melhorD := soma$; $jMax := j$
  retorna $(melhorE + melhorD,\ iMax,\ jMax)$

- **Complexidade:** $T(n) = 2T(n/2) + \Theta(n) \Rightarrow \Theta(n \log n)$ (Cormen §4.1).
- **Todos negativos:** funciona, pois o caso base devolve um elemento e o cruzado usa pelo menos $A[meio]$ e $A[meio + 1]$; o máximo das três opções é o maior elemento. Ex. 2 do teste: $[-8, -3, -6, -2, -5, -4] \to -2$.
- Ex. 1 do teste (conferido): $[-2, 1, -3, 4, -1, 2, 1, -5, 4] \to$ soma 6, subvetor $[4, -1, 2, 1]$. *O enunciado indexa a partir de 0* (posições 3 a 6); com índices a partir de 1, posições 4 a 7.
- (Fora de D&C: o algoritmo de Kadane resolve em $\Theta(n)$ mantendo o melhor sufixo terminando em cada posição. Também existe D&C linear devolvendo (soma total, melhor prefixo, melhor sufixo, melhor) de cada metade: $T(n) = 2T(n/2) + O(1) = \Theta(n)$.)

8.3 Multiplicação de matrizes: Strassen (Teste 2, Exercício 2)

Produto por blocos ingênuo: 8 multiplicações de $n/2 \times n/2$, $T(n) = 8T(n/2) + \Theta(n^2) = \Theta(n^3)$. Strassen usa **7**: $T(n) = 7T(n/2) + \Theta(n^2) = \Theta(n^{\log_2 7}) \approx \Theta(n^{2{,}807})$ (Teorema Mestre caso 1, pois $n^2 = O(n^{\log_2 7 - \varepsilon})$).

Strassen(A, B, n) – n potência de 2; Soma/Subtrai são dadas

Função Strassen(A, B, n)
  se $n = 1$ **então**
    retorna $[\, A[1,1] \cdot B[1,1] \,]$
  particione $A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}$, $B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$ em blocos $\frac{n}{2} \times \frac{n}{2}$
  $M_1 :=$ Strassen$(A_{11} + A_{22},\ B_{11} + B_{22},\ n/2)$
  $M_2 :=$ Strassen$(A_{21} + A_{22},\ B_{11},\ n/2)$
  $M_3 :=$ Strassen$(A_{11},\ B_{12} - B_{22},\ n/2)$
  $M_4 :=$ Strassen$(A_{22},\ B_{21} - B_{11},\ n/2)$
  $M_5 :=$ Strassen$(A_{11} + A_{12},\ B_{22},\ n/2)$
  $M_6 :=$ Strassen$(A_{21} - A_{11},\ B_{11} + B_{12},\ n/2)$
  $M_7 :=$ Strassen$(A_{12} - A_{22},\ B_{21} + B_{22},\ n/2)$
  $C_{11} := M_1 + M_4 - M_5 + M_7$; $\quad C_{12} := M_3 + M_5$
  $C_{21} := M_2 + M_4$; $\quad C_{22} := M_1 - M_2 + M_3 + M_6$

retorna $\begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix}$

São 18 somas/subtrações de matrizes $\frac{n}{2} \times \frac{n}{2}$ ($\Theta(n^2)$) por chamada. Na prática usa-se o método clássico abaixo de um certo n (constantes maiores, menos estabilidade numérica).

8.4 Outros exemplos clássicos de D&C (treino para a Q5)

Busca binária, potência rápida e máximo/mínimo

Função BuscaBin(V, ini, fim, x) % V ordenado; $T(n) = T(n/2) + O(1) = O(\log n)$
  se $ini > fim$ **então**
    retorna -1
  $meio := \lfloor (ini + fim)/2 \rfloor$
  se $V[meio] = x$ **então**
    retorna $meio$
  senão se $x < V[meio]$ **então**
    retorna BuscaBin($V, ini, meio - 1, x$)
  senão
    retorna BuscaBin($V, meio + 1, fim, x$)

Função Pot(a, n) % a^n com $O(\log n)$ multiplicações
  se $n = 0$ **então**
    retorna 1
  $t := \text{Pot}(a, \lfloor n/2 \rfloor)$
  se n par **então**
    retorna $t \cdot t$
  senão
    retorna $t \cdot t \cdot a$

Função MaxMin(V, ini, fim) % $\approx 3n/2$ comparações (contra $2n - 2$ do ingênuo)
  se $ini = fim$ **então**
    retorna $(V[ini], V[ini])$
  se $fim = ini + 1$ **então**
    retorna (max, min) de $V[ini], V[fim]$ com 1 comparação
  $meio := \lfloor (ini + fim)/2 \rfloor$
  $(M_1, m_1) :=$ MaxMin($V, ini, meio$); $(M_2, m_2) :=$ MaxMin($V, meio + 1, fim$)
  retorna $(\max\{M_1, M_2\},\ \min\{m_1, m_2\})$

Elemento majoritário (aparece $> n/2$ vezes), $T(n) = 2T(n/2) + \Theta(n) = \Theta(n \log n)$

Função Maj(V, ini, fim) % retorna o majoritário de $V[ini..fim]$ ou NENHUM
  se $ini = fim$ **então**
    retorna $V[ini]$
  $meio := \lfloor (ini + fim)/2 \rfloor$
  $a :=$ Maj($V, ini, meio$); $b :=$ Maj($V, meio + 1, fim$)
  para $c \in \{a, b\}$, $c \neq$ NENHUM **faça**
    se c ocorre mais de $(fim - ini + 1)/2$ vezes em $V[ini..fim]$ **então**
      retorna c
  % contagem $O(n)$
  retorna NENHUM

Correção: se x é majoritário no todo, é majoritário em pelo menos uma das metades (senão ocorreria $\leq n/2$ vezes).

Mais ideias de D&C. *Karatsuba* (inteiros de n dígitos com 3 produtos de $n/2$: $\Theta(n^{\log_2 3}) \approx n^{1{,}585}$); *par de pontos mais próximos* no plano ($\Theta(n \log n)$: divide pela mediana de x, combina olhando a faixa central com no máximo 7 vizinhos por ponto); *pico em vetor unimodal* e *raiz de função monótona* (busca binária, $O(\log n)$); *contar ocorrências de x em vetor ordenado* (duas buscas binárias).

Erros frequentes em pseudocódigo de D&C

- Esquecer o caso base ou fazê-lo errado (ex.: subvetor *vazio* permitido quando o enunciado pede não vazio).
- Dividir em $[ini, meio-1]$ e $[meio, fim]$ com $meio = \lfloor (ini+fim)/2 \rfloor$: com 2 elementos gera recursão infinita. Use $[ini, meio]$ e $[meio+1, fim]$.
- Combinação que olha só um elemento de cada lado (o subvetor cruzado precisa do *melhor sufixo* e do *melhor prefixo*).
- Não devolver a informação extra necessária (índices; vetor ordenado nas inversões).
- Não dizer a complexidade (o enunciado sempre pede!).

Pontos-chave

D&C = dividir (meio), conquistar (2 chamadas), combinar (o que cruza o meio). $2T(n/2)+\Theta(n) = \Theta(n \log n)$; $T(n/2)+O(1) = \Theta(\log n)$; $7T(n/2)+\Theta(n^2) = \Theta(n^{2{,}807})$; $T(n/5)+T(3n/4)+\Theta(n) = \Theta(n)$.

9 Complementos

9.1 Tabelas hash (pesquisa)

Complemento – pesquisa proposta em 02/09, entrega 19/09 (Cormen, cap. 11; Szwarcfiter, cap. 10)

Motivação: implementar um *dicionário* (inserir, buscar, remover por chave) com acesso "direto". Endereçamento direto (vetor indexado pela chave) é $O(1)$ mas exige memória do tamanho do universo de chaves (infinito ou enorme). Uma **função hash** $h : U \to \{0, \ldots, m-1\}$ transforma a chave num endereço de uma tabela de m posições.
Funções clássicas: *divisão* $h(k) = k \bmod m$ (m primo longe de potências de 2); *multiplicação* $h(k) = \lfloor m\,(kA \bmod 1) \rfloor$ ($A \approx (\sqrt{5}-1)/2$); *dobra/meio do quadrado*; para strings, polinomial $h(s) = \sum s_i\, b^i \bmod m$; *hashing universal* (escolher h aleatoriamente de uma família, p.ex. $((ak+b) \bmod p) \bmod m$), que garante bom caso esperado para qualquer entrada.
Colisões ($h(k_1) = h(k_2)$, $k_1 \neq k_2$) são inevitáveis (princípio da casa dos pombos: $|U| > m$). Tratamento:

- **Encadeamento exterior:** cada posição aponta para uma lista das chaves com aquele endereço.
- **Encadeamento interior:** as listas usam posições da própria tabela (uma zona de colisões, ou ponteiros entre posições livres).
- **Endereçamento aberto:** tudo fica na tabela; em colisão, sonda-se uma sequência de posições: *linear* $h(k)+i$ (agrupamento primário), *quadrática* $h(k)+c_1 i+c_2 i^2$, *duplo hashing* $h_1(k)+i\,h_2(k)$. Remoção exige marca "removido".

Complexidade: com fator de carga $\alpha = n/m$ e hash uniforme, busca em $\Theta(1+\alpha)$ esperado com encadeamento, e $\leq \frac{1}{1-\alpha}$ sondagens esperadas (busca sem sucesso) no endereçamento aberto; mantendo $\alpha = O(1)$ (redimensionando a tabela, custo amortizado $O(1)$), as operações são $O(1)$ esperado. **Pior caso:** $\Theta(n)$ (todas as chaves no mesmo endereço). Hash não mantém ordem (sem mínimo, sucessor, intervalos — aí usa-se AVL/rubro-negra).

10 Banco de questões com respostas

10.1 Testes aplicados

Teste 1 (heaps, Union-Find, AVL).

1. Aeroporto: filas de prioridade por setor, fusões muito frequentes, alterações raras, tempo *previsível por operação*. Qual heap? ⇒ **binomial** (§3.4).
2. Central médica: $2 \cdot 10^6$ chamadas de 16 bytes, limite de 40 MB, ponteiros de 8 bytes, sem fusão, poucas alterações. ⇒ **heap binário** (§3.4, conta de memória).
3. Supercomputador: $5 \cdot 10^6$ inserções/remoções e $8 \cdot 10^7$ aumentos de prioridade, custo total importa. ⇒ **Fibonacci** (§3.4).
4. Plataforma com 10^7 usuários: 10^5 uniões e 10^8 consultas; árvore vs. lista; estratégia de união. ⇒ Union-Find, lista com ponteiro ao representante (FIND $O(1)$), união "menor na maior" (§4.4).
5. AVL: árvore $50(30(20(10,25),40),70(60,80))$, inserir $5, 15, 35, 45$, identificar desregulados e rotações; ABB vs. AVL ⇒ Exemplo 5.5.

Teste 2 (ordenação e D&C, 2h, 3 questões).

1. Subvetor contíguo de soma máxima por D&C, com posições e complexidade ⇒ §8.
2. Strassen: algoritmo recursivo com as 7 multiplicações ⇒ §8.
3. Escolher Counting/Radix/Bucket/Merge/Quick para 5 cenários ⇒ §6.4.

10.2 Lista – heaps binários, binomiais e de Fibonacci

Obs.: a numeração segue a da lista, que **pula o exercício 9** (vai do 8 ao 10).

1. *Verificar se são heaps.* (a) 33 32 28 31 26 29 25 30 27: **não** — posição 6 (29) tem pai na posição 3 (28) e $29 > 28$. (b) 33 32 28 31 29 26 25 30 27: **sim** (todas as 8 relações pai–filho valem).
2. *Troca com $i < j$, $s_i < s_j$ gera heap?* Falso: $[10, 9, 1, 8, 7, 0, 0, 6]$, troca das posições 3 e 4 (§2.7).
3. *Idem com $s_i > s_j$.* Falso: $[10, 9]$, troca das posições 1 e 2.
4. *Número de permutações que são min-heap.* $H(n) = \binom{n-1}{L} H(L) H(n-1-L)$; valores $1, 1, 2, 3, 8, 20, 80, 210, \ldots$ (§2.7).
5. *Construção linear de* 18 25 41 34 14 10 52 50 48. $[52, 50, 41, 48, 14, 10, 18, 34, 25]$ (traço em §2).
6. *No máximo $\lceil n/2^j \rceil$ nós de altura j.* Lema 2.3.
7. *Construir heap binomial em $O(n)$.* n inserções sucessivas: o custo total é o do contador binário, $\sum_k n/2^k \leq 2n$ links (§3.1). Alternativa: formar pares $B_0 + B_0 \to B_1$, depois pares de B_1, etc., como uma soma binária de n uns.
8. *Alturas para o pior caso da inserção num heap com k árvores.* Quando o novo B_0 provoca "vai um" em todas as posições: o heap contém $B_0, B_1, \ldots, B_{k-1}$ ($n = 2^k - 1$, todos os bits 1), isto é, alturas $0, 1, \ldots, k-1$ em arestas ($1, \ldots, k$ em níveis, convenção do Szwarcfiter). São feitos k links e o resultado é uma única B_k.
10. *Remoção do máximo no pior caso.* A remoção custa percorrer as k raízes, gerar os h filhos da raiz removida e fundir H' (as outras $k-1$ árvores) com $H'' = \{B_0, \ldots, B_{h-1}\}$. O máximo de links ocorre quando H' também contém $B_0, \ldots, B_{h-1}$ (cada posição gera "vai um") e h é o maior possível: $H = \{B_0, B_1, \ldots, B_{k-1}\}$ **com o máximo na raiz de** B_{k-1}, logo $h = k - 1$ (em arestas).
11. $k \geq \min\{k_1, k_2\}$? **Falso**: H_1 com $3 = 11_2$ nós ($k_1 = 2$), H_2 com $5 = 101_2$ ($k_2 = 2$); a fusão tem $8 = 1000_2$ nós, $k = 1 < 2$.

12. $k \leq k_1 + k_2$? **Verdadeiro**: a união inicial tem $k_1 + k_2$ árvores e cada link transforma duas árvores em uma; logo $k = k_1 + k_2 - \#\text{links} \leq k_1 + k_2$.
13. *Altura mínima e máxima de heap de Fibonacci em função do posto k.* Hipótese do exercício (Szwarcfiter §9.5): árvores de Fibonacci de posto k, formadas por subordinações de árvores de mesmo posto e cortes controlados por marcas em nós *não raiz*. **Mínima**: pelo lema $\text{grau}(y_i) \geq i - 2$, a menor altura $A(k)$ satisfaz $A(k) = 1 + A(k-2)$, $A(0) = 0$, $A(1) = 1$, logo $A(k) = \lceil k/2 \rceil$ (atingida pelas árvores de Fibonacci mínimas). **Máxima**: k, atingida pela própria B_k (por indução: subordinar duas árvores de posto h e altura $\leq h$ dá posto $h+1$ e altura $\leq h+1$; cortes não aumentam a altura nem mudam o posto da raiz). Alturas em arestas. (Observação fora do escopo do exercício: numa implementação geral, em que a raiz pode perder filhos livremente, a altura não é limitada por uma função do posto — o que é sempre $O(\log n)$ é o posto/grau.)

14–15. *Algoritmos dos heaps binomiais e de Fibonacci.* §3.1 e §3.3.

16. *Heap binário sem recursão.*

Subir e descer iterativos (ex. 16; Szwarcfiter ex. 6.7–6.8)

```
Procedimento SUBIR(i)
    enquanto i > 1 e T[i].chave > T[⌊i/2⌋].chave faça
        T[i] ⇔ T[⌊i/2⌋]; i := ⌊i/2⌋
Procedimento DESCER(i, n)
    enquanto 2i ≤ n faça
        j := 2i
        se j < n e T[j + 1].chave > T[j].chave então
            j := j + 1
        se T[i].chave < T[j].chave então
            T[i] ⇔ T[j]; i := j
        senão
            pare
```

Inserção, remoção, alteração e construção usam exatamente os mesmos esquemas da §2 com essas versões.

10.3 Lista – Union-Find e AVL

1. União por rank sem compressão: altura $\leq \lfloor \log n \rfloor$ e sequência que atinge $\Theta(\log n)$ ⇒ Teo. 4.1 e exemplos da §4.
2. Só compressão: sequência que gera árvore alta e custo $O(m \log n)$ ⇒ §4.
3. Remoção em AVL com rotação em todos os ancestrais ⇒ árvore de Fibonacci T_7, remover a folha 33 (§5).
4. Fusão de duas AVL ⇒ junção pela espinha direita $O(\log n)$, ou intercalação dos percursos em ordem $O(n+m)$ (§5).
5. AVL como conjuntos disjuntos: FIND $O(\log n)$, UNION $O(k \log n)$ ⇒ §5.

10.4 Monitoria de revisão: respostas curtas

- **UF-1.** Objetivo: manter partição em conjuntos disjuntos com consultas "mesmo conjunto?" e uniões. MAKE-SET cria $\{x\}$ ($pai[x] = x$); FIND sobe até a raiz; UNION liga uma raiz na outra. Cada elemento armazena o **pai** (e a raiz, o rank). União por rank: liga o de menor rank sob o de maior; altura $\leq \log n$.
- **AVL-1.** Balanceada: altura $O(\log n)$ em toda subárvore. AVL: $|h_D - h_E| \leq 1$ em todo nó. Toda AVL é balanceada (Teo. 5.3), nem toda balanceada é AVL. Fator de balanço $\text{bal}(v) = h_D(v) - h_E(v)$.

- **AVL-2** (inserção à esquerda de p): bal $= 1 \to 0$, altura não muda, para; bal $= 0 \to -1$, altura aumenta, continua; bal $= -1 \to -2$, rotação e para.
- **AVL-3.** bal$(p) = -2$, bal$(u) = -1$: simples à direita. bal$(p) = -2$, bal$(u) = +1$: dupla à direita (esquerda em u, direita em p). Diferença: no primeiro o excesso está "em linha" (esquerda-esquerda) e uma rotação resolve; no segundo está em "zigue-zague" (esquerda-direita) e é preciso primeiro alinhar.
- **Heap-1.** Definição $s_i \leq s_{\lfloor i/2 \rfloor}$; seleção $O(1)$ porque o máximo está na posição 1 (Prop. 2.6); custos: $O(1)$, $O(\log n)$, $O(\log n)$, $O(\log n)$, $O(n)$.
- **Binomial (Q6–Q8, Q11).** B_k: duas B_{k-1} ligadas; 2^k nós; grau da raiz k; o heap é a lista de raízes com ordens distintas (bits de n). Link de duas B_2 de raízes 10 e 15 (min-heap): raiz 10, a de 15 vira sua filha; resultado B_3 com 8 nós e grau 3. Mínimo: percorre as $O(\log n)$ raízes (ou ponteiro). União $O(\log n)$: só percorre as listas de raízes (não os n nós), fazendo links como numa soma binária.
- **Fibonacci (Q9, Q10, Q12).** Diferença estrutural: permite várias árvores de mesma ordem (preguiçoso), listas circulares duplas, marcas. $O(1)^*$: inserção, união, aumento de prioridade (*decrease-key* no min-heap), seleção. Remoção do extremo: $O(\log n)^*$. Aumentar prioridade pode violar a ordem com o pai; corta-se x e o leva à lista de raízes; cortes em cascata sobem pelos pais marcados; custo $O(1)^*$ porque cada corte em cascata é pago pela operação que marcou o nó (potencial $t + 2m$).

Checklist final para a AV1

1. **AVL**: inserir/remover desenhando a árvore e os balanços a cada passo; rotacionar no desregulado *mais profundo*; saber os 4 casos e o pseudocódigo (papel de h e de pt).
2. **Union-Find**: tabela de implementações e cenários (listar membros, rollback, offline, Kruskal); provar altura $\leq \log n$ com rank.
3. **Heap**: provas por caminho/transitividade, índices e indução; construção $O(n)$; contraexemplos pequenos.
4. **Escolha**: identificar a operação dominante e o tipo da chave; justificar com complexidade, memória, previsibilidade (pior caso vs. amortizado), estabilidade.
5. **D&C**: caso base, divisão, duas chamadas, combinação do "cruzamento", recorrência resolvida.

Parte II
Conteúdo pós-AV1 (AV2)

11 Programação dinâmica

Motivação (slides). Na decomposição recursiva, **subproblemas idênticos** podem aparecer muitas vezes; resolvendo-os de novo a cada aparição, o algoritmo pode ser **exponencial** mesmo que o número de subproblemas *distintos* seja polinomial. **PD**: uma tabela com uma entrada por subproblema distinto; cada um é resolvido uma única vez, e as próximas consultas só leem a tabela.

- **D&C × PD:** D&C é bom quando os subproblemas são **disjuntos** (mergesort); quando eles **compartilham** subproblemas, troca-se o *top-down* por *bottom-up*: resolver dos menores para os maiores, de modo que, ao visitar um subproblema, todos de que ele depende já estejam prontos.
- **Subestrutura ótima:** uma solução ótima é composta de soluções ótimas de subproblemas (prova típica: "recorta e cola" — se a parte não fosse ótima, troque-a por uma melhor e obtenha uma solução global melhor, contradição).
- **Sobreposição de subproblemas:** a recursão ingênua revisita os mesmos subproblemas (é isso que faz a tabela valer a pena).
- **Memorização** (*top-down*): a recursão original + "se já calculei, devolvo da tabela". **Bottom-up**: laços na ordem certa. Mesma complexidade; o bottom-up evita pilha de recursão e às vezes permite economizar memória.

Fibonacci: recursão ingênua $\Theta(\varphi^n)$, memorização $O(n)$, bottom-up $O(n)$ e memória $O(1)$

```
Função FibMemo(n)                                   % F[] inicializado com −1
    se F[n] = −1 então
        se n ≤ 1 então
            F[n] := n
        senão
            F[n] := FibMemo(n − 1) + FibMemo(n − 2)
    retorna F[n]
Função FibBottomUp(n)
    a := 0; b := 1                                  % a = F_i, b = F_{i+1}
    para i := 1 até n faça
        (a, b) := (b, a + b)
    retorna a
```

Receita para "resolva por programação dinâmica" (vale para toda a parte de PD)

1. **Subproblema em palavras**: "$C(i,j)$ = tamanho da LCS de X_i e Y_j". Sem essa frase a recorrência não tem sentido.
2. **Decomposição**: o que a solução ótima faz com o *último* elemento (ou com a *primeira* decisão)? Cada caso vira um termo da recorrência.
3. **Recorrência + casos base**.
4. **Complexidade** = (número de subproblemas) × (custo de cada um): LCS $nm \times O(1)$; cadeia de matrizes $n^2 \times O(n)$; mochila $nW \times O(1)$.
5. **Ordem de preenchimento**: cada célula só depende de células já calculadas (linha a linha; por diagonais; por k crescente...).
6. **Onde está a resposta** ($C[n,m]$, $m[1,n]$, $M[W,n]$, $\max_i L[i]$...).

7. **Reconstrução**: guarde a escolha feita em cada célula ($s[i,j]$, "seta") ou recompute-a voltando pela tabela.

11.1 Subsequência comum máxima – LCS (aula 14, 29/09; Cormen §15.4)

Z é **subsequência** de $X = x_1 \dots x_n$ se seus símbolos aparecem em X na mesma ordem, **não necessariamente contíguos** (ex.: $6, 10, 12$ é subsequência de $3, 6, 9, 10, 12, 14, 6$, mas não de $3, 9, 10, 12, 14, 6$). Dadas X ($|X| = n$) e Y ($|Y| = m$), queremos uma subsequência comum de tamanho máximo. Força bruta: 2^n subsequências de X.

Decomposição (olhando os últimos símbolos; $X_i = x_1 \dots x_i$): se $x_n = y_m$, esse símbolo termina alguma LCS e o resto é uma LCS de X_{n-1}, Y_{m-1}; se $x_n \neq y_m$, a LCS não usa x_n ou não usa y_m. Com $C(i,j)$ = tamanho da LCS de X_i e Y_j:

$$C(i,j) = \begin{cases} 0 & i = 0 \text{ ou } j = 0 \\ 1 + C(i-1, j-1) & x_i = y_j \\ \max\{C(i-1,j),\, C(i,j-1)\} & x_i \neq y_j \end{cases} \qquad \text{resposta: } C(n,m).$$

A recursão direta é exponencial; há só $(n+1)(m+1)$ subproblemas distintos $\Rightarrow$ PD.

LCS bottom-up (slides) + reconstrução (monitoria) – $\Theta(nm)$ tempo e memória

```
Função LCS(X, Y)
    para j := 0 até m faça
        C[0, j] := 0
    para i := 1 até n faça
        C[i, 0] := 0
    para i := 1 até n faça
        para j := 1 até m faça
            se x_i = y_j então
                C[i, j] := 1 + C[i − 1, j − 1]
            senão
                C[i, j] := max{C[i − 1, j], C[i, j − 1]}
    i := n; j := m; Z := sequência vazia                    % reconstrução: O(n + m)
    enquanto i > 0 e j > 0 faça
        se x_i = y_j então
            coloque x_i no início de Z; i := i − 1; j := j − 1
        senão se C[i − 1, j] ≥ C[i, j − 1] então
            i := i − 1
        senão
            j := j − 1
    retorna Z                                               % |Z| = C[n, m]
```

Exemplo 11.1 (Monitoria: X = ABCBDAB, Y = BDCABA — conferido)**.**

		j=0	1	2	3	4	5	6
			B	D	C	A	B	A
i=0		0	0	0	0	0	0	0
1	A	0	0	0	0	1	1	1
2	B	0	**1**	1	1	1	2	2
3	C	0	1	1	**2**	2	2	2
4	B	0	1	1	2	2	**3**	3
5	D	0	1	2	2	2	3	3
6	A	0	1	2	2	3	3	**4**
7	B	0	1	2	2	3	4	4

$C[7,6] = 4$. A reconstrução (empate ⇒ sobe) passa pelas células em negrito: $(7,6) \uparrow (6,6)$ casa **A** $\rightarrow (5,5) \uparrow (4,5)$ casa **B** $\rightarrow (3,4) \leftarrow (3,3)$ casa **C** $\rightarrow (2,2) \leftarrow (2,1)$ casa **B**. Como cada símbolo entra no *início*, $Z =$ BCBA. As LCS de tamanho 4 são exatamente **BCAB, BCBA e BDAB**.

Ex. dos slides: $X = 5,2,1,3,9,10,14$ e $Y = 2,3,1,10$: tamanho 3, soluções $2,3,10$ e $2,1,10$. Ex. da lista APA (ex. 6): $X =$ dacdba, $Y =$ cadcda: tamanho **4** (acda ou dcda).

- **Só o tamanho** com memória $O(\min\{n,m\})$: cada linha só usa a anterior (duas linhas). Mas aí não dá para reconstruir voltando na tabela.
- **Todas as LCS** (lista l6, ex. 3): na reconstrução, quando $x_i \neq y_j$ e $C[i-1,j] = C[i,j-1]$, siga *os dois* caminhos (recursão) e elimine repetições; pode haver exponencialmente muitas.

11.2 Multiplicação de cadeia de matrizes (aula 15, 02/10; Cormen §15.2)

$M = M_1 \times M_2 \times \cdots \times M_n$, com M_i de dimensões $r_{i-1} \times r_i$ (consecutivas compatíveis). Multiplicar $(p \times q) \cdot (q \times s)$ custa $p\,q\,s$ multiplicações escalares. O produto é **associativo**: a ordem não muda o resultado, mas muda (muito) o custo. **Objetivo:** a parentização de custo mínimo.

Exemplo 11.2 (Aula de 02/10 — conferido)**.** $M_1 : 20 \times 50$, $M_2 : 50 \times 10$, $M_3 : 10 \times 1$, $M_4 : 1 \times 100$ ($r = (20,50,10,1,100)$). $M_1(M_2(M_3M_4))$: $1000 + 50\,000 + 100\,000 =$ **151 000**; $((M_1M_2)M_3)M_4$: $10\,000 + 200 + 2000 =$ **12 200**. *Nenhuma das duas é a ótima*: a PD abaixo dá $(M_1(M_2M_3))M_4$ com $500 + 1000 + 2000 =$ **3500**.

Força bruta é exponencial: o número de parentizações de n matrizes é o número de Catalan C_{n-1} $(1,1,2,5,14,42,132,\dots)$, que é $\Omega(4^n/n^{3/2})$.

Decomposição: a *última* multiplicação de $M_i \cdots M_j$ é $(M_i \cdots M_k) \times (M_{k+1} \cdots M_j)$ para algum $i \le k < j$; numa solução ótima, as duas metades também estão parentizadas de forma ótima (recorta e cola). Com $m[i,j]$ = número mínimo de multiplicações para calcular $M_i \cdots M_j$:

$$m[i,j] = \begin{cases} 0 & i = j \\ \min\limits_{i \le k < j} \{\, m[i,k] + m[k+1,j] + r_{i-1}\, r_k\, r_j \,\} & i < j \end{cases} \qquad \text{resposta: } m[1,n].$$

$O(n^2)$ subproblemas, cada um testa $O(n)$ valores de $k \Rightarrow \Theta(n^3)$ **tempo,** $\Theta(n^2)$ **memória**. Ordem: $m[i,j]$ depende de intervalos *mais curtos* ⇒ preencher **por diagonais** $d = j - i = 1,2,\dots,n-1$ (a diagonal principal é zero).

Cadeia de matrizes (quadro da aula, com a tabela s da lista l6, ex. 2)

```
Função OrdemÓtima(r_0, ..., r_n)
    para i := 1 até n faça
        m[i,i] := 0
    para d := 1 até n − 1 faça                          % "para cada diagonal"
        para i := 1 até n − d faça                      % "para cada linha da diagonal"
            j := i + d; m[i,j] := ∞
            para k := i até j − 1 faça
                q := m[i,k] + m[k+1,j] + r_{i−1} r_k r_j
                se q < m[i,j] então
                    m[i,j] := q; s[i,j] := k
    retorna m[1,n] e s
Procedimento Imprime(s, i, j)
    se i = j então
        imprima "M_i"
    senão
        imprima "("; Imprime(s, i, s[i,j]); Imprime(s, s[i,j] + 1, j); imprima ")"
Função Multiplica(A, s, i, j)                           % faz A_i ··· A_j na ordem ótima
    se i = j então
```

retorna A_i
retorna MULTIPLICA$(A, s, i, s[i,j]) \times$ MULTIPLICA$(A, s, s[i,j]+1, j)$

$m[i,j]$ $(s[i,j])$	$j=1$	2	3	4
$i=1$	0	10 000 (1)	1500 (1)	**3500 (3)**
2		0	500 (2)	5500 (3)
3			0	1000 (3)
4				0

Tabela do Exemplo 11.2. Como $s[1,4]=3$, a última multiplicação é $(M_1M_2M_3)\cdot M_4$; como $s[1,3]=1$, a primeira parte é $M_1(M_2M_3)$. Logo a ordem ótima é $((M_1(M_2M_3))M_4)$, com custo 3500.

- Monitoria: $r=(10,100,5,50)$: $m[1,2]=5000$, $m[2,3]=25\,000$, $m[1,3]=\min\{75\,000,\ 7500\}=$ **7500**, ordem $((M_1M_2)M_3)$.
- Lista l6, ex. 1 (Cormen 15.2-1), $r=(5,10,3,12,5,50,6)$: custo **2010**, ordem $((M_1M_2)((M_3M_4)(M_5M_6)))$.
- Exemplo do Cormen (Fig. 15.5): $r=(30,35,15,5,10,20,25)$, custo 15 125, ordem $((M_1(M_2M_3))((M_4M_5)M_6))$.

11.3 Mochila 0-1 (monitoria de PD)

n itens, item i com peso p_i e valor v_i (inteiros); capacidade W. Escolher $S\subseteq\{1,\dots,n\}$ com $\sum_{i\in S}p_i\le W$ maximizando $\sum_{i\in S}v_i$ (cada item entra no máximo uma vez). Com $M(w,i)=$ maior valor usando só os itens $1..i$ com capacidade w (o item i fica de fora, ou entra se couber):

$$M(w,i)=\begin{cases}0 & i=0\\ M(w,i-1) & p_i>w\\ \max\{M(w,i-1),\ v_i+M(w-p_i,\,i-1)\} & p_i\le w\end{cases}\qquad\text{resposta: } M(W,n).$$

Mochila 0-1 – $\Theta(nW)$ (pseudo-polinomial)

Função MOCHILA(p, v, n, W)
  para $w := 0$ **até** W **faça**
    $M[w,0] := 0$
  para $i := 1$ **até** n **faça**
    para $w := 0$ **até** W **faça**
      $M[w,i] := M[w,i-1]$
      se $p_i \le w$ **e** $v_i + M[w-p_i, i-1] > M[w,i]$ **então**
        $M[w,i] := v_i + M[w-p_i, i-1]$
  $w := W$ % reconstrução
  para $i := n$ **decrescendo até** 1 **faça**
    se $M[w,i] \neq M[w,i-1]$ **então**
      o item i está na solução; $w := w - p_i$
  retorna $M[W,n]$

Exemplo 11.3 (Monitoria: $W=4$; itens $(p,v)=(1,2),(3,7),(2,1)$ — conferido)**.**

$w\backslash i$	0	1	2	3
0	0	0	0	0
1	0	2	2	2
2	0	2	2	2
3	0	2	7	7
4	0	2	**9**	**9**

Ótimo: itens 1 e 2 (peso 4, valor **9**). *O slide da monitoria mostra 7 na última linha; está errado*: $M(4,2)=\max\{M(4,1),\ 7+M(1,1)\}=\max\{2,\ 9\}=9$.

- Monitoria: $W = 50$, $(p, v) = (10, 60), (20, 100), (30, 120)$: ótimo **220** (itens 2 e 3). O guloso pela razão v/p $(6, 5, 4)$ pega os itens 1 e 2 e obtém só 160 — guloso **não** resolve a mochila 0-1 (resolve a mochila *fracionária*: 240).
- Lista APA, ex. 7: $W = 100$, $(p, v) = (20, 50), (30, 70), (40, 90), (50, 120), (60, 140)$: ótimo **240** com os itens 1, 2 e 4 (peso exatamente 100).
- **Por que pseudo-polinomial?** O tempo $\Theta(nW)$ depende do *valor* de W, mas a entrada guarda W em $\log_2 W$ bits: com $W = 10^9$ (30 bits) o algoritmo faz 10^9 iterações por item. É exponencial no tamanho da entrada (a mochila é NP-difícil).
- **Memória** $O(W)$ (MC458, ex. 4): um único vetor $K[0..W]$; para cada item, w **decrescendo** de W até p_i: $K[w] := \max\{K[w], K[w - p_i] + v_i\}$. Com w *crescendo*, o item poderia ser reutilizado: vira a mochila *ilimitada*.

11.4 Lista de PD resolvida

1. **Cadeia de matrizes *maximizando* o custo tem subestrutura ótima? Sim.** Se a última multiplicação de uma parentização ótima (máxima) de $M_i \cdots M_j$ é no ponto k, o custo é $c(i..k) + c(k+1..j) + r_{i-1} r_k r_j$, e o último termo depende só de k, não de como cada metade foi parentizada. Se uma metade não fosse máxima, trocá-la por uma máxima aumentaria o total: contradição. Recorrência igual, com max; $\Theta(n^3)$.
2. **Maior subsequência crescente em $O(n^2)$.** $L[i]$ = tamanho da maior subsequência crescente *terminando em* a_i: $L[i] = 1 + \max\{L[j] : j < i,\ a_j < a_i\}$ ($\max \emptyset = 0$); resposta $\max_i L[i]$; guarde $ant[i]$ (o j escolhido) para reconstruir. Ex.: $5, 2, 8, 6, 3, 6, 9, 7$: $L = 1, 1, 2, 2, 2, 3, 4, 4$; resposta 4 $(2, 3, 6, 9)$. *Alternativa:* LCS entre a e a ordenado sem repetições. $O(n \log n)$ (l6, ex. 4): mantenha $d[\ell]$ = menor final possível de uma subsequência crescente de tamanho ℓ (vetor crescente); para cada a_i, busca binária pelo primeiro $d[\ell] \geq a_i$ e substitua (ou acrescente no fim).
3. **Maior subsequência palíndroma.** $P(i, j)$ para o trecho $s_i..s_j$: $P(i, i) = 1$, $P(i, i-1) = 0$; se $s_i = s_j$, $P(i, j) = 2 + P(i+1, j-1)$; senão $\max\{P(i+1, j), P(i, j-1)\}$. Preenche por tamanho do trecho; $\Theta(n^2)$. "character" → 5 (carac). (Equivale ao tamanho da LCS entre s e s invertida.)
4. **Caminho mais longo de s a t num DAG com pesos.** Ordene topologicamente (§12.2.1); $D(s) = 0$, demais $-\infty$; na ordem topológica, $D(v) = \max_{uv \in E}\{D(u) + w(u, v)\}$; guarde o predecessor. $\Theta(n + m)$. Num DAG todo caminho é simples, por isso a subestrutura funciona; em grafos gerais, caminho simples mais longo é NP-difícil (contém o caminho hamiltoniano).
5. **Mochila inteira (ilimitada).** $K(w) = \max\{0,\ \max_{i:\, p_i \leq w}\{K(w - p_i) + v_i\}\}$, $w = 0..W$; $\Theta(nW)$. Ex.: $(p, v) = (3, 5), (4, 7), (2, 3)$, $W = 10$: 17 (p.ex. $4 + 4 + 2$).
6. **Cortar uma string (mesma ideia da cadeia de matrizes).** Ordene os pontos e acrescente sentinelas $L_0 = 0$ e $L_{m+1} = n$. $C(i, j)$ = custo mínimo de fazer todos os cortes estritamente entre L_i e L_j: $C(i, i+1) = 0$ e

$$C(i, j) = (L_j - L_i) + \min_{i<k<j}\{C(i, k) + C(k, j)\} \quad \text{(o primeiro corte do trecho é em } L_k\text{)}.$$

   Resposta $C(0, m+1)$; guarde o k ótimo para listar a sequência de cortes (pré-ordem). $\Theta(m^3)$. Ex. do enunciado ($n = 20$, cortes $2, 8, 10$): ótimo **38**, cortando primeiro em 10 e depois em 2 e 8 (qualquer ordem); as seis ordens custam $50, 46, 40, 40, 38, 38$. (A "tora" da l6, ex. 14, é o mesmo problema: 10 m com cortes $2, 4, 7 \Rightarrow 20$.)

Armadilhas de PD

- **Subsequência ≠ subvetor**: LCS não exige contiguidade; o "subvetor de soma máxima" (§8) exige.
- Recorrência sem caso base, ou sem dizer *o que* $C(i,j)$ significa.
- Ordem de preenchimento errada: na cadeia de matrizes, preencher por *linhas* $i = 1, 2, \ldots$ usa $m[i+1, j]$ ainda não calculado. Vá por diagonais (ou i decrescente).
- Cadeia de matrizes: o termo é $r_{i-1}\, r_k\, r_j$ (não $r_i r_k r_j$); e k vai de i a $j-1$.
- Dizer que $O(nW)$ é polinomial. Não é: é pseudo-polinomial.
- Material da monitoria: a tabela da mochila ($M(4,2) = 9$, não 7) e a lista de LCS de ABCBDAB/BDCABA (são três: BCAB, **BCBA**, BDAB — e a reconstrução mostrada devolve justamente BCBA, já na ordem certa, porque insere no início).

Pontos-chave

PD = recorrência com subproblemas que se repetem + tabela. LCS $\Theta(nm)$; cadeia de matrizes $\Theta(n^3)$ por diagonais; mochila 0-1 $\Theta(nW)$ pseudo-polinomial; Floyd-Warshall $\Theta(n^3)$ (§13.2). Sempre: significado do subproblema, recorrência, base, ordem, complexidade, reconstrução.

12 Grafos: representação e buscas

$G = (V, E)$, $n = |V|$, $m = |E|$; $N(v)$ é a vizinhança de v; num **digrafo**, $N_{out}(v)$ são os vértices w com aresta $v \to w$. **Busca**: processo sistemático para visitar todos os vértices e arestas, extraindo informação da estrutura do grafo.

Representação	Memória	"$vw \in E$?"	Percorrer $N(v)$
Matriz de adjacência (ou de pesos W)	$\Theta(n^2)$	$O(1)$	$\Theta(n)$
Listas de adjacência	$\Theta(n+m)$	$O(\text{grau}(v))$	$\Theta(\text{grau}(v))$

Como cada busca percorre cada vizinhança uma vez, ela custa $\Theta(n+m)$ com listas e $\Theta(n^2)$ com matriz.

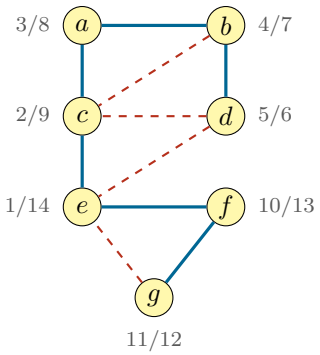
12.1 Busca em profundidade em grafos (aula 19, 16/10; slides 1–122; Cormen §22.3)

"*Esgotar o filho antes de esgotar o pai*": a próxima aresta visitada parte sempre do vértice **mais recente** da busca (pilha/recursão). Cada vértice recebe $PE(v)$ (profundidade de entrada) e $PS(v)$ (de saída), com um relógio global t.

Busca em profundidade (slides), com o cálculo de $back(v)$ e dos blocos (aplicação 6)

```
Procedimento BuscaProfundidade(G)
    t := 0; para todo v: PE(v) := 0; PS(v) := 0; pai(v) := λ
    enquanto existe v com PE(v) = 0 faça P(v)        % nova raiz; uma por componente
Procedimento P(v)
    t := t + 1; PE(v) := t; back(v) := PE(v)
    para cada w ∈ N(v) faça
        se PE(w) = 0 então                           % aresta de árvore (azul)
            pai(w) := v; empilhe vw; P(w)
            se back(w) ≥ PE(v) então
                desempilhe e imprima as arestas até vw    % um bloco
            back(v) := min{back(v), back(w)}
        senão se PS(w) = 0 e w ≠ pai(v) então        % aresta de retorno (vermelha)
```

empilhe vw; $back(v) := \min\{back(v), PE(w)\}$
$t := t + 1$; $PS(v) := t$



Exemplo dos slides (conferido): raiz e, vizinhos em ordem alfabética. Rótulos PE/PS. Azul grosso: árvore e–c–a–b–d e e–f–g. Vermelho tracejado: retorno bc, dc, de, ge. $back(v) = 1$ para todos. **Articulação**: só e (raiz com 2 filhos, c e f). **Blocos**: $\{a, b, c, d, e\}$ e $\{e, f, g\}$. Com raiz a: e não é raiz e tem o filho f com $back(f) = 5 \geq PE(e) = 5$: articulação pelo outro caso do teorema.

Propriedades.

- Se G é conexo, as arestas azuis formam uma **árvore geradora** T (a árvore de profundidade). Senão, o laço externo gera uma floresta, uma árvore por componente.
- Em grafo **não direcionado** só existem arestas de **árvore** e de **retorno**. Toda aresta de retorno liga v a um **ancestral** em T e fecha um ciclo.
- **Intervalos de vida** $I(v) = [PE(v), PS(v)]$ (teorema dos parênteses): dois intervalos são disjuntos ou um contém o outro; v é descendente de $w \iff PE(w) < PE(v)$ e $PS(v) < PS(w)$ ("entra depois e sai antes").
- Complexidade: $O(n + m)$ (listas). Cada vértice entra uma vez; cada aresta é examinada duas vezes (uma por extremo).

Aplicações (slides).

1. **Conexidade**: conte quantas vezes o laço externo chama P: é o número de componentes conexas; G é conexo se e só se a contagem é 1.
2. **Caminho de v a w** (e labirintos, modelados como grafos): rode P(v); se $PE(w) = 0$, não há caminho. Senão, suba de w por $pai(\cdot)$ até v, colocando cada vértice à *esquerda*. O caminho **não** é necessariamente o mais curto (para isso, BFS).
3. **Ciclo**: sem aresta de retorno, G é acíclico (floresta). Com uma aresta de retorno vw, w é ancestral de v: suba de v por *pai* até w e feche com v.
4. **2-coloração (bipartição)**: G é bipartido $\iff$ não tem ciclo ímpar. Na busca, $cor(w) := 1 - cor(v)$ nas arestas de árvore; uma aresta de retorno com $cor(w) = cor(v)$ fecha um ciclo ímpar.
5. **Articulações e blocos.** Articulação: vértice cuja remoção aumenta o número de componentes. Bloco: subgrafo maximal conexo e sem articulação (um bloco com uma aresta só é uma **ponte**). Os blocos particionam as *arestas*. Um vértice em mais de um bloco é articulação. $back(v) =$ menor PE alcançável a partir de v descendo por arestas de T e usando no máximo uma aresta de retorno.

   Teorema 12.1 (Articulações via *back*)**.** *Seja T uma árvore de profundidade de G. A raiz de T é articulação $\iff$ tem ≥ 2 filhos em T. Um vértice v que não é raiz é articulação $\iff$ tem um filho w com $back(w) \geq PE(v)$ (nenhum descendente de w "escapa" para cima de v; w é um* demarcador *de v).*

   Os blocos saem com uma pilha de arestas: ao voltar de w com $back(w) \geq PE(v)$, desempilhe até vw. *Pontes*: arestas de árvore vw com $back(w) > PE(v)$. Tudo em $O(n + m)$.

12.2 Busca em profundidade em digrafos (aula 20, 20/10; slides 123–170)

Mesmo procedimento percorrendo $N_{out}(v)$, com o laço externo (a floresta de profundidade alcança todos os vértices). Agora há **quatro** tipos de aresta:

Classificação das arestas vw na busca em digrafos (slides)

```
Procedimento P(v)
    t := t + 1; PE(v) := t
    para cada w ∈ N_out(v) faça
        se PE(w) = 0 então
            vw é de árvore (azul); pai(w) := v; P(w)
        senão se PS(w) = 0 então
            vw é de retorno (vermelha)              % w ainda está na busca: ancestral
        senão se PE(v) < PE(w) então
            vw é de avanço (amarela)                % w é descendente já terminado
        senão
            vw é de cruzamento (verde)              % w terminou antes de v começar
    t := t + 1; PS(v) := t
```

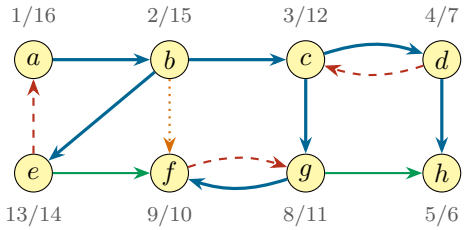
Tipo de vw	Caracterização por intervalos	Observação
Árvore	$I(v) \supset I(w)$ e $PE(w) = 0$ na hora da visita	liga pai e filho
Avanço	$I(v) \supset I(w)$ e $PE(w) \neq 0$ na hora da visita	vai a um descendente não filho
Retorno	$I(v) \subset I(w)$	vai a um ancestral: **fecha ciclo direcionado**
Cruzamento	$I(v)$ inteiro à **direita** de $I(w)$	sempre "para trás" no tempo

Nunca há aresta vw com $I(v)$ inteiro à esquerda de $I(w)$ (quando v examina vw, w seria visitado como filho). G é **acíclico** $\iff$ uma busca (qualquer uma) não produz aresta de retorno.

Componentes fortemente conexas (CFC). Subdigrafo maximal em que, para quaisquer v, w, há caminho de v a w *e* de w a v. As CFC particionam os *vértices*; algumas arestas não ficam em nenhuma CFC (são de árvore, avanço ou cruzamento). O resultado não depende da ordem da busca, embora o tipo de cada aresta dependa. $old(v) = PE$ do vértice mais antigo, **na mesma CFC** que v, alcançável descendo por arestas de árvore e usando no máximo uma aresta de retorno ou de cruzamento. v é **forte** (a "raiz" da sua CFC) se $old(v) = PE(v)$ quando v sai da busca.

CFC (slides; é o algoritmo de Tarjan) – $O(n + m)$

```
Procedimento P(v)
    t := t + 1; PE(v) := t; old(v) := PE(v); empilhe v em Q
    para cada w ∈ N_out(v) faça
        se PE(w) = 0 então
            pai(w) := v; P(w); old(v) := min{old(v), old(w)}      % árvore
        senão se PS(w) = 0 então
            old(v) := min{old(v), PE(w)}                          % retorno
        senão se PE(v) < PE(w) então
            (avanço: nada a fazer)
        senão se w está em Q então
            old(v) := min{old(v), PE(w)}    % cruzamento dentro de uma CFC ainda aberta
    t := t + 1; PS(v) := t
    se old(v) = PE(v) então
        desempilhe de Q todos os vértices até v (inclusive): formam uma CFC
```



Exemplo 12.2 (Digrafo do Cormen, Fig. 22.9 — conferido)**.** Busca a partir de a, vizinhos em ordem alfabética; rótulos PE/PS. Azul: árvore; vermelho tracejado: retorno (dc, ea, fg); laranja pontilhado: avanço (bf); verde: cruzamento (ef, gh). *old*: a=1, b=1, e=1, c=3, d=3, g=8, f=8, h=5. Vértices fortes, na ordem em que saem: h, g, c, a. CFC: $\{h\}$, $\{f,g\}$, $\{c,d\}$, $\{a,b,e\}$. Note que ef é de cruzamento, mas f já saiu de Q (sua CFC já foi fechada), então não altera $old(e)$.

12.2.1 Ordenação topológica (slides 208–273)

Num digrafo **acíclico** (DAG), uma ordenação $v_1, \ldots, v_n$ tal que toda aresta v_iv_j tem $i < j$ (escalonamento de tarefas com dependências). **Algoritmo:** rode uma busca em profundidade; se houver aresta de retorno, pare (há ciclo); senão, ordene os vértices por PS **decrescente**. Implementação $O(n+m)$: ao dar PS a um vértice, coloque-o imediatamente à *esquerda* da lista (dispensa ordenar).

Teorema 12.3 (Correção)**.** *Num DAG, toda aresta vw tem $PS(w) < PS(v)$.*

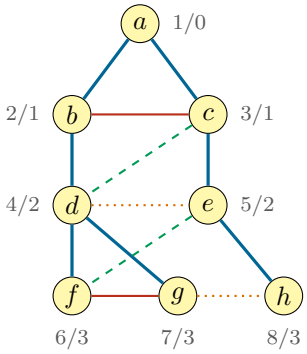
Prova: não há retorno. Árvore ou avanço: w é descendente de v, então sai antes. Cruzamento: $I(w)$ está inteiro à esquerda de $I(v)$. (Visão dos slides: o vértice de menor PS é um *sumidouro*; tirando-o, o de menor PS restante é sumidouro do que sobrou, e assim por diante.) *Exemplo dos slides* (DAG com $a{\to}c$, $a{\to}d$, $d{\to}c$, $b{\to}c$, $e{\to}b$, $e{\to}f$, $e{\to}g$, $f{\to}g$, $h{\to}f$, $i{\to}h$, $i{\to}j$, $j{\to}h$, $j{\to}g$): com raízes em ordem alfabética, PS decrescente dá $i, j, h, e, f, g, b, a, d, c$; o slide mostra $a, d, e, b, c, i, j, h, f, g$ (outra ordem de raízes: i, c, b, e, d, a). **As duas são válidas**: a ordenação topológica não é única. *Alternativa* (exercício dos slides): todo DAG tem um sumidouro (o fim de um caminho máximo) e uma fonte. Retire repetidamente um sumidouro e coloque-o à esquerda (ou uma fonte, à direita). Com contadores de grau e uma fila dos graus zerados, fica $O(n+m)$ (algoritmo de Kahn).

12.3 Busca em largura (aula 18, 13/10; slides 274–312; Cormen §22.2)

"*Esgotar o pai antes de processar os filhos*": a próxima aresta parte sempre do vértice **mais antigo** na busca (**fila** F). $L(v)$ é a ordem de chegada e $nivel(v)$ a profundidade na floresta de largura.

Busca em largura (slides) com classificação das arestas

Procedimento BuscaLargura(G)
  $t := 0$; $F := \emptyset$; para todo v: $L(v) := 0$; $pai(v) := \lambda$
  enquanto existe v com $L(v) = 0$ **faça**
    $nivel(v) := 0$; $t := t+1$; $L(v) := t$; insira v no fim de F
    enquanto $F \neq \emptyset$ **faça**
      $v :=$ primeiro de F; retire v de F
      para cada $w \in N(v)$ **faça**
        se $L(w) = 0$ **então** % aresta-**pai** (azul)
          $pai(w) := v$; $nivel(w) := nivel(v)+1$; $t := t+1$; $L(w) := t$; insira w no fim de F
        senão se $nivel(w) = nivel(v)$ **e** $w \in F$ **então** % $w \in F$: não classificar duas vezes
          se $pai(w) = pai(v)$, vw é aresta-**irmão** (vermelha); senão, aresta-**primo** (amarela)
        senão se $nivel(w) = nivel(v)+1$ **então**
          vw é aresta-**tio** (verde)



Exemplo dos slides (conferido), raiz a; rótulos $L/nivel$. Azul grosso: arestas-pai. Vermelho: irmão (bc, fg). Amarelo pontilhado: primo (de, gh). Verde tracejado: tio (cd, ef). Distâncias de a: $b, c \mapsto 1$; $d, e \mapsto 2$; $f, g, h \mapsto 3$. Não é bipartido: o triângulo abc é um ciclo ímpar, e isso aparece como as arestas-irmão/primo.

- **Caracterizações:** pai $\iff$ $nivel(w) = nivel(v) + 1$ e $L(w) = 0$ na visita; tio $\iff$ $nivel(w) = nivel(v) + 1$ e $L(w) \neq 0$; irmão $\iff$ mesmo nível e mesmo pai; primo $\iff$ mesmo nível e pais diferentes.
- **Propriedade fundamental:** para toda aresta vw, $|nivel(v) - nivel(w)| \leq 1$. Em qualquer momento, os vértices na fila diferem em no máximo um nível.
- **Aplicação 1: distâncias.** Com raiz x, $\text{dist}(x, v) = nivel(v)$, e o caminho de x a v em T é **mínimo** (em número de arestas). Labirinto: o menor caminho da entrada à saída.
- **Aplicação 3: bipartição.** G é bipartido $\iff$ a floresta de largura não tem arestas-irmão nem primo (elas fecham ciclos ímpares). A 2-coloração é dada pela paridade do nível.
- **Complexidade:** $O(n + m)$ com listas.
- **Pilha $\times$ fila** (exercício dos slides): no esquema "marque a raiz e insira em D; enquanto $D \neq \emptyset$, seja v o *primeiro* de D; se v tem vizinho w desmarcado, marque w, $pai(w) := v$ e insira w em D; senão remova v", usar $D =$ **pilha** dá a busca em profundidade e $D =$ **fila** dá a busca em largura. Com $D =$ **fila de prioridades** chega-se a Dijkstra (§13.1).

Armadilhas das buscas

- Esquecer o laço externo: num grafo desconexo, P(v) só alcança a componente de v.
- Em grafo não direcionado, a aresta vw com $w = pai(v)$ *não* é de retorno (é a própria aresta de árvore vista do outro lado).
- O caminho achado por DFS não é mínimo; o achado por BFS é mínimo só em *número de arestas* (com pesos, Dijkstra).
- Em digrafos, "acíclico" é sobre ciclos *direcionados*: arestas de cruzamento e avanço não indicam ciclo; só as de retorno.
- Regra da articulação: para a raiz vale "$\geq$ 2 filhos", não o teste com *back* (na raiz, $back(w) \geq PE(\text{raiz}) = 1$ é sempre verdade).
- O pseudocódigo das CFC nos slides tem `executar P(w)` repetido na mesma linha; é uma chamada só.

Pontos-chave

DFS: pilha/recursão, PE/PS, intervalos aninhados; não direcionado tem só árvore+retorno; digrafo tem árvore, retorno, avanço e cruzamento. Retorno $\iff$ ciclo. Articulação: raiz com ≥ 2 filhos, ou filho w com $back(w) \geq PE(v)$. CFC: $old(v) = PE(v) \Rightarrow$ desempilha. Topológica: PS decrescente. BFS: fila, níveis, $|\Delta nivel| \leq 1$, distâncias em arestas, bipartido $\iff$ sem irmão/primo. Tudo $O(n + m)$.

13 Caminhos mínimos: Dijkstra e Floyd-Warshall

Entrada (slides): digrafo G com **matriz de pesos** W: $W(x,x)=0$; $W(x,y)=$ peso da aresta xy, se houver; $W(x,y)=\infty$ se não houver. O custo de um caminho é a soma dos pesos; $\mathrm{dist}(x,y)$ é o menor custo. Grafo não direcionado: troque cada aresta xy de peso p por $x \to y$ e $y \to x$, ambas com peso p.

W	$1\,(a)$	$2\,(b)$	$3\,(c)$	$4\,(d)$	$5\,(e)$
$1\,(a)$	0	1	3	∞	6
$2\,(b)$	∞	0	1	3	∞
$3\,(c)$	1	2	0	1	∞
$4\,(d)$	3	∞	∞	0	2
$5\,(e)$	∞	∞	∞	1	0

Exemplo dos slides, usado nas duas seções: os vértices são $a,\ldots,e$ no Dijkstra e $1,\ldots,5$ no Floyd-Warshall (mesmo digrafo).

13.1 Algoritmo de Dijkstra (aula 18, 13/10; slides 313–341; Cormen §24.3)

Problema: caminhos mínimos de uma raiz s a todos os vértices, com pesos **não negativos**. É a busca genérica com $D =$ **fila de prioridades** (chave L): sai sempre o vértice não marcado de menor rótulo L.

Dijkstra (slides)

Procedimento DIJKSTRA(G, W, s)
  para todo w: $L(w) := \infty$; $pai(w) := \lambda$; desmarcado
  $L(s) := 0$; insira s em D
  enquanto $D \neq \emptyset$ **faça**
    $v :=$ elemento de D com menor $L(v)$; remova v de D; marque v % $L(v)$ agora é definitivo
    para cada $w \in N_{out}(v)$ desmarcado **faça**
      se $w \notin D$ **então**
        insira w em D com prioridade $L(w)$
      se $L(v) + W(v,w) < L(w)$ **então** % "relaxamento" da aresta vw
        $L(w) := L(v) + W(v,w)$; reposicione w em D; $pai(w) := v$

Exemplo 13.1 (Raiz a no digrafo acima — conferido)**.**

Remove	$L(a)$	$L(b)$	$L(c)$	$L(d)$	$L(e)$	mudanças em pai
a (0)	**0**	1	3	∞	6	$b, c, e \leftarrow a$
b (1)		**1**	2	4	6	$c, d \leftarrow b$
c (2)			**2**	3	6	$d \leftarrow c$
d (3)				**3**	5	$e \leftarrow d$
e (5)					**5**	—

Árvore de caminhos mínimos (os ponteiros pai): $a \to b \to c \to d \to e$. Ex.: o caminho mínimo de a a e é a, b, c, d, e, com custo $L(e) = 5$ (a aresta direta custa 6). A linha 1 da matriz-solução do Floyd-Warshall abaixo confirma $0, 1, 2, 3, 5$.

Teorema 13.2 (Correção)**.** *Com pesos ≥ 0, quando v é removido de D vale $L(v) = \mathrm{dist}(s, v)$.*

Prova. Sempre vale $L(v) \geq \mathrm{dist}(s,v)$, pois $L(v)$ é o custo de algum caminho. Suponha que u é o primeiro vértice removido com $L(u) > \mathrm{dist}(s,u)$. Num caminho mínimo de s a u, seja y o primeiro vértice ainda não marcado e x o anterior (marcado, logo $L(x) = \mathrm{dist}(s,x)$). Quando x foi marcado, a aresta xy foi relaxada: $L(y) \leq \mathrm{dist}(s,x) + W(x,y) = \mathrm{dist}(s,y) \leq \mathrm{dist}(s,u) < L(u)$, onde a segunda desigualdade usa **pesos não negativos** (o trecho de y a u custa ≥ 0). Mas então u não teria o menor L em D. Contradição. □

Estrutura D	Custo	Quando usar
Vetor L + varredura (matriz de adjacência)	$O(n^2)$	grafo **denso** ($m = \Theta(n^2)$)
Heap binário (de mínimo) com posição de cada vértice	$O((n+m)\log n)$	grafo esparso (slides)
Heap de Fibonacci	$O(m + n\log n)$	teórico: *decrease-key* $O(1)^*$

Os custos vêm de n inserções, n remoções do mínimo e até m reduções de prioridade (§2, §3.3). No heap binário, "reposicionar" é um *subir* e exige um vetor $pos[w]$ com a posição de w no heap.

- **Pesos todos iguais a 1**: Dijkstra se comporta como a busca em largura, e $L(w)$ é a distância em arestas.
- **Aresta negativa quebra o algoritmo**: $x \to y$ (1), $x \to z$ (2), $z \to y$ (-2). Dijkstra fixa $L(y) = 1$, mas x, z, y custa 0. (Para pesos negativos sem ciclo negativo: Bellman-Ford, $O(nm)$, fora do curso; ou Floyd-Warshall para todos os pares.)

13.2 Floyd-Warshall: PD em grafos (aula 16, 06/10; slides 342–405; Cormen §25.2)

Problema: caminhos mínimos entre **todos os pares** ordenados (i, j). Vértices numerados $1, \ldots, n$.

$D^k(i,j)$ = custo do caminho mínimo de i a j cujos vértices *intermediários* estão em $\{1, \ldots, k\}$.

$D^0 = W$ e D^n = matriz-solução. **Decomposição** (quadro da aula): o caminho mínimo restrito a $\{1..k\}$ ou **não usa** k (custa $D^{k-1}(i,j)$), ou passa **uma vez** por k e se divide em "mínimo de i a k" + "mínimo de k a j", ambos com intermediários em $\{1..k-1\}$ (voltar a k seria um ciclo, que não ajuda):

$$D^k(i,j) = \begin{cases} W(i,j) & k = 0 \\ \min\{\, D^{k-1}(i,j),\ D^{k-1}(i,k) + D^{k-1}(k,j) \,\} & k \geq 1. \end{cases}$$

Por que só "prefixos" $\{1..k\}$ (quadro): com um conjunto *arbitrário* $S \subseteq V$ de intermediários, $D(i,j,S) = \min_{v \in S}\{D(i,v,S\setminus\{v\}) + D(v,j,S\setminus\{v\})\}$ também é correto, mas há $n^2\,2^n$ subproblemas. Fixando uma ordem dos vértices e usando só os $n+1$ prefixos, sobram $O(n^3)$.

Floyd-Warshall com matriz de predecessores P – $\Theta(n^3)$ tempo, $\Theta(n^2)$ memória

```
Função FloydWarshall(W, n)
    para i, j := 1 até n faça
        D[i,j] := W[i,j]
        se i ≠ j e W[i,j] < ∞ então
            P[i,j] := i
        senão
            P[i,j] := λ
    para k := 1 até n faça                          % k é o laço externo
        para i := 1 até n faça
            para j := 1 até n faça
                se D[i,k] + D[k,j] < D[i,j] então
                    D[i,j] := D[i,k] + D[k,j]; P[i,j] := P[k,j]  % penúltimo vértice vem do trecho k ⇝ j
    retorna D, P
Procedimento Caminho(P, i, j)
    se i = j então
        imprima i
    senão se P[i,j] = λ então
        imprima "não há caminho"
    senão
        Caminho(P, i, P[i,j]); imprima j
```

$P^k(i,j)$ é o **penúltimo** vértice do caminho mínimo de i a j (slides). Se o mínimo não muda, $P^k(i,j) = P^{k-1}(i,j)$; se muda, $P^k(i,j) = P^{k-1}(k,j)$. Uma só matriz basta (atualização "no lugar"): na iteração k, a linha e a coluna k não mudam, pois $D(k,k) = 0$.

Exemplo 13.3 (Slides, matriz W acima — conferido)**.**

k	entradas que diminuem (via k)
1	$D(3,5)\colon \infty \to 7$; $D(4,2)\colon \infty \to 4$; $D(4,3)\colon \infty \to 6$
2	$D(1,3)\colon 3 \to 2$; $D(1,4)\colon \infty \to 4$; $D(4,3)\colon 6 \to 5$
3	$D(1,4)\colon 4 \to 3$; $D(2,1)\colon \infty \to 2$; $D(2,4)\colon 3 \to 2$; $D(2,5)\colon \infty \to 8$
4	$D(1,5)\colon 6 \to 5$; $D(2,5)\colon 8 \to 4$; $D(3,5)\colon 7 \to 3$; $D(5,1)\colon \infty \to 4$; $D(5,2)\colon \infty \to 5$; $D(5,3)\colon \infty \to 6$
5	nenhuma

D^5	1	2	3	4	5
1	0	1	2	3	5
2	2	0	1	2	4
3	1	2	0	1	3
4	3	4	5	0	2
5	4	5	6	1	0

P^5	1	2	3	4	5
1	–	1	2	3	4
2	3	–	2	3	4
3	3	3	–	3	4
4	4	1	2	–	4
5	4	1	2	5	–

Caminho de 5 a 3: $P(5,3) = 2$, $P(5,2) = 1$, $P(5,1) = 4$, $P(5,4) = 5 \Rightarrow 5 \to 4 \to 1 \to 2 \to 3$, custo $1+3+1+1 = 6 = D^5(5,3)$.

- **Pesos negativos**: os slides pedem pesos positivos, mas o algoritmo funciona com arestas negativas desde que **não haja ciclo negativo**. Há ciclo negativo $\iff$ ao final algum $D(i,i) < 0$.
- **Floyd-Warshall $\times$ n vezes Dijkstra**: $\Theta(n^3)$ simples, contra $O(n\,(n+m)\log n)$ com heap, ou $O(n^3)$ com vetor. Em grafo esparso e sem pesos negativos, n Dijkstras ganham; em grafo denso ou com pesos negativos, Floyd-Warshall.

Armadilhas de caminhos mínimos

- Laço do k **por dentro** (i, j fora e k dentro) dá resposta errada: k tem de ser o laço externo.
- Usar Dijkstra com aresta negativa.
- Confundir com AGM: a árvore de caminhos mínimos (Dijkstra) minimiza a distância de cada vértice à raiz; a AGM (§14.3) minimiza a *soma* dos pesos da árvore. São, em geral, árvores diferentes.
- Em Dijkstra com heap, "reposicionar" exige saber onde w está no heap; sem $pos[\cdot]$ a busca custaria $O(n)$.

Pontos-chave

Dijkstra: fila de prioridades em L; pesos ≥ 0; $O(n^2)$ (vetor), $O((n+m)\log n)$ (heap), $O(m+n\log n)$ (Fibonacci). Floyd-Warshall: $D^k(i,j) = \min\{D^{k-1}(i,j), D^{k-1}(i,k) + D^{k-1}(k,j)\}$, k externo, $\Theta(n^3)$; $P(i,j)$ = penúltimo vértice; ciclo negativo $\iff$ $D(i,i) < 0$.

14 Algoritmos gulosos, matroides e árvores geradoras mínimas

14.1 Método guloso (slides de Matroides)

Esquema geral: queremos $S' \subseteq S$ que satisfaça uma propriedade P e seja máximo (ou mínimo) por um critério. O guloso começa com $S'_0 = \emptyset$ e, a cada passo, acrescenta o elemento s que mantém P e é o **melhor segundo o critério naquele momento**, sem nunca desfazer escolhas. Para enquanto houver tal s.

- **Quando dá o ótimo?** Quando vale a **propriedade gulosa**: existe uma solução ótima que contém o elemento escolhido, e, fixado esse elemento, resta resolver um subproblema bem definido do mesmo tipo (subestrutura ótima).
- **Como provar:** *argumento de troca.* Tome uma ótima O que concorde com o guloso o máximo possível; no primeiro ponto de discordância, troque um elemento de O pelo do guloso sem piorar. Isso contradiz a escolha de O, ou mostra que o guloso também é ótimo.
- **O que os problemas gulosos têm em comum?** Em grande parte deles o sistema de conjuntos viáveis é uma **matroide** (§14.2).
- **PD × guloso:** a PD testa todas as escolhas de cada subproblema (tabela); o guloso faz uma escolha só e nunca volta atrás. É mais rápido, mas só funciona com a propriedade gulosa.

Troco guloso (lista de AGM e gulosos, ex. 8)

Função TrocoGuloso(S, $v_1 > v_2 > \cdots > v_n$) % moedas em ordem decrescente
  $total := 0$
  para $i := 1$ **até** n **faça**
    $q_i := \lfloor S/v_i \rfloor$; $S := S - q_i\, v_i$; $total := total + q_i$ % o máximo possível da maior moeda
  retorna $total$ % se $S \neq 0$ ao final, não há troco exato

Ótimo para $\{10, 5, 2, 1\}$. Seja (a, b, c, d) o número de moedas de $10, 5, 2, 1$ numa solução ótima. Então $d \leq 1$ (duas de 1 viram uma de 2), $b \leq 1$ (duas de 5 viram uma de 10), $c \leq 2$ (três de 2 viram $5 + 1$) e não ocorre $c = 2$ com $d = 1$ ($2 + 2 + 1$ vira uma de 5). Logo as moedas pequenas somam $5b + 2c + d \leq 9 < 10$, e então $a = \lfloor S/10 \rfloor$, como no guloso. Do mesmo modo, $2c + d \leq 4 < 5$ força b igual ao do guloso, e $d \leq 1$ força c. A ótima *é* a gulosa (conferido para $S \leq 500$). **Não funciona sempre:** $\{4, 3, 1\}$ com $S = 6$: o guloso dá $4 + 1 + 1$ (3 moedas), mas $3 + 3$ usa 2. Outro: $\{25, 10, 1\}$ com $S = 30$: guloso 6 moedas, ótimo $10 + 10 + 10$. (O caso geral se resolve por PD, como a mochila ilimitada: $T(s) = 1 + \min_{v_i \leq s} T(s - v_i)$, em $\Theta(nS)$.)

14.2 Matroides (slides de Matroides; pesquisa)

Definição 14.1 (Matroide; Whitney, 1935)**.** $M = (E, \mathcal{I})$, com E finito e $\mathcal{I} \subseteq 2^E$ (os **independentes**), tal que: (I1) $\emptyset \in \mathcal{I}$; (I2) se $I \in \mathcal{I}$ e $I' \subseteq I$, então $I' \in \mathcal{I}$ (*sistema de subconjuntos*, fechado para baixo); (I3) se $I_1, I_2 \in \mathcal{I}$ e $|I_1| < |I_2|$, existe $e \in I_2 \setminus I_1$ com $I_1 \cup \{e\} \in \mathcal{I}$ (**aumento**, ou troca).

O modelo é a independência linear: "independente" generaliza "linearmente independente" (matrizes) e "acíclico" (grafos).

Matroide	Independentes
Linear	conjuntos de colunas (vetores) de uma matriz linearmente independentes.
Gráfica (de G)	conjuntos de arestas **acíclicos** (florestas). Bases = árvores geradoras (se G é conexo).
Uniforme $U_{n,k}$	todos os $I \subseteq E$ com $\lvert I \rvert \leq k$ (*livre* se $k = n$).
Partição	$E = E_1 \cup \cdots \cup E_t$ (partição) e I com $\lvert I \cap E_i \rvert \leq k_i$ para todo i.
Transversal	em G bipartido (X, Y): os $I \subseteq X$ *emparelháveis* (cobertos por algum emparelhamento).
Emparelhamento	em G qualquer: os $I \subseteq V$ cobertos por algum emparelhamento.
Dual M^*	bases são os complementos $E \setminus B$ das bases de M (dual da gráfica: *cográfica*).

(I3) para a gráfica (exercício dos slides): uma floresta com k arestas em n vértices tem $n - k$ componentes. Como $|I_1| < |I_2|$, I_2 tem menos componentes que I_1, então alguma aresta $e \in I_2$ liga duas componentes diferentes de I_1 (se todas as arestas de I_2 ficassem dentro de componentes de I_1, cada componente de I_2 caberia numa de I_1, e I_2 teria pelo menos tantas componentes quanto I_1). Logo $I_1 \cup \{e\}$ é acíclico. **(I3) para a linear**: $\text{span}(I_1)$ tem dimensão $|I_1| < |I_2|$, então algum vetor de I_2 está fora dele, e acrescentá-lo a I_1 mantém a independência.

Formas equivalentes de (I3) e bases.

- **(I3′)** Para todo $X \subseteq E$, todos os independentes *maximais* contidos em X têm o mesmo tamanho (o **posto** $r(X)$). [$\Rightarrow$: se $|I_1| < |I_2|$ maximais em X, (I3) aumentaria I_1 dentro de X. $\Leftarrow$: em $X = I_1 \cup I_2$, estenda I_1 a um maximal B_1; como $|B_1| \geq |I_2| > |I_1|$, existe $e \in B_1 \setminus I_1 \subseteq I_2 \setminus I_1$, e $I_1 \cup \{e\} \subseteq B_1$ é independente por (I2).]
- **(I3″)** Todos os maximais de $\mathcal{I}$ têm o mesmo tamanho r, e se $|I| = r$, $|I'| = r - 1$, existe $x \in I \setminus I'$ com $I' \cup \{x\} \in \mathcal{I}$.
- **Base**: independente maximal; todas têm o mesmo tamanho. A coleção $\mathcal{B}$ de bases satisfaz (B1) $\mathcal{B} \neq \emptyset$ e (B2) (**troca de bases**): se $B_1, B_2 \in \mathcal{B}$ e $x \in B_1 \setminus B_2$, existe $y \in B_2 \setminus B_1$ com $(B_1 \setminus \{x\}) \cup \{y\} \in \mathcal{B}$. Reciprocamente, toda $\mathcal{B}$ com (B1)(B2) é a coleção de bases da matroide $\mathcal{I} = \{I : I \subseteq B \text{ para algum } B \in \mathcal{B}\}$.

Algoritmo guloso para matroides (base de peso máximo). Dados M e $c : E \to \mathbb{R}$, achar uma base de peso $c(B) = \sum_{e \in B} c(e)$ máximo. Os independentes são dados *implicitamente*, por um **oráculo** que responde "X é independente?" (listá-los pode ser exponencial).

GreedyAlgorithm(M, c) **(slides)** – $O(n \log n + n\,h)$, h = **custo do oráculo**

```
Função Guloso(M, c)
    B := ∅
    ordene E: c(e_1) ≥ c(e_2) ≥ ··· ≥ c(e_n)        % não crescente
    para i := 1 até n faça
        se B ∪ {e_i} é independente em M então
            B := B ∪ {e_i}
    retorna B
```

Teorema 14.2 (Rado–Edmonds)**.** *Se M é matroide,* Guloso *devolve uma base de peso máximo, para qualquer c.*

Prova (slides). Seja B a saída (é base, pois nada mais cabe) e B^* uma base ótima com $|B \cap B^*|$ máximo. Se $B \neq B^*$, seja f o primeiro elemento (na ordem do algoritmo) de $B \setminus B^*$; os escolhidos antes de f formam $B' \subseteq B^*$. Pela troca de bases, existe $e \in B^* \setminus B$ com $(B^* \setminus \{e\}) \cup \{f\}$ base. Pela otimalidade de B^*, $c(f) \leq c(e)$. Se $c(f) = c(e)$, essa nova base seria ótima e teria mais elementos em comum com B: contradição. Logo $c(f) < c(e)$, e e vem antes de f na ordem. Mas quando e foi examinado, o conjunto escolhido estava contido em B', e $B' \cup \{e\} \subseteq B^*$ é independente: o algoritmo teria escolhido e. Contradição. □

A recíproca também vale (definição alternativa): se $\mathcal{I}$ satisfaz (I1)(I2) e o guloso acerta para *todo* c, então vale (I3). [Suponha $|I_1| = k < |I_2|$ e que nenhum $e \in I_2 \setminus I_1$ aumente I_1. Use $c = k + 2$ em I_1, $c = k + 1$ em $I_2 \setminus I_1$ e $c = 0$ no resto. O guloso pega I_1 primeiro, não consegue pegar nada de $I_2 \setminus I_1$ e termina com peso $k(k+2)$. Mas I_2 pesa pelo menos $(k+1)(k+1) > k(k+2)$. Contradição.] Ou seja, **matroides são exatamente os sistemas em que o guloso é sempre ótimo**.

- **Base de peso mínimo**: ordem não decrescente (ou pesos $-c$). Kruskal é exatamente isso na matroide gráfica (§14.3.1).
- **Testar se k colunas de uma matriz $m \times k$ são LI** ("para praticar"): escalonamento (Gauss), $O(m\,k \min\{m, k\})$; se $k > m$, já são dependentes.
- **Escalonamento de intervalos** (pesos unitários): o guloso "termina primeiro" é ótimo, mas **não** é matroide. Com $I_1 = \{[1, 10]\}$ e $I_2 = \{[1, 2], [3, 4]\}$, nenhum intervalo de I_2 cabe junto com $[1, 10]$, e (I3) falha. A otimalidade vem de um argumento de troca específico; com pesos arbitrários o guloso falha e é preciso PD.
- **Emparelhamento** (conjuntos de arestas) num grafo qualquer **não** é matroide: no caminho $a-b-c-d$, $\{bc\}$ não pode ser aumentado por ab nem por cd (de $\{ab, cd\}$). Um guloso que pegue bc primeiro fica com 1 aresta, e o máximo é 2.

Complemento – pesquisa "Matroides, algoritmos gulosos e generalizações polinomiais"

Interseção de matroides. Dadas $M_1 = (E, \mathcal{I}_1)$ e $M_2 = (E, \mathcal{I}_2)$, achar $I \in \mathcal{I}_1 \cap \mathcal{I}_2$ de tamanho (ou peso) máximo. Exemplos: *emparelhamento bipartido* = interseção de duas matroides de partição (no máximo uma aresta por vértice de X; idem para Y); *floresta colorida* (exemplo 2 da pesquisa) = gráfica ∩ partição pelas cores (no máximo uma aresta de cada cor); *árvore geradora "arco-íris"*. **Por que o guloso falha** (exemplo 3): a interseção de duas matroides em geral não é matroide (o caminho $a{-}b{-}c{-}d$ acima é bipartido). **Algoritmo (Edmonds/Lawler):** caminhos aumentantes num *grafo de troca*. Dado I comum, para $y \in I$ e $x \notin I$: arco $y \to x$ se $I - y + x \in \mathcal{I}_1$, e arco $x \to y$ se $I - y + x \in \mathcal{I}_2$. As fontes são os x com $I + x \in \mathcal{I}_1$ e os destinos os x com $I + x \in \mathcal{I}_2$. Um caminho *mais curto* de uma fonte a um destino dá $I \triangle$ caminho comum com um elemento a mais; se não há caminho, I é máximo. São $\leq r$ aumentos, cada um com $O(nr)$ chamadas ao oráculo: **polinomial**. Vale o teorema min-max: $\max |I| = \min_{U \subseteq E} \{r_1(U) + r_2(E \setminus U)\}$. **Três matroides: NP-difícil.** O *caminho hamiltoniano* num digrafo é um independente comum de tamanho $n-1$ de três matroides: a gráfica do grafo subjacente (sem ciclos), a partição "grau de entrada ≤ 1" e a partição "grau de saída ≤ 1". **Particionamento de matroides** (cobrir E com o menor número de independentes; ex.: cobrir as arestas com o menor número de florestas, a *arboricidade*) também é polinomial. Moral: reconhecer que um problema é interseção ou partição de matroides prova que ele é polinomial.

14.3 Árvore geradora mínima (AGM)

Problema: dado G conexo com pesos c nas arestas, achar uma árvore geradora (conexa, acíclica, com os n vértices e, portanto, $n-1$ arestas) de peso total mínimo. Os dois algoritmos são gulosos e a correção de ambos vem do lema abaixo.

Lema 14.3 (Propriedade do corte; Cormen, Teo. 23.1)**.** *Seja A um conjunto de arestas contido em alguma AGM, e $(S, V \setminus S)$ um corte que nenhuma aresta de A atravessa. Se e é uma aresta de peso mínimo entre as que atravessam o corte, então $A \cup \{e\}$ está contido em alguma AGM.*

Prova (troca). Seja T uma AGM com $A \subseteq T$. Se $e \in T$, acabou. Senão, $T + e$ tem um ciclo que passa por e; como e atravessa o corte, o ciclo o atravessa de novo por outra aresta $f \in T$, com $f \notin A$. $T' = T + e - f$ é árvore geradora e $c(T') = c(T) + c(e) - c(f) \leq c(T)$. Logo T' é AGM e contém $A \cup \{e\}$. □

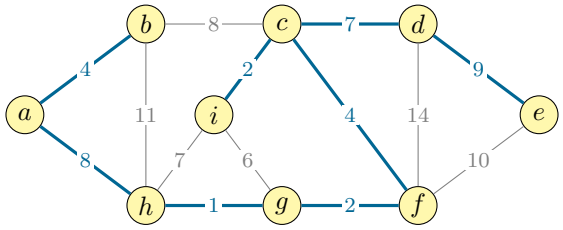
14.3.1 Algoritmo de Kruskal (aula 17, 09/10; slides 406–449; Cormen §23.2)

A solução parcial é uma **floresta**: percorre as arestas em ordem crescente de peso e aceita cada uma que **não forma ciclo**, isto é, que une duas árvores diferentes. O teste de ciclo usa conjuntos disjuntos (§4).

Kruskal com Union-Find (slides, "nova versão")

```
Função KRUSKAL(G, c)
    E(T) := ∅
    para cada v ∈ V faça
        MAKE-SET(v)                                   % n árvores triviais
    ordene as arestas: c(e_1) ≤ c(e_2) ≤ ··· ≤ c(e_m)   % O(m log m) = O(m log n)
    para j := 1 até m faça
        seja e_j = uv
        se FIND(u) ≠ FIND(v) então                    % une duas árvores distintas: não fecha ciclo
            E(T) := E(T) ∪ {e_j}; UNION(u, v)
    retorna T                                         % pode parar quando |E(T)| = n − 1
```

Complexidade: a ordenação domina, $O(m \log m) = O(m \log n)$, porque $m \le n^2$ dá $\log m \le 2 \log n$. O laço custa $O(m + n \log n)$ com listas e união "menor na maior" (slides: cada vértice muda de conjunto no máximo $\log n$ vezes), ou $O(m\,\alpha(n))$ com floresta, rank e compressão. **Total:** $O(m \log n)$. **Correção:** ao aceitar $e = uv$, tome $S =$ árvore atual de u. Nenhuma aresta aceita atravessa esse corte, e e é a mais leve que o atravessa: uma mais leve teria sido examinada antes, quando seus extremos já estavam em árvores diferentes (as árvores só crescem), e teria sido aceita. Pelo Lema 14.3, segue por indução. (Também é o Teorema 14.2 aplicado à matroide gráfica.)



Exemplo 14.4 (Grafo dos slides (Cormen, Fig. 23.4) — conferido)**. Kruskal** aceita $hg(1)$, $ic(2)$, $gf(2)$, $ab(4)$, $cf(4)$, $cd(7)$, $ah(8)$, $de(9)$ e rejeita $ig(6)$, $hi(7)$, $bc(8)$, $ef(10)$, $bh(11)$, $df(14)$, que fechariam ciclo. **Custo 37** (azul). **Prim** com raiz a acrescenta $ab(4)$, $ah(8)$, $hg(1)$, $gf(2)$, $fc(4)$, $ci(2)$, $cd(7)$, $de(9)$: mesma árvore, custo 37. *A AGM não é única*: ah e bc pesam 8, e trocar uma pela outra dá outra AGM de custo 37 (é a que aparece no Cormen). Com pesos todos distintos, a AGM é única.

14.3.2 Algoritmo de Prim (aula 17; slides 450–461 e slide de AGM)

A solução parcial é uma **árvore** que cresce a partir de uma raiz: a cada passo entra a aresta mais leve com um extremo em $V(T)$ e o outro fora. **Correção:** Lema 14.3 com $S = V(T)$.

Prim em $O(n^2)$ com vetores (lista, ex. 4 e 5)

```
Função PRIM(G, W, r)
    para todo v: chave(v) := ∞; pai(v) := λ; dentro(v) := falso
    chave(r) := 0
    para it := 1 até n faça
        u := vértice com dentro(u) = falso e chave(u) mínima        % varredura: O(n)
        dentro(u) := verdadeiro                                     % se u ≠ r, a aresta pai(u) u entra em T
        para cada v ∈ N(u) com dentro(v) = falso faça               % matriz: linha u, O(n); listas: O(grau(u))
            se W(u, v) < chave(v) então
                chave(v) := W(u, v); pai(v) := u
    retorna { pai(v) v : v ≠ r }
```

$chave(v)$ é o peso da aresta mais leve de v até a árvore. É o mesmo esqueleto do Dijkstra, trocando "$L(u) + W(u, v)$" por "$W(u, v)$".

Implementação	Custo	Comentário
Matriz de adjacência + vetor *chave*	$O(n^2)$	n varreduras de $O(n)$; ótimo para matriz
Listas de adjacência + vetor *chave*	$O(n^2 + m) = O(n^2)$	a varredura domina
Listas + heap binário (*decrease-key*)	$O(m \log n)$	n remoções + $\le m$ reduções, $O(\log n)$ cada
Listas + heap de Fibonacci	$O(m + n \log n)$	*decrease-key* $O(1)^*$

Atenção (slide de AGM): a versão com heap não é necessariamente melhor que a de $O(n^2)$. Em grafo denso, $m = \Theta(n^2)$ e $m \log n = \Theta(n^2 \log n)$. Com heap compensa quando $m = o(n^2 / \log n)$.

14.4 Lista – Árvores geradoras e métodos gulosos (resolvida)

1. **Correção de Kruskal**: Lema 14.3 com $S =$ componente de u na floresta (acima); ou Teorema 14.2 na matroide gráfica com pesos $-c$. Complete dizendo que a saída é árvore

geradora: é acíclica por construção, e é conexa porque uma aresta entre duas componentes finais teria sido aceita.

2. **Correção de Prim**: invariante "$E(T)$ está contida em alguma AGM". Vale no início ($\emptyset$) e se mantém pelo Lema 14.3 com o corte $(V(T), V \setminus V(T))$, que nenhuma aresta de T atravessa. Após $n-1$ passos, T é geradora, logo é AGM.
3. **Prim em $O(m \log n)$ com listas**: heap de mínimo dos vértices fora da árvore com chave $chave(v)$ e vetor de posições. São n remoções do mínimo e, para cada aresta uv examinada, no máximo uma redução de chave ($O(\log n)$, um *subir*): $O((n+m)\log n) = O(m \log n)$, pois G conexo tem $m \geq n-1$.
4. **Prim em $O(n^2)$ com listas**: pseudocódigo acima. As varreduras custam $n \cdot O(n)$, e as atualizações percorrem cada lista uma vez, $O(m)$. Total $O(n^2+m) = O(n^2)$, pois $m \leq n^2$.
5. **Prim em $O(n^2)$ com matriz**: igual, percorrendo a linha u da matriz ($O(n)$) em vez da lista.
6. **Com matriz, dá para fazer melhor que n^2? Não**: com matriz de adjacência, *qualquer* algoritmo correto precisa ler $\Omega(n^2)$ posições. Se alguma posição (u, v) não fosse lida, um adversário poderia colocar ali uma aresta levíssima, que estaria em toda AGM. Logo $O(n^2)$ é ótimo nessa representação (e a versão com heap seria pior: $O(n^2 \log n)$).
7. **Árvore geradora de peso máximo**: (i) Kruskal com as arestas em ordem **não crescente**, que é o guloso de matroides diretamente com c; (ii) trocar os pesos por $c' = C - c$ (com $C > \max c$, mantém os pesos positivos) ou por $-c$ e rodar qualquer algoritmo de AGM. Funciona porque toda árvore geradora tem exatamente $n-1$ arestas, então $c'(T) = (n-1)C - c(T)$, e minimizar c' é maximizar c; (iii) Prim escolhendo a aresta *mais pesada* do corte. No grafo do exemplo: peso máximo 71.
8. **Troco**: §14.1.

Armadilhas de gulosos e AGM

- "Guloso funciona porque escolhe o melhor a cada passo" não é prova. A prova é por troca, ou pela estrutura de matroide.
- Kruskal aceita a aresta se ela *não fecha ciclo*, e não "se o vértice ainda não foi visitado". Isso seria Prim, e no Kruskal a solução parcial pode ser desconexa.
- A AGM minimiza a soma dos pesos, não as distâncias à raiz (isso é Dijkstra).
- Complexidade de Kruskal: $O(m \log n)$ por causa da *ordenação*. O Union-Find sozinho é quase linear.
- Dizer que Prim com heap é sempre melhor. Em grafo denso não é.

Pontos-chave

Matroide: (I1) $\emptyset$, (I2) hereditário, (I3) aumento $\Rightarrow$ o guloso (ordem decrescente + oráculo) acha a base de peso máximo, e só nas matroides ele acerta para todo peso. Exemplos: linear, gráfica, uniforme, partição, transversal. AGM: propriedade do corte. Kruskal: arestas ordenadas + Union-Find, $O(m \log n)$. Prim: árvore crescendo, $O(n^2)$ (vetor/matriz) ou $O(m \log n)$ (heap) ou $O(m + n \log n)$ (Fibonacci).

15 Banco de questões pós-AV1 e checklist

As listas do curso estão resolvidas nas próprias seções: PD em §11.4, AGM e gulosos em §14.4. Aqui ficam as listas de outras disciplinas (pasta "Outros Materiais": APA/UFF, MAC5711/USP lista 6, MC458/Unicamp lista 7), que servem de treino extra. Os valores numéricos foram conferidos por programa.

15.1 Listas extras: respostas e ideias

Exercício	Resposta / recorrência
APA 1–5	Backtracking (permutações, combinações, ciclo hamiltoniano, caixeiro viajante, damas). Fora do programa; 6 damas, primeira solução da busca linha a linha (colunas crescentes): $(2,4,6,1,3,5)$.
APA 6	LCS de dacdba e cadcda: 4 (acda ou dcda) — §11.1.
APA 7	Mochila 0-1, $W = 100$: 240, com os itens 1, 2 e 4 — §11.3.
APA 8	Kadane (PD em $O(n)$): $S(i) = \max\{a_i,\ S(i-1) + a_i\}$ é a melhor soma terminando em i; resposta $\max_i S(i)$. $5, 15, -30, 10, -5, 40, 10 \to$ **55** $(10, -5, 40, 10)$.
l6 1	Cadeia $\langle 5, 10, 3, 12, 5, 50, 6\rangle$: **2010**, $((A_1A_2)((A_3A_4)(A_5A_6)))$ — §11.2.
l6 2, 3, 4	Tabela s e MULTIPLICA; todas as LCS; maior subsequência crescente $O(n^2)$ e $O(n \log n)$ — §11.2, §11.1, §11.4.
l6 5	Número de ABBs com 6 chaves: Catalan $C_6 = \binom{12}{6}/7 =$ **132**.
l6 6–10	ABB ótima (Cormen §15.5, não dada em aula): $c(i,j) = \sum_{k=i}^{j} a_k + \min_{i \le r \le j}\{c(i, r-1) + c(r+1, j)\}$, $c(i, i-1) = 0$, em $\Theta(n^3)$ (mesma forma da cadeia de matrizes). Ex. 6: frequências $1, 1, 2, 3$: a raiz ótima é a chave 3 (custo 13; com a 4 na raiz, 14). Ex. 7: custo 185, raiz *if*. Ex. 8: some também os b: $w(i,j) = \sum_{i}^{j} a_k + \sum_{i-1}^{j} b_k$, base $c(i, i-1) = b_{i-1}$. Ex. 9 (Cormen 15.5-2): custo **312** (3,12 em probabilidades), raiz k_5.
l6 11	Subsequência crescente de soma máxima: $S[i] = a_i + \max\{S[j] : j < i,\ a_j < a_i\}$ (0 se vazio), $O(n^2)$; guarde $ant[i]$. Ex.: $1, 101, 2, 3, 100, 4, 5 \to 106$ $(1, 2, 3, 100)$.
l6 12 / MC458 8	Impressão nítida: $c(j) = \min_{i \le j,\ \text{cabe}}\{c(i-1) + \text{folga}(i,j)^3\}$ (na l6 a última linha custa 0; na MC458 a penalidade é o quadrado e inclui a última), $O(n^2)$.
l6 13	Festa da empresa (PD em árvore): $com(v) = s(v) + \sum_f sem(f)$; $sem(v) = \sum_f \max\{com(f), sem(f)\}$; resposta max na raiz. $\Theta(n)$ (pós-ordem).
l6 14 / MC458 10	Cortes (tora/string): §11.4, ex. 6. Tora de 10 m com cortes $2, 4, 7$: **20**.
l6 15	Torres gêmeas = LCS das duas sequências de raios.
l6 20	Bandeiras: $f(1) = f(2) = 2$ e $f(n) = f(n-1) + f(n-2)$: a última faixa é sempre B ou V; se a penúltima não é azul, cada configuração válida de $n-1$ faixas se estende de um único jeito; se é azul, cada uma de $n-2$ faixas se estende de um único jeito (A + a cor que falta). Valores $2, 2, 4, 6, 10, 16, \ldots$

MC458 3	Mochila exata (subset sum), pesos $2, 7, 5, 2$, $K = 0..14$; "O" = há solução sem o item, "I" = só usando o item:
	`p3=5:  O - O - - I - O - O - - I - I`
	`p4=2:  O - O - I O - O - O - I O - O`

MC458 4	Memória $O(K)$: um vetor, percorrendo K *decrescendo* — §11.3.
MC458 5	Mochila ilimitada — §11.4, ex. 5.
MC458 7	Submatriz de soma máxima: fixe o par de linhas (i_1, i_2) ($O(m^2)$ pares), some as colunas e rode Kadane ($O(n)$): $O(m^2 n)$.
MC458 9	Laje de mármore: $P(a,b) = \max\{p_{ab},\ \max_{a'<a} P(a',b) + P(a-a',b),\ \max_{b'<b} P(a,b') + P(a,b-b')\}$; $O(nm(n+m)) \subseteq O(n^2m^2)$.

15.2 Como estudar a parte nova

O formato da AV2 **ainda não foi anunciado** nos materiais disponíveis. Pelo estilo da AV1 e das listas, as questões mais prováveis são:

Tipos de questão prováveis (baseado nas listas e no estilo da AV1)

- **PD**: dado um problema novo, escrever subproblema, recorrência, pseudocódigo e complexidade; preencher uma tabela pequena (LCS, cadeia de matrizes, mochila) e reconstruir a solução.
- **Simulação**: rodar BFS, DFS (com PE/PS, tipos de arestas, *back*/*old*), Dijkstra, Floyd-

Warshall (matrizes D^k e P), Kruskal e Prim num grafo dado.

- **Provas**: correção de Kruskal e Prim (propriedade do corte), do guloso em matroides, de Dijkstra; verificar se um sistema de conjuntos é matroide (provar (I3) ou dar contraexemplo).
- **Escolha de implementação**: Prim/Dijkstra com matriz ($O(n^2)$) ou com heap ($O(m \log n)$), conforme a densidade; Union-Find no Kruskal.

Checklist da parte pós-AV1

1. **PD**: subestrutura ótima + sobreposição; as três recorrências (LCS, cadeia, mochila) de cor, com as ordens de preenchimento; por que a mochila é pseudo-polinomial.
2. **Floyd-Warshall**: significado de D^k, recorrência, k externo, matriz P (penúltimo vértice) e reconstrução.
3. **Dijkstra**: pseudocódigo, tabela de execução, prova, contraexemplo negativo, as três complexidades.
4. **Buscas**: pseudocódigos com PE/PS e com níveis; tipos de arestas e suas caracterizações; aplicações (componentes, ciclos, bipartição, distâncias, articulações, CFC, ordenação topológica).
5. **Gulosos e matroides**: axiomas; exemplos e não exemplos; teorema do guloso (com a prova por troca) e a recíproca.
6. **AGM**: propriedade do corte; Kruskal (Union-Find, $O(m \log n)$) e Prim (as quatro implementações); AGM de peso máximo.